ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

Máster en Ingeniería Biomédica y Salud Digital

Explicabilidad en sistemas de inteligencia artificial para la detección de tumores

Realizado por
José Andrés González

Dirigido por
Juan Pedro Domínguez Morales

Cotutora
Lourdes Durán López

Departamento
Arquitectura y Tecnología de Computadores

Sevilla, octubre 2024

Agradecimientos

Primeramente, me gustaría agradecer a todo el profesorado y el personal de la universidad, que han hecho que adquiera todos los conocimiento y habilidades para poder realizar el trabajo que está presente.
Especialmente me gustaría agradecer a Juan Pedro Domínguez, tutor de este TFM. Por la gran ayuda que me ha ofrecido para poder llevar lo mejor posible la elaboración de este.
Debo de agradecer a mis padres y familia por todo lo que me han dado para ser la persona de ahora, capaz de afrontar todos los retos que han surgido durante el camino.
Quisiera agradecer a todas las personas que han formado durante todos estos últimos años durante el recorrido de mis estudios, hasta llegar justo al momento de ahora. Se han convertido en una parte indispensable de mi vida y han hecho posible que pueda llegar hasta aquí. Gracias por escucharme, apoyarme en los momentos más complicados y compartir con ellos los mejores momentos por mi paso por la universidad.

Resumen

Con la elaboración de este trabajo de fin de máster se propone crear un procedimiento con el cual dar una explicación a los resultados que se obtienen de aplicar un sistema de clasificación, para predecir si una persona padece de la presencia de algún tumor.

La enfermedad del cáncer es una de las más dañinas, con un porcentaje cada vez más alto de la población en padecerla y que conlleva a estar sometidos a duros tratamientos. El diagnóstico precoz es de suma importancia ya que, esto puede cambiar totalmente el modo con el que tratar la enfermedad.

Debido a esto, nace como objetivo hacer un estudio con el que aportar algún tipo de herramienta que pueda mejorar en alguna medida a conseguir un diagnóstico que sea eficaz, precoz y sin ser muy costoso. Lo que beneficiaría directamente a la sociedad y a todo el personal de la salud, encargado en llevar este tipo de labores.

Por lo tanto, la idea es crear un sistema que, a partir de una imagen tomada de un paciente, sea capaz de predecir si contiene alguna presencia de un tumor maligno. Y, además, dar información sobre por qué a tomado dicho resultado, indicando qué zona de la imagen es en la que detectado dicho tumor.

Para la elaboración de estos mismo se ha empleado modelos de clasificación, usando redes neuronales convolucionales para trabajar con imágenes. El modelo será empleado a partir de un conjunto de datos con imágenes y cierta información extra de los nódulos, etiquetados correctamente para la construcción del modelo. A partir de la tener construidos los mismo, se aplicarán distintas técnicas de explicabilidad para ofrecer información relevante hacía la solución obtenida.

Con la creación de un prototipo completamente funcional con estas capacidades, se conseguiría dar cavidad al uso de tecnologías impulsada por IA en el ámbito sanitario, al aportar confianza al personal. Y mejorar como se propone en este caso, la eficacia en los diagnósticos del cáncer.

Abstracts

With the development of this master's thesis we propose to create a procedure with which to explain the results obtained by applying a classification system to predict whether a person suffers from the presence of a tumor.

Cancer is one of the most harmful diseases, with an increasingly higher percentage of the population suffering from it and being subjected to harsh treatments. Early diagnosis is of utmost importance because it can totally change the way the disease is treated.

Because of this, the objective of a study was born with the aim of providing some kind of tool that can improve to some extent to achieve a diagnosis that is effective, early and without being very expensive. This would directly benefit society and all health personnel in charge of carrying out this type of work.

Therefore, the idea is to create a system that, from an image taken of a patient, is able to predict whether it contains any presence of a malignant tumor. And, in addition, to give information about why it has taken such a result, indicating which area of the image is the one in which the tumor has been detected.

For the elaboration of these models, classification models have been used, using convolutional neural networks to work with images. The model will be used from a data set with images and some extra information of the nodes, correctly labeled for the construction of the model. After the model has been built, different explainability techniques will be applied to provide relevant information to the solution obtained.

With the creation of a fully functional prototype with these capabilities, the use of AI-driven technologies in the healthcare environment would be enabled by providing confidence to the staff. And improve, as proposed in this case, the efficiency of cancer diagnostics.

Índice

Agradecimientos **1**
Resumen **2**
Abstracts **3**
Índice **5**
Índice de figuras **6**
Índice de tablas **7**
1 INTRODUCCIÓN Y MOTIVACIÓN 9
2 OBJETIVOS DEL PROYECTO 15
3 ESTADO DEL ARTE 19
4 ELICITACIÓN DE REQUISITOS Y ANÁLISIS DE 23
4.1 REQUISITOS 23
4.2 ANÁLISIS DE RIESGOS 27
5 DISEÑO DEL SISTEMA 32
6 IMPLEMENTACIÓN 34
6.1 TECNOLOGÍAS EMPLEADAS 34
6.2 Estudio Previo 44
6.3 DESARROLLO 66
8 PLANIFICACIÓN TEMPORAL INICIAL 87
9 PLANIFICACIÓN TEMPORAL FINAL 91
10 PLANIFICACIÓN DE COSTES INICIALES 92
10 PLANIFICACIÓN DE COSTES FINAL 94
11 ESTUDIO DE MERCADO 95
Conclusiones **99**
Trabajo Futuro **100**
Bibliografía **102**
Anexo ..... Error! Bookmark not defined.

Índice de figuras

Figura 1Gráfico de distribución de muertes por tipos de cáncer en el mundo 11
Figura 2 Evolución de nuevos casos positivos y muertes por año 12
Figura 3 Muestra de imagen médica computarizada con la presencia de un nódulo 13
Figura 4 Resonancia magnética..... 19
Figura 5 Biopsia con aguja..... 21
Figura 6 Matriz de riesgos..... 30
Figura 7 Diagrama del diseño del sistema..... 32
Figura 8 Esquema del funcionamiento de Python..... 34
Figura 9 Esquema de abstracción de creación de una red neuronal 38
Figura 10 Diagrama de trabajo de ultralytics 40
Figura 11 Logotipo del entorno de google colab 42
Figura 12 Aspecto de una red neuronal..... 44
Figura 13 Suma de las ponderaciones de una neurona..... 44
Figura 14 Funciones de activación..... 45
Figura 15 Cálculo del error cuadrático..... 46
Figura 16 Gráfico de la evolución del error durante el entrenamiento de la red..... 47
Figura 17 Esquema de funcionamiento de un árbol de decisión..... 53
Figura 18 Diagrama de funcionamiento de Grad-CAM..... 60
Figura 19 Comparación entre los distintos métodos de Grad-CAM 61
Figura 20 Proceso de detección de objetos por YOLO 63
Figura 21 Arquitectura de la red de YOLO..... 64
Figura 22 Esquema de las anotaciones de un archivo de YOLO 67
Figura 23 Esquema del archivo yml de YOLO..... 69
Figura 24 Gráfico de evolución de entrenamiento de la red secuencial..... 73
Figura 25 Matriz de confusión del modelo secuencial al set de train 74
Figura 26 Matriz de confusión del modelo secuencial al set de test 74
Figura 27 Gráfico de evolución del entrenamiento del modelo Xception..... 75
Figura 28 Matriz de confusión del modelo Xception al set de train 76
Figura 29 Matriz de confusión del modelo Xception al set de test 76
Figura 30 Gráfico de evolución del entrenamiento del modelo VGG16..... 77
Figura 31 Matriz de confusión del modelo VGG16 al set de train 78
Figura 32 Matriz de confusión del modelo VGG16 al set de test 78
Figura 33 Aplicación de Grad-CAM sobre una imagen..... 80
Figura 34 Evaluación de los resultados de evaluación de YOLO 81
Figura 35 Imagen del conjunto de test con el resultado de la detección de YOLO 82
Figura 36 Imagen del conjunto test 82
Figura 37 Matriz de confusión del conjunto de test con el modelo de YOLO..... 82
Figura 38 Resultado de aplicar Grad-CAM a VGG16 83
Figura 39 Resultado de aplicar Grad-CAM a Xception 83
Figura 40 Diagrama de Gantt de planificación inicial 90
Figura 41 Diagrama de Gantt de planificación final 91
Figura 42 Impacto del cáncer por porcentaje de pacientes distribuidos en cada entidad del sector sanitario 96

Índice de tablas

Tabla 1 Valores de la probabilidad de un incidente 27
Tabla 2 Valores del impacto de una incidencia 27
Tabla 3 Riesgos posibles que ocurran durante el desarrollo 28
Tabla 4 Riesgos posibles que ocurran durante el desarrollo II 29
Tabla 5 Representación de cada riesgo en la matriz 30
Tabla 6 Estructura de una matriz de confusión 50
Tabla 7 Comparación entre distintos modelos de detección de objetos 65
Tabla 8 Métricas del modelo secuencial 73
Tabla 9 Métricas del modelo Xception 76
Tabla 10 Métricas del modelo VGG16 78
Tabla 11 Distribución de los costes iniciales 92
Tabla 12 Precio medio por profesión 93

1 INTRODUCCIÓN Y MOTIVACIÓN

Con el desarrollo realizado en el proyecto que se presenta, se pretende llevar a cabo una investigación de los resultados obtenidos por medio de algunos algoritmos de deep learning para la clasificación de imágenes médicas. Con la finalidad de poder posteriormente centrarse en comprender y dar una explicación al modo en el que la IA ha conseguido hallar o determinar dicha clasificación.

La ciencia y la tecnología en los últimos años ha tenido un impacto de gran relevancia y siendo completamente transformador en el ámbito de la medicina, creciendo ésta en multitud de aspectos. Así como; mejorando diagnósticos a través de técnicas de imagen y biomarcadores, ofreciendo tratamientos innovadores como terapias genéticas, nanotecnología y medicina personalizada.

La ingeniería ha generado numerosas mejoras en dicho ámbito, mediante la creación de dispositivos que son de gran utilidad para el personal médico como; las máquinas de ECG, unidades electroquirúrgicas, desfibriladores y monitores de pacientes. Estos equipos son esenciales en los hospitales, donde se encuentran por doquier, facilitando el trabajo diario de los médicos. Así procesos complicados como la cirugía han avanzado con la robótica y técnicas mínimamente invasivas.

Con la llegada de la era digital, la informática ha adquirido un rol crucial en la medicina. Hoy en día, existen estudios especializados en sus aplicaciones en el sector de la salud, como la bioinformática e informática clínica. La informática se utiliza para agilizar sistemas administrativos, desarrollar maquinaria médica y apoyar la investigación científica. Además, dispositivos de monitoreo y telemedicina que mejoran la gestión de enfermedades crónicas, vacunas avanzadas junto con apps de salud que promueven la prevención, etc.
La inteligencia artificial es un campo de la informática que ha experimentado un crecimiento notable en los últimos años y sigue en pleno desarrollo. Este avance ha sido impulsado por matemáticos e informáticos que, a través de numerosos estudios, han mejorado las técnicas empleadas, con el respaldo tanto del mundo científico como del sector empresarial. La razón de su auge es el descubrimiento de múltiples aplicaciones que permiten resolver una gran variedad de problemas.
Además, el progreso tecnológico y las capacidades de los supercomputadores actuales han permitido que el uso de la inteligencia artificial se expanda de manera exponencial. La investigación en el campo de la salud es uno de los ámbitos donde se aplica con mayor relevancia, donde se busca enfrentar y resolver los complejos problemas que hemos mencionado previamente.

Grupos de investigación emplean las capacidades computacionales para realizar experimentos y analizar grandes volúmenes de datos. Esto avances en inteligencia artificial, big data y telecomunicaciones han permitido realizar estudios para descubrir tratamientos para enfermedades actuales o predecir la aparición de patologías, entre otras iniciativas en desarrollo como la bioimpresión y la medicina regenerativa, que abren nuevas posibilidades en la regeneración de órganos. Todos estos progresos, sin embargo, también plantean desafíos éticos y regulatorios que requieren atención constante.

El desarrollo de nuevas tecnologías actualmente sigue con una alta curva creciente en cuanto a su estudio e investigación. Parte de ello debido a la complejidad que presenta el cuerpo humano, que aún se presenta como uno de los mayores desafíos de la actualidad. El funcionamiento del cerebro, las emociones, la inteligencia, los pensamientos o los recuerdos; quedan muy alejados de nuestro entendimiento.

Existen infinidad de enfermedades, como son las infecciosas o parasitarias, las enfermedades endocrinas, del sistema inmunológico, mentales, tumores malignos, entro otras muchas. Entre estas enfermedades el cáncer es de las más relevantes y va a ser la analizada durante el desarrollo de esta investigación.

Y esto se presenta como uno de los mayores retos actuales, ya que, siguen apareciendo, evolucionando o mutando, lo que dificulta su tratamiento. Algunas, como el cáncer, siguen siendo especialmente difíciles de erradicar, ya que aún no se ha encontrado una cura efectiva y sencilla para eliminarlas por completo. Y va a ser la analizada durante el desarrollo de esta investigación.

El cáncer es una de las enfermedades más complejas y desafiantes a nivel mundial. A nivel general, el cáncer se refiere a un grupo de enfermedades caracterizadas por el crecimiento descontrolado de células anormales en el cuerpo, que pueden invadir y dañar los tejidos circundantes. Existen más de 100 tipos diferentes de cáncer, que pueden afectar prácticamente cualquier parte del cuerpo, siendo algunos de los más comunes el cáncer de mama, pulmón, colon, próstata, y piel. (Sociedad Estadounidense del Cáncer, s.f.)

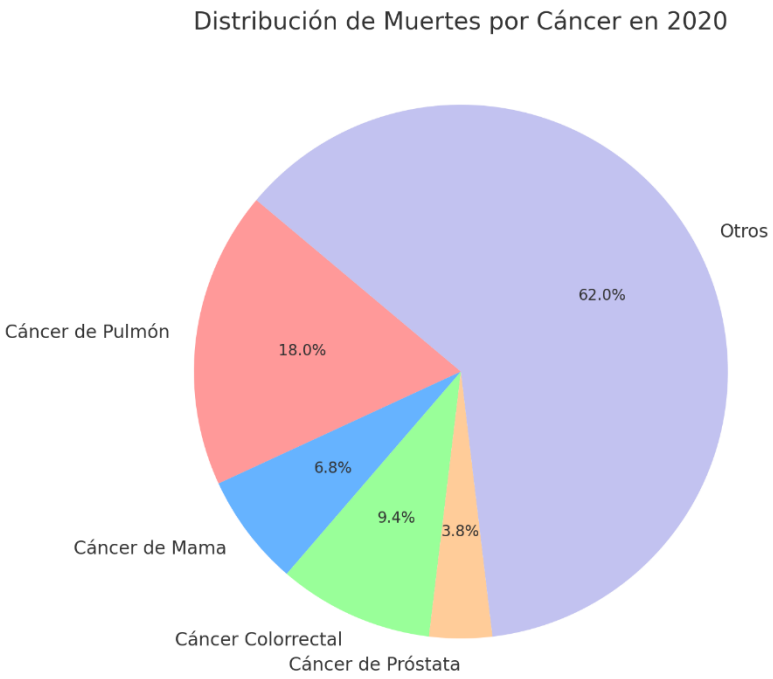

Figura 1: Gráfico de distribución de muertes por tipos de cáncer en el mundo

En la actualidad, el cáncer sigue siendo una de las principales causas de muerte en todo el mundo. A pesar de los avances en la detección temprana, tratamiento y prevención, la incidencia de cáncer ha aumentado debido a factores como el envejecimiento de la población, el estilo de vida, y la exposición a factores de riesgo como el tabaquismo, la mala alimentación, la falta de actividad física, y la exposición a sustancias cancerígenas.
El desarrollo de tratamientos más efectivos, como la inmunoterapia, la terapia dirigida, y las nuevas técnicas de radioterapia, ha mejorado significativamente las tasas de supervivencia en algunos tipos de cáncer. Sin embargo, todavía existen muchos tipos de cáncer que son difíciles de tratar, especialmente en etapas avanzadas, y en muchos casos, el cáncer se presenta con recurrencias después del tratamiento inicial.

A nivel global, el cáncer afecta a millones de personas. Según datos de la Organización Mundial de la Salud (OMS), en 2020 se registraron alrededor de 19.3 millones de nuevos casos de cáncer en todo el mundo y casi 10 millones de muertes relacionadas con la enfermedad. El cáncer de pulmón, el cáncer de mama, y el cáncer colorrectal fueron los más diagnosticados. Se espera que estas cifras sigan aumentando en los próximos años, con estimaciones que indican que el número de nuevos casos anuales podría superar los 29 millones para el año 2040.
El cáncer no afecta a todas las regiones del mundo por igual. Los países con ingresos altos suelen tener mayores tasas de incidencia debido a factores como el envejecimiento de la población y la detección temprana, mientras que, en los países de ingresos bajos y medios, el acceso limitado a servicios de salud y diagnósticos oportunos conduce a tasas más altas de mortalidad.

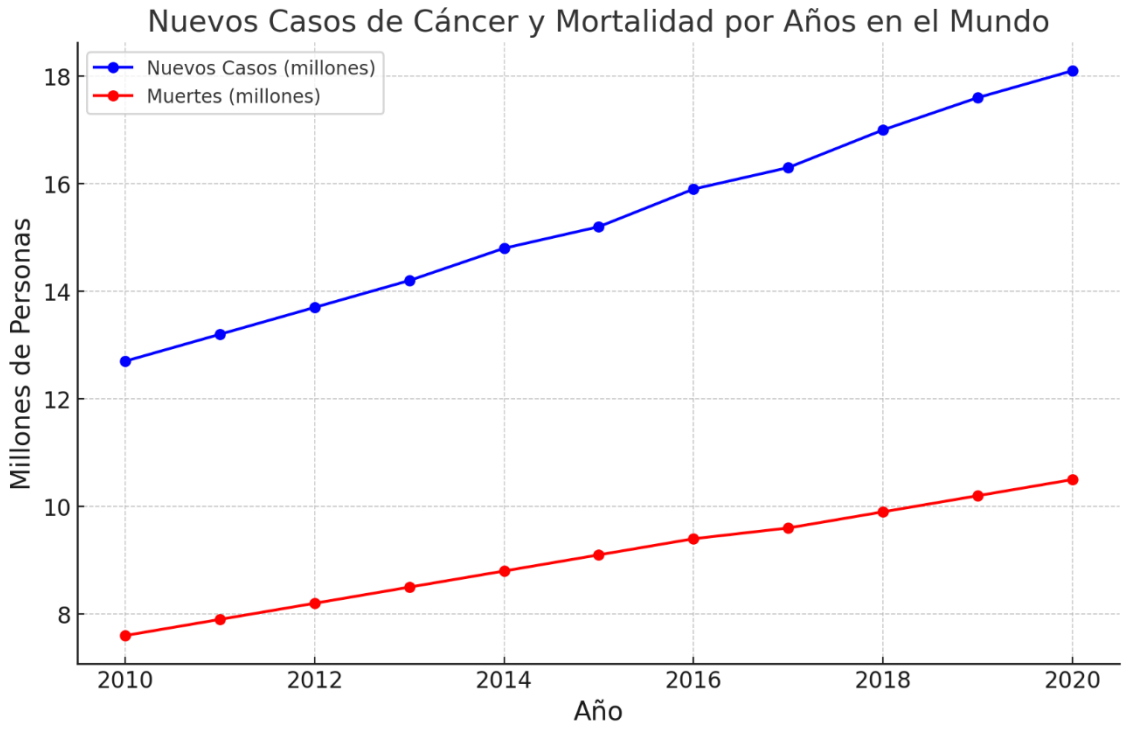

Figura 2: Evolución de nuevos casos positivos y muertes por año

El cáncer tiene un enorme impacto en la sociedad, tanto a nivel emocional como económico. Las personas afectadas por el cáncer, así como sus familias, suelen enfrentar grandes desafíos emocionales, psicológicos y financieros. El tratamiento del cáncer es costoso y a menudo prolongado, lo que genera un alto gasto en atención médica tanto para los individuos como para los sistemas de salud. Además, la pérdida de productividad debido a la enfermedad y el tratamiento puede afectar tanto a los pacientes como a la economía en general.
Además del impacto directo en los pacientes, el cáncer también tiene un efecto en la salud pública y en la planificación de los recursos de salud. Las campañas de concienciación, la investigación, y los programas de detección temprana son esenciales para reducir la carga de esta enfermedad en la sociedad.
Es importante destacar que la prevención es clave en la lucha contra el cáncer. Adoptar un estilo de vida saludable, evitar el tabaquismo, reducir el consumo de alcohol, y protegerse contra las infecciones que pueden causar cáncer (como el virus del papiloma humano y la hepatitis B) son medidas que pueden reducir significativamente el riesgo de desarrollar esta enfermedad.

Dado el gran impacto que tiene en la sociedad en la actualidad, ofrecer nuevas vías para poder mejorar las técnicas actuales es indispensable. Aprovechar tecnologías como la inteligencia artificial, pueden suponer un gran apoyo.
En concreto, en los últimos años se han iniciado muchos experimentos con algoritmos de clasificación de imágenes, empleando redes neuronales

convolucionales (CNN). Así analizando una gran variedad de imágenes médicas como radiografías, tomografías computarizadas o resonancias magnéticas. Para identificar aquellos patrones asociados con un tumor u otras anomalías de una forma rápida y precisa.

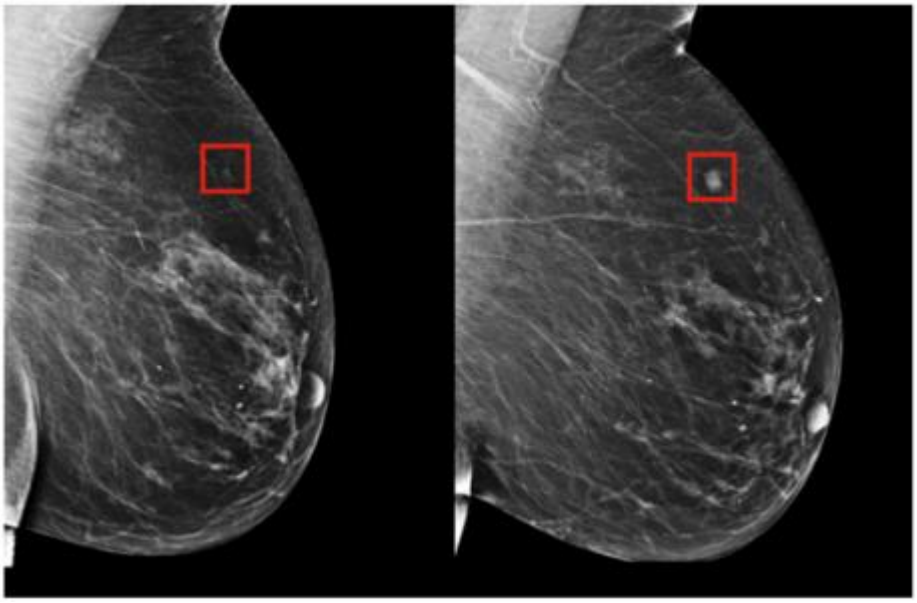

Figura 3: Muestra de imagen médica computarizada con la presencia de un nódulo

Estos sistemas ya han sido probados para analizar mamografías y poder detectar signos tempranos de cáncer de mama. Consiguiendo una mejora en la tasa de detección y reducir el número de falsos positivos.
También se han empleados estos sistemas para la clasificación de imágenes de lesiones cutáneas y conseguir determinar si son benignas o malignas, ayudando así a la detección de melanomas.
Otro claro uso en la actualidad ha sido analizar tomografías computarizadas de pulmones e identificar aquellos nódulos que podrían ser cancerosos.

A pesar de los avances, aún existen desafíos en la implementación de estas tecnologías en la práctica médica. Como la necesidad de datos de alta calidad y bien etiquetados, la interpretación de resultados complejos y la integración con los sistemas de salud existentes son algunos de los obstáculos. Sin embargo, a medida que se desarrollan y mejoran los algoritmos, se espera que la IA se convierta en una herramienta estándar en la detección y diagnóstico de tumores y otras enfermedades.

Uno de los principales desafíos en el uso de la inteligencia artificial (IA), especialmente en aplicaciones críticas como la detección de enfermedades, es la interpretabilidad y la comprensión del funcionamiento de los modelos. Este problema se conoce comúnmente como el problema de la "caja negra" en IA, ya que los modelos, en particular los basados en *deep learning*, pueden hacer

predicciones precisas, pero no siempre es claro cómo o por qué llegaron a esas conclusiones.

Estos problemas vienen derivados por diversos motivos como son la complejidad de los modelos, la falta de transparencia, el riesgo de errores o el sesgo y la generalización.

En medicina, la interpretabilidad es especialmente crítica porque los médicos necesitan confiar en las herramientas que utilizan. Un diagnóstico erróneo o una falta de explicación clara pueden tener graves consecuencias para el paciente. Por ejemplo, si una IA clasifica incorrectamente un tumor como benigno, sin una comprensión clara de por qué ocurrió el error, los médicos no pueden corregir el diagnóstico o mejorar el modelo.
Además, la capacidad de explicar las decisiones de la IA es importante para la aceptación del sistema por parte de los profesionales de la salud. Los médicos necesitan comprender cómo la IA llega a sus conclusiones para sentirse seguros de usarla en su práctica diaria y combinarla de manera efectiva con su propio juicio clínico.

Debido a este incentivo, se procede a llevar a cabo la investigación. Con la que se pretende poder realizar diversas experimentaciones con distintas técnicas y analizar los resultados obtenidos por cada una de ellas, con la finalidad de poder conocer con cual se consiguen unos mejores resultados. Aplicándolos a un caso de clasificación de imágenes médicas para detectar la presencia de un tumor.

2 OBJETIVOS DEL PROYECTO

El objetivo principal del proyecto va a consistir en conseguir un método, que sea con un alto nivel de exhaustividad, que nos permita comprender los resultados obtenidos de un sistema clasificador que determina la existencia de un tumor a partir de una imagen.

Con la finalidad de poder cumplir con dicha propuesta, se ha empleado una subdivisión en pequeños objetivos agrupados en dos categorías:

2.1 OBJETIVOS A NIVEL PROFESIONAL

- **Mejorar el panorama actual.** Las enfermedades cancerígenas se esperan que aumenten su presencia con el paso de los años. Avances tecnológicos en esta dirección pueden intentar mejorar la situación, para suavizar la curva de crecimiento.

- **Dar apoyo y soporte a los médicos.** Hacer sentir a los médicos con una mayor confianza para fiarse del sistema, además de poder con ello tomar unas decisiones más adecuadas y precisas. Rompiendo así la barrera de aceptación por parte del personal médico.

- **Agilizar el proceso de determinación del tumor maligno.** Se pretende obtener una mayor velocidad para poder dar un resultado concluyente, ya que el doctor podría analizar el resultado con mucha más información y así obtener una solución al paciente en un tiempo mucho menor.

- **Aminorar la posibilidad de un diagnóstico erróneo.** Reducir la posibilidad tanto a que se dé un diagnóstico falso positivo, el cual conlleva a un mayor gasto en los recursos sanitarios, como un falso negativo que implicaría un tratamiento tardío de la enfermedad, que en el caso del cáncer es crítico. Pudiendo poner en gran riesgo la vida del paciente.

- **Reducir los costes en recursos hospitalarios.** Con un menor número de pruebas, empleando menos equipamiento y con menos personal; se podría obtener los mismos resultados. Así disminuyendo tiempo de esperas de los pacientes, pudiendo diagnosticar más en un

menor tiempo y optimizando mejor todos los recursos.

- **Una expansión global.** Al no requerir de unos grandes recursos para poder ser implementada y utilizada la tecnología, permite que de un modo sencillo pueda ser empleada en casi cualquier lugar a pesar de que no sea un centro médico de grandes dimensiones o recursos.

- **Educación al personal sanitario.** Puede ser útil como apoyo para el aprendizaje de los sanitarios en formación. Siendo práctico para conocer aquellos detalles importantes a los que prestar atención para determinar la existencia de un tumor. Y que sean capaces de adquirir la destreza de detectar con una mayor exactitud y menor tiempo la posibilidad de que el paciente padezca algún tipo de tumor.

- **Mayor aceptación.** Dar una mayor confianza al uso de tecnologías impulsadas por inteligencia artificial para la mejora de los servicios sanitarios, tanto por parte de todo el personal sanitario como por los propios pacientes. Permitiendo el uso y la implementación de estas paulatinamente, consiguiendo todas las mejoras que conllevaría, siendo sí importante llevar una serie de medidas y estandarizaciones regulatorias para su correcta aplicación.

- **Impulso a la investigación.** Este tipo de metodología puede ser abordado posteriormente en un amplio abanico dentro de las investigaciones médicas. Siendo de apoyo para conseguir mejorar los resultados que se obtienen y poder comprender en mejor medida dichas soluciones obtenidas. Así como ofrecer nuevas medidas para abordar la lucha contra las enfermedades o la agilización del diagnóstico de estas.

2.2 OBJETIVOS A NIVEL EDUCATIVO

- **Redes neuronales.** Conocer algunos detalles del funcionamiento de las redes y como implementarlas. Para ello usando algunas librerías como TensorFlow, altamente empleadas y con un gran soporte. Además de analizar la variedad existente de las mismas y su desempeño a las que están destinadas.

- **Redes Convolucionales (CNN).** Aprender las características de este tipo de redes en específico y emplear las tecnologías necesarias para implementarlas y usarlas para la clasificación de imágenes.

- **Algoritmos de detección de objetos.** Descubrir el funcionamiento de esta tecnología, que permite dentro de imágenes o vídeos, incluso para algunos casos en tiempo real, la detección de objetos. Y hacer uso de ella adaptándola a la problemática del problema que se pretende solucionar.

- **Imágenes médicas.** Investigar las características principales de este tipo de imágenes, para poderlas emplear en la experimentación y obtener unos resultados concluyentes.

- **Examinación de los datos.** Hacer un estudio previo y analítico para valorar que conjunto de datos puede ser el más enriquecedor a emplear para hacer las pruebas. Puesto que la elección de este puede determinar en gran medida el desempeño de la experimentación y la conclusión a la que se pueda llegar.

- **Procesamiento de los datos.** Preparar y realizar las operaciones necesarias sobre el conjunto de datos, imágenes, para que puedan ser empleados por los modelos de CNN sin que genere problemas y con los que se consiga obtener unos mejores resultados.

- **Conocimientos de la enfermedad.** Hallar una extensa información con respecto al cáncer como problemática a la que se quiere presentar esta nueva investigación. Para así, poder tener los conocimientos necesarios para hablar del tema con la propiedad requerida en cualquier explicación dada durante la investigación.

- **Bioinformática.** Aplicar de manera adecuada las metodologías y directrices necesarias, propias de la aplicación de la informática al campo de la medicina para desempeñar correctamente la labor de investigación.

- **Analista.** De manera exhaustiva, analizar en cada caso las técnicas empleadas para cada requerimiento del proyecto y tomar las decisiones más efectivas. Además, evaluar dentro de las distintas posibilidades y con los resultados obtenidos por los distintos experimentos, cual es la solución que se presenta óptima acorde a los distintos parámetros a tener en cuenta

- **Planificación.** Plantear una estructura organizada en cuanto a las distintas actividades y partes a desempeñar. Para así conseguir cumplir con cada una de las tareas de una forma más coherente y siendo más consciente de los tiempos dedicados, pudiendo actuar en respuesta según los márgenes establecidos para cada una.

- **Documentación científica.** Exponer en el presente documento, toda la información de la forma más clara y divulgativa. A modo que quede constancia de la conclusión con el proceso realizado tras la investigación.

- **Investigar.** Extraer la información de los medios más fiables, con la veracidad de que tenga validez para poder ser empleada de manera objetiva. Y hacer uso de los criterios y directrices propias que deben llevarse a cabo en una investigación.

- **Explicabilidad.** Exponer el comportamiento de las redes neuronales ante los resultados que ofrecen. Para así comprender en cierta manera la información relevante que toman en cuenta para llevar a cabo la decisión.

3 ESTADO DEL ARTE

La detección de un tumor maligno se hace en la actualidad con la combinación de diversas técnicas, consiguiendo obtener unos resultados más efectivos con aquellas que combinan precisión diagnóstica, accesibilidad y seguridad del paciente.

Pruebas de imagen

El objetivo es visualizar estructuras internas y detectar anomalías que podrían indicar la presencia de un tumor maligno. Son técnicas no invasivas que utilizan diferentes tipos de energía, como rayos X, ondas de sonido o campos magnéticos, para crear imágenes detalladas del interior del cuerpo. Estas imágenes permiten a los médicos identificar masas anormales, determinar su tamaño y localización, y evaluar si hay evidencia de invasión a tejidos circundantes o metástasis a otras partes del cuerpo. (Mayo Clinic, s.f.)

- **Mamografía**: Es el estándar de oro en la detección temprana del cáncer de mama y se utiliza ampliamente como herramienta de cribado. Es efectiva para detectar pequeños tumores antes de que sean palpables.
- **Tomografía computarizada (TC)**: Frecuentemente utilizada en la detección de cánceres como el de pulmón, hígado, páncreas y otros órganos internos. Es especialmente efectiva para detectar tumores y evaluar su tamaño y extensión.
- **Resonancia magnética (RM)**: Se emplea a menudo en el cáncer de cerebro, médula espinal, y para caracterizar mejor los tumores en tejidos blandos, como en el cáncer de mama y próstata. Su capacidad de ofrecer imágenes detalladas sin radiación la hace una opción muy efectiva, especialmente para áreas complejas.
- **Ecografía**: Común en la detección de cánceres como el de tiroides, hígado, ovarios y testículos. Es una técnica no invasiva y sin radiación, lo que la hace muy utilizada para estudios iniciales.
- **PET-CT**: Es particularmente efectiva en la detección de cánceres metastásicos o en la evaluación de la respuesta al tratamiento. Combina la tomografía por emisión de positrones con la tomografía computarizada, permitiendo localizar y evaluar la actividad metabólica del tumor.

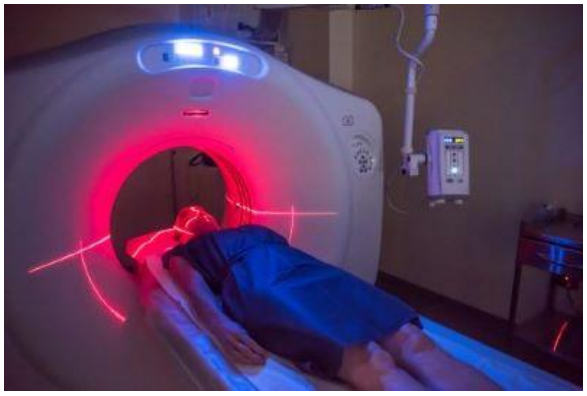

Figura 4: Resonancia magnética

Estudios de laboratorio

Consiste en detectar sustancias en el cuerpo que puedan indicar la presencia de cáncer y evaluar la función orgánica. Implican el análisis de muestras biológicas, como sangre, orina, o tejidos, para detectar marcadores específicos asociados con el cáncer. Estos marcadores pueden ser proteínas, hormonas, o fragmentos de ADN que las células cancerosas liberan al torrente sanguíneo. Los resultados de estas pruebas pueden indicar la presencia de cáncer o ayudar a monitorear la respuesta al tratamiento. (Sociedad Española de Oncología Médica, s.f.)

- **Análisis de sangre para marcadores tumorales**: Aunque no son diagnósticos por sí mismos, los marcadores tumorales como el PSA (en cáncer de próstata), CA-125 (en cáncer de ovario) y CEA (en cáncer colorrectal) son pruebas recurrentes en el seguimiento y monitoreo de ciertos cánceres. Son útiles para la detección temprana en pacientes de alto riesgo o para evaluar la efectividad del tratamiento.
- **Pruebas genéticas**: Se están utilizando cada vez más para detectar mutaciones que predisponen a ciertos cánceres, como las mutaciones BRCA en cáncer de mama y ovario. Son particularmente efectivas en la identificación de individuos de alto riesgo y en la orientación de tratamientos específicos.

Biopsia

Sirve para obtener una muestra de tejido para un análisis directo y confirmar la presencia de células cancerosas. Con este procedimiento se extrae una pequeña muestra de tejido sospechoso para su examen bajo el microscopio. Esto permite a los patólogos determinar si el tejido es benigno o maligno. Existen varios métodos para realizar una biopsia, dependiendo de la localización y el tamaño del tumor.

- **Biopsia con aguja**: Es una de las técnicas más comunes para confirmar un diagnóstico de cáncer. Este procedimiento es mínimamente invasivo y proporciona una muestra de tejido que permite determinar la malignidad de una lesión sospechosa.
- **Biopsia líquida**: Aunque relativamente nueva, está ganando popularidad por su capacidad de detectar ADN tumoral en la sangre. Es menos invasiva que una biopsia tradicional y se utiliza cada vez más para monitorear la respuesta al tratamiento o detectar recaídas.
- **Biopsia por escisión**: Es común en casos donde se requiere un diagnóstico definitivo o cuando el tumor es accesible y su tamaño o ubicación permite una extirpación completa.

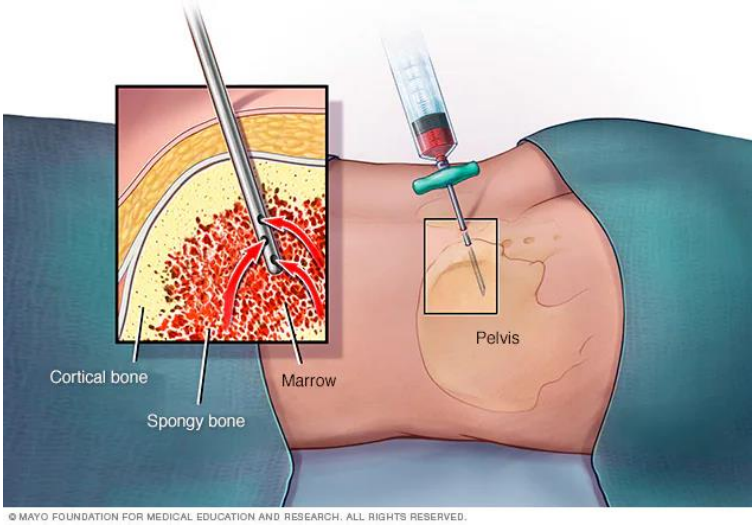

Figura 5: Biopsia con aguja

Técnicas de tamizaje o cribado

Detectan el cáncer en etapas tempranas en personas asintomáticas. Implican la realización de pruebas en personas que no presentan síntomas, con el objetivo de detectar el cáncer en sus etapas iniciales. Estas pruebas están diseñadas para encontrar signos tempranos de cáncer, cuando es más tratable y curable.

- **Papanicolaou (PAP)**: Es una de las pruebas de cribado más exitosas y recurrentes en la prevención del cáncer de cuello uterino. Su efectividad ha llevado a una disminución significativa en la mortalidad por este tipo de cáncer.
- **Colonoscopia**: Es altamente efectiva para detectar cáncer colorrectal en etapas tempranas y para eliminar pólipos precancerosos, lo que la convierte en una técnica preventiva muy recurrente y valiosa.
- **Prueba de sangre oculta en heces (SOH)**: Se utiliza ampliamente como prueba de cribado para el cáncer de colon y es efectiva cuando se combina con otras pruebas como la colonoscopia.

Técnicas moleculares y genéticas

Se usa para identificar mutaciones genéticas y alteraciones moleculares que puedan indicar la presencia de cáncer o guiar el tratamiento personalizado. Estas técnicas analizan el material genético de las células (ADN o ARN) para identificar mutaciones o alteraciones específicas que están asociadas con ciertos tipos de cáncer. También pueden incluir la evaluación de proteínas específicas en las células tumorales para comprender mejor el tipo de cáncer y su agresividad.

- **Inmunohistoquímica (IHC)**: Es una técnica recurrente en la clasificación de cánceres y en la determinación de la agresividad del tumor. Es muy efectiva para guiar decisiones terapéuticas, como la elección de terapias dirigidas.
- **Secuenciación de ADN**: Se está volviendo más común en el manejo de

cánceres avanzados o difíciles de tratar, como el cáncer de pulmón de células no pequeñas o el melanoma. Es extremadamente efectiva para personalizar el tratamiento y mejorar los resultados del paciente.

Técnicas emergentes

Se investiga y se explora con nuevas formas de detectar el cáncer de manera más temprana o menos invasiva. Se trabaja en diversos enfoques innovadores que están en desarrollo o se están utilizando de manera experimental para la detección de cáncer. Estas técnicas pueden involucrar el análisis de nuevas muestras biológicas (como el aliento) o la utilización de tecnologías avanzadas como la nanotecnología para detectar el cáncer en etapas muy tempranas.

Cada una de estas técnicas tiene su propio enfoque y objetivos específicos. Mientras que las pruebas de imagen permiten observar el interior del cuerpo, los estudios de laboratorio detectan indicadores químicos o biológicos del cáncer, las biopsias confirman la presencia de células malignas, y las técnicas moleculares ofrecen un análisis genético más profundo. Las pruebas de cribado están destinadas a detectar el cáncer de manera preventiva en poblaciones aparentemente sanas.

También existen diversas investigaciones y estudios sobre lo propuesto en este mismo trabajo. Hacer uso de la inteligencia artificial como elemento clave para avanzar en mejoras del diagnóstico y el tratamiento del paciente, pero a su vez poder otorgar la mayor trasparencia posible al respecto, para que pueda ser empleada por los centros sanitarios con total confianza. Además de la inversión por empresas del sector interesadas en ser las primeras en desarrollar una herramienta con estas capacidades al mercado. (XAI in medical image analysis, s.f.)
Las pruebas de imagen, como la mamografía, la tomografía computarizada (TC), y la resonancia magnética (RM), son muy recurrentes debido a su capacidad para detectar cánceres en diversas etapas y su amplia disponibilidad. En cuanto a las biopsias, la biopsia con aguja es la más común y efectiva para confirmar un diagnóstico. Las pruebas de laboratorio, como los marcadores tumorales y las pruebas genéticas, también juegan un papel crucial, especialmente en la detección temprana y el seguimiento del cáncer.

4 ELICITACIÓN DE REQUISITOS Y ANÁLISIS DE RIESGOS

4.1 REQUISITOS

En este apartado se va a tratar de todos los requisitos que deberá de cumplir el sistema a distintos niveles, haciendo un desglose de ellos. Con el fin de que sean satisfechos los objetivos preestablecidos.

4.1.1 Requisitos técnicos

Están relacionados con la infraestructura tecnológica necesaria para que el sistema o proyecto funcione. Esto incluye hardware, software, redes, bases de datos y otros que serán utilizados. También pueden involucrar aspectos de seguridad o desempeño del sistema.

- Se debe de tener un control de la calidad de los datos a usar, para poder realizar un buen sistema de entrenamiento. Además de tener creada una estructuración específica para poder después manipular los datos correctamente. El sistema ha de usar una cantidad considerable de datos y tienen que estar etiquetados correctamente, ya que sin esta información impide que se pueda crear los modelos especialmente para la detección de objetos. Y también tienen que estar divididos en grupos según algún criterio riguroso a usar.

- Para que el sistema haga un entrenamiento correcto las imágenes que se emplean deben de ser procesadas para que puedan conseguirse unos modelos que ofrezcan soluciones óptimas.

- La solución debe de estar implementada con un modelo fiable con el cual, tras todo el proceso de entrenamiento, haber hecho el test y usar pruebas aleatorias de muestras dadas, consiga obtener un porcentaje de acierto alto. El modelo será escogido tras la comparación entre varias métricas, usando distintas arquitecturas de redes neuronales para las construcciones del modelo. Estas redes serán implementadas dando un diseño de su arquitectura o empleando alguna existente ofrecida por algunas de las librerías.

- Para abordar lo propuesto con respecto a ofrecer un entendimiento a los resultados que ofrecen los modelos previamente construidos. Se debe de aplicar con rigor las técnicas de explicabilidad con las distintas metodologías disponibles. De forma que, puedan ser adecuadas para poder emplearse con los modelos utilizados y el conjunto de datos seleccionado.

- La evaluación hecha sobre la calidad de la explicabilidad, obtenida por los distintos métodos, debe de ser realizada siguiendo una serie de métricas.

Teniendo que operar sobre los resultados conseguidos, para así posteriormente hacer una comparación y determinar con cual se ha logrado llegar a una mejor solución.

4.1.2 Requisitos funcionales

Describen las funciones y comportamientos específicos que debe realizar el sistema o la aplicación. Detallan qué debe hacer el sistema desde el punto de vista del usuario, como, por ejemplo; las tareas, procesos y operaciones que debe ejecutar.

- El producto debe de ser capaz de usarse indefinidamente, sin necesidad de tener que reiniciarlo cada vez que vaya a utilizarse para un nuevo diagnóstico, ni tampoco tener que realizar ninguna configuración adicional. De forma que pueda prestarse el servicio constantemente.

- El sistema debe de ser capaz de dar una solución, dentro de lo posible, lo más viable a pesar de la calidad de la imagen ofrecida para su detección. O en caso de no ser así, proveer una solución en base a esta petición, pero informando de que consta con una menor precisión y por lo tanto puede ser errónea. Dejando por parte del personal médico determinar la decisión a tomar.

- El tiempo de respuesta desde la percepción de la imagen, hasta la obtención de la clasificación y determinar el objeto detectado, en caso de que fuese un resultado de clasificación positivo, debe de ser lo más rápida posible sin repercutir en la validez del resultado obtenido.

- Los resultados ofrecidos por el sistema deben presentarse con la información clara y detallada, para que el sanitario encargado de dar el informe pueda dar una respuesta con la mayor rigurosidad posible.

4.1.3 Requisitos de Interfaz

Son las condiciones que deben cumplir las interfaces del sistema, tanto para el usuario (interfaz gráfica, usabilidad, accesibilidad) como para otras aplicaciones o sistemas (APIs, integración, formatos de datos). Estos requisitos determinan cómo interactuarán los usuarios o sistemas con el proyecto.

- La interfaz debe de ser clara y sencilla para que el sistema pueda ser emplead sin la necesidad de tener que dedicar mucho tiempo a conocer el funcionamiento de esta o recibir una profunda formación. De manera que se presente como una solución lo más intuitiva posible.

- Los resultados ofrecidos por el sistema deben de presentarse con la información clara y detallada, para que el sanitario encargado de dar el informa pueda dar una respuesta con la mayor rigurosidad posible.

- Debe tener algún apartado que ofrezca información sobre contacto o ayuda en caso de no comprender el resultado o funcionamiento de algo del sistema o en caso de que de algún error inesperado.

4.1.4 Requisitos de calidad

- Definen los atributos de calidad que debe cumplir el sistema, como rendimiento, fiabilidad, seguridad, mantenibilidad, escalabilidad, disponibilidad, etc. Están enfocados en asegurar que el sistema sea robusto y cumpla con estándares de calidad.

- Debe de tener un sistema que informe al usuario en caso de tener problemas en el sistema por motivos de actualización, configuración, etc.

- Las imágenes que el sistema reciba por parte del sanitario deben de ser con una calidad mínima. En caso de que se detecte que no es así, avisar al personal encargado de su creación para realizar otra muestra y evitar falsas predicciones.

- La solución debe de cumplir con todos los estándares de calidad establecidos y las leyes regulatorias para poder salir al mercado y ofrecerse como una herramienta de ayuda para la salud ciudadana.

4.1.5 Requisitos de evolución

Se refieren a la capacidad del sistema para adaptarse a cambios futuros. Incluyen la facilidad de actualizar, mejorar o modificar el sistema a medida que cambian las necesidades del negocio o las tecnologías. Garantizan que el sistema sea extensible y flexible a largo plazo.

- El sistema debe de ser capaz en la mayoría de lo posible capaz de adaptarse a las nuevas exigencias por parte de los usuarios como de las nuevas tecnologías resurgentes. Además de actualizar el modelo periódicamente en caso de encontrar soluciones más eficientes o de que se pretenda generalizar a otras enfermedades de las que se quiera ofrecer una nueva alternativa para su diagnóstico y en lo que solución planteada pueda ser de utilidad.

- También poderse adaptar a cambios en el centro donde se haga uso del producto. Haciendo que sea lo más integro posible a la infraestructura para llevar un uso sencillo de la aplicación.

4.1.6 Requisitos de proyecto

Son los requisitos que involucran la planificación y gestión del proyecto, como presupuesto, plazos, recursos humanos y materiales. Incluyen los cronogramas, entregables, hitos y la gestión de riesgos para que el proyecto se lleve a cabo de manera exitosa.

- Tener un alto grado de ajuste con la planificación dada tanto a nivel

temporal como financiera. Para que los márgenes de coste y el tiempo de desarrollo sean coherentes.

- Hacer varias pruebas de testeo y usar una métrica fiable para evaluar los resultados y poder ir ajustando el modelo para conseguir mejorar su fiabilidad y aplicarlo como solución a la implementación final.

- Dentro de lo posible hacer uso con las herramientas planteadas originalmente para no exceder en más problemáticas que pueda conllevar en cuanto a demoras de tiempos por problemas de ajustes entre tecnologías o el coste que puedan tener el hacer uso de las mismas.

4.1.7 Requisitos de soporte

Definen las necesidades relacionadas con el soporte técnico y operativo del sistema una vez esté en producción. Esto incluye la documentación, capacitación del personal, mantenimiento, soporte post-implementación, resolución de incidencias y acuerdos de nivel de servicio (SLAs).

- Previamente a ser usado en un entorno real sobre pacientes. El personal sanitario debería de llevar a cabo una serie de pruebas previas como soporte, para familiarizarse con el funcionamiento y entender cómo se muestran los resultados para dar un diagnóstico correcto.

- Ofrecer toda la documentación necesaria para que pueda implantarse el sistema sin problemas en la institución, además de contar con toda la información relevante para su manejo, usabilidad, casos de uso, etc. Y también de direcciones para contactar en caso de necesitar ayuda con cualquier aspecto de la herramienta ya sea por uso o fallo.

4.2 ANÁLISIS DE RIESGOS

Para realizar un correcto análisis de riesgos es importante distinguir entre varias (subdivisiones) para clasificar los posibles riesgos, indicando la posibilidad de que ocurran y el impacto que supondría en el desarrollo del proyecto.

El nivel de riesgo ha sido diferenciado en dos. El impacto que tendrían en el correcto desarrollo del proyecto y la probabilidad en la que estos sucedan.

En las siguientes tablas se muestran los valores de dichos impactos:

Tabla de Probabilidad	
Valor	**Descripción**
Baja	Puede no llegar a ocurrir en todo el desarrollo del proyecto
Media	La amenaza puede presentarse muy ocasionalmente durante el ciclo de desarrollo
Alta	El riesgo de que suceda puede darse un día o alguno más por cada mes
Muy Probable	Es un problema muy común que puede llegar a suceder cada semana

Tabla 1: Valores de la probabilidad de un incidente

Tabla de Impacto	
Valor	**Descripción**
Bajo	El daño presentado no tiene consecuencias relevantes para la organización
Moderado	El daño percibido puede tener consecuencias reseñables para el proyecto
Alto	El daño derivado puede tener consecuencias graves reseñables para el desarrollo
Catastrófico	El daño presentado puede suponer unos de gran magnitud, complicando mucho su recuperación

Tabla 2: Valores del impacto de una incidencia

4.2.1 Tabla de riegos

En la siguiente tabla se van a mostrar los posibles problemas identificados que pueden suceder a lo largo de todo el desarrollo del proceso. Junto con los valores mencionados en las tablas anteriores. Además, añadiendo una posible solución en caso de que se llegara a suceder.

Etiqueta	**Riesgo**	**Probabilidad de ocurrencia**	**Impacto**	**Plan de mitigación**
R0	Incumplimiento con los tiempos	Media	Alto	Control riguroso en cuanto al avance de estas con una alta frecuencia
R1	Problemas en el control de versiones	Media	Moderado	Archivar todas las versiones realizadas anteriormente
R2	Fallos en la implementación del código	Alta	Alto	Analizar el comportamiento del código periódicamente
R3	Errores de los datos a emplear para el sistema	Baja	Moderado	Comprobar los datos previamente de su uso y buscarlos de fuentes fiables
R4	Falta organización	Baja	Moderado	Seguir la planificación propuesta inicialmente
R5	Mal uso o importación de las librerías	Alta	Alto	Estudiar el manejo de ellas antes de usarlas y comprobar que es compatible con el proyecto para su importación

Tabla 3: Riesgos posibles que ocurran durante el desarrollo

Etiqueta	Riesgo	Probabilidad de ocurrencia	Impacto	Plan de mitigación
R6	Pérdida de los resultados obtenidos	Baja	Catastrófico	Realizar subidas automáticas sobre los avances del proyecto
R7	Fallos con las Herramientas o tecnologías empleadas	Media	Alto	Hallar alternativas al empleo de ellas en caso de mal funcionamiento o mitigar los errores producidos
R8	Incongruencia con los resultados	Muy probable	Bajo	Analizar los posibles motivos que han llevado a esa solución y encontrar vías para su mejora
R9	Dependencia de nuevos recursos	Media	Alto	Explotar nuevas metodologías para poder llegar a resolución que sea óptima
R10	Aborto de la ejecución del sistema	Muy probable	Bajo	Tener que reiniciar la ejecución y volver a configurar todos los parámetros
R11	Actualizaciones o modificaciones de alguna tecnología utilizada	Media	Moderado	Explotar nuevas metodologías para poder llegar a resolución que sea óptima

Tabla 4: Riesgos posibles que ocurran durante el desarrollo II

4.2.2 Matriz de riesgo

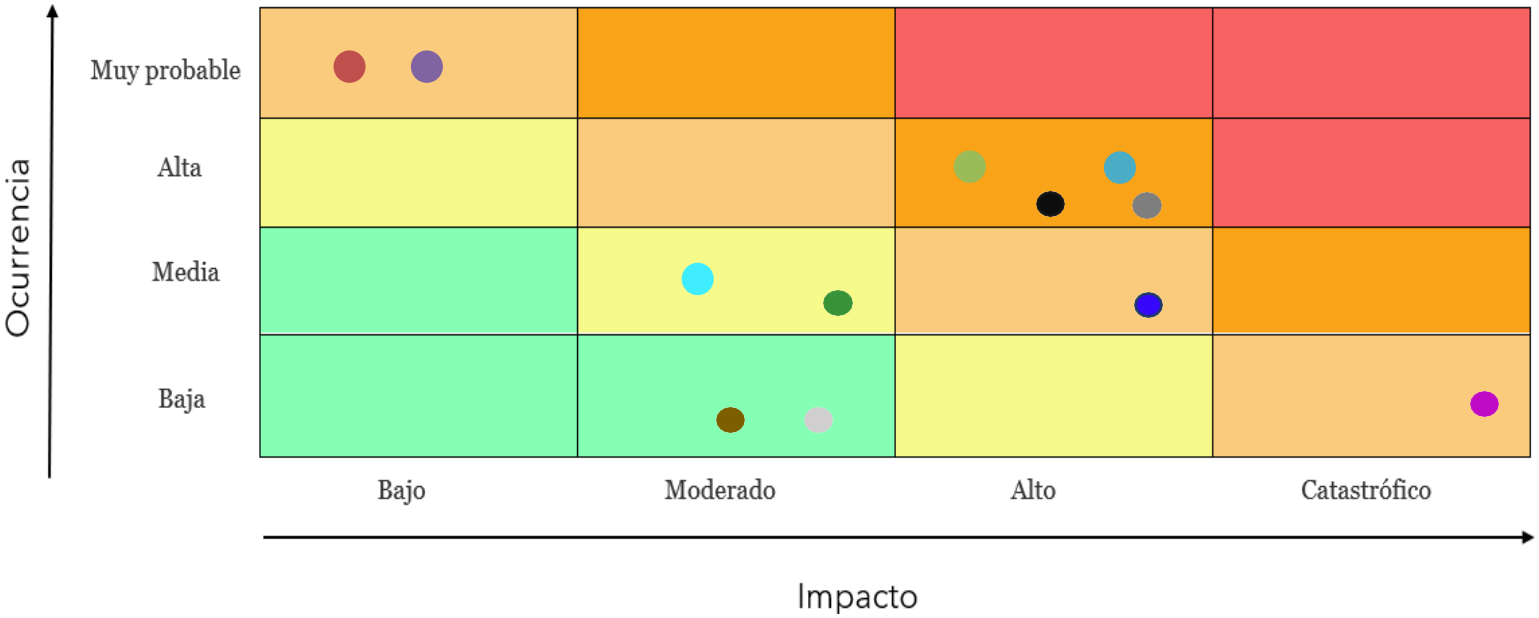

Figura 6: Matriz de riesgos

Etiqueta	Representación
R0	
R1	
R2	
R3	
R4	
R5	

Etiqueta	Representación
R6	
R7	
R8	
R9	
R10	
R11	

Tabla 5: Representación de cada riesgo en la matriz

5 DISEÑO DEL SISTEMA

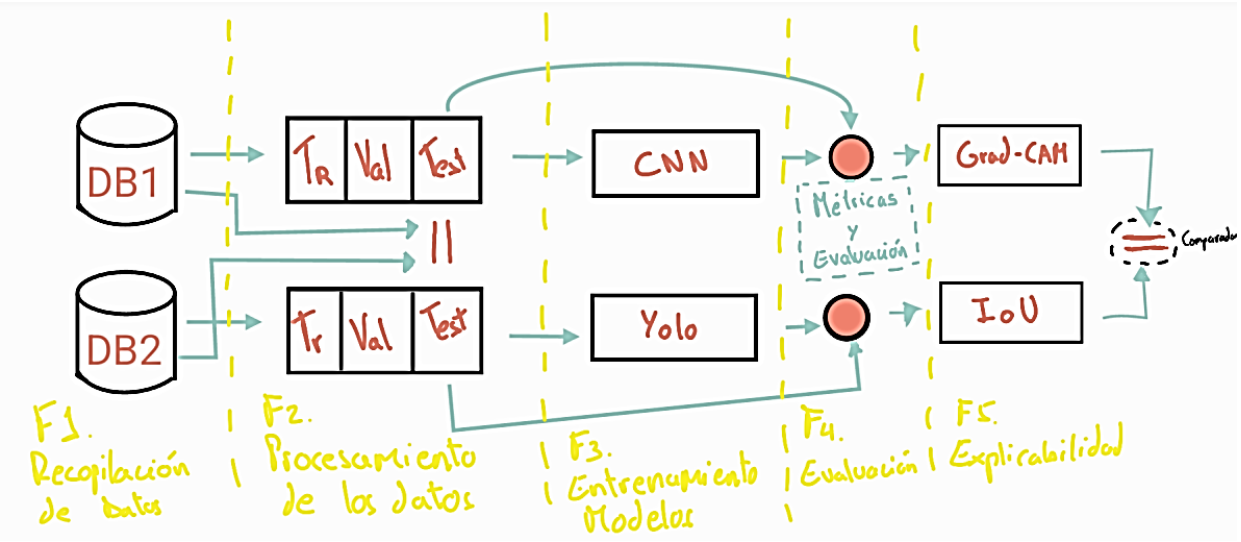

Figura 7: Diagrama del diseño del sistema

Datos:

Es la recopilación del conjunto de datos que se va a utilizar para el desarrollo del sistema, el modo en el que estos van a ser almacenados y estructurados, y las propiedades en específico de los datos.
Los datos han sido recopilados de diversas fuentes de buena fidelidad, ya que provienen de plataformas técnicas para el desarrollo de proyectos de investigación del mismo ámbito al que se propone con este proyecto. Por lo que estos datos normalmente son procedentes de grandes instituciones como centros hospitalarios, de investigación o universitarios.
Estos, se han almacenados en carpetas que depende del conjunto de datos, están divididos por etiqueta de clasificación o bien por subconjuntos para el entrenamiento, validación y test. Cada dato presente, consta de una imagen que representa alguna parte del cuerpo, para este caso pulmones, donde pueden verse presencia de tumores o no, dependiendo de la imagen. A su vez todas las imágenes tienen un archivo con información donde se expone la etiqueta indicando la presencia de tumor y en algunas de ellas una aclaración extra, señalando las ubicaciones de donde dichos tumores se encuentran en la imagen.

Procesamiento de los datos:

Sobre los datos se aplican una serie de procesos con la finalidad de que sean más aptos para construir los modelos con una mejor precisión y facilitar el manejo de ellos para cada una de las etapas de la implementación.
A las imágenes se les aplica ciertas modificaciones en cuanto a su dimensión,

escalado, algunos procesos de normalización; de forma que todas puedan emplearse de un modo más homogéneo para cada uno de los distintos modelos a probar y para posteriormente hacer las comparaciones de una manera más exhausta.
También se hacen manejos sobre los conjuntos de datos con el fin de tener algunos conjuntos nuevos que sean mezcla de otros provenientes de distintas fuentes. Así tener un conjunto de test con el que poder evaluar a los modelos de un modo más general. Y sobre ellos también se hace algunos procesos para añadir o modificar la información que se proporciona a cada imagen.

Red Neuronal, Entrenamiento, Construcción del modelo:

Se implementará un modelo secuencial diseñando cada una de las capas de su arquitectura, además de comprobar algunos de los modelos más conocidos de Keras y escoger entre dos para su uso. Así como determinar los parámetros de entrenamiento para cada uno de ellos. También se realizará lo mismo, pero con el modelo de detección de objetos Yolo.

Evaluación y clasificador

Con los modelos obtenidos tras la fase de entrenamiento, se evaluarán los resultados conseguidos tras hacer un proceso de clasificación al conjunto de test. Esta comparación se llevará a través de varias métricas a partir de los resultados anteriores Y así comprobar cual ofrece una mayor fiabilidad. Este será el que se tome como la solución óptima y por lo tanto el emplear para el resto de la investigación.

Explicabilidad

A partir del modelo tomado como solución, se aplicará alguna metodología del ámbito de la interpretabilidad y la explicabilidad de la IA. Esto al igual que con el proceso de evaluación, se aplicará para todo el conjunto de test. Para comprobar el correcto funcionamiento de los resultados conseguidos por este proceso, se harán una serie de cálculos para calcular el error cometido con respecto a los valores reales, que se conocen por el propio conjunto de datos. A partir de esta métrica se comparará entre las dos metodologías empleadas para conocer cuál es más precisa y así, proponer la implementación para la solución final, la cual será la que obtenga mejores resultados para cada una de las fases

6 IMPLEMENTACIÓN

6.1 TECNOLOGÍAS EMPLEADAS

6.1.1 Python

Características:

Python es un lenguaje de programación de alto nivel, orientado a objetos, con una semántica dinámica integrada. Es un lenguaje de programación de propósito general e interpretado. Por lo que el código fuente se ejecuta directamente, instrucción a instrucción sin un proceso de compilación, a través de un programa llamado interprete que lee las instrucciones en tiempo real y las ejecuta. El intérprete común mente utilizado es la Máquina Virtual de Python (PVM).

Python es relativamente simple, esto reduce el costo de mantenimiento y de desarrollo del programa. Uno de los beneficios más importantes de Python es que tanto la librería estándar como el intérprete están disponibles gratuitamente y todas las herramientas necesarias están disponibles en todas las plataformas principales.

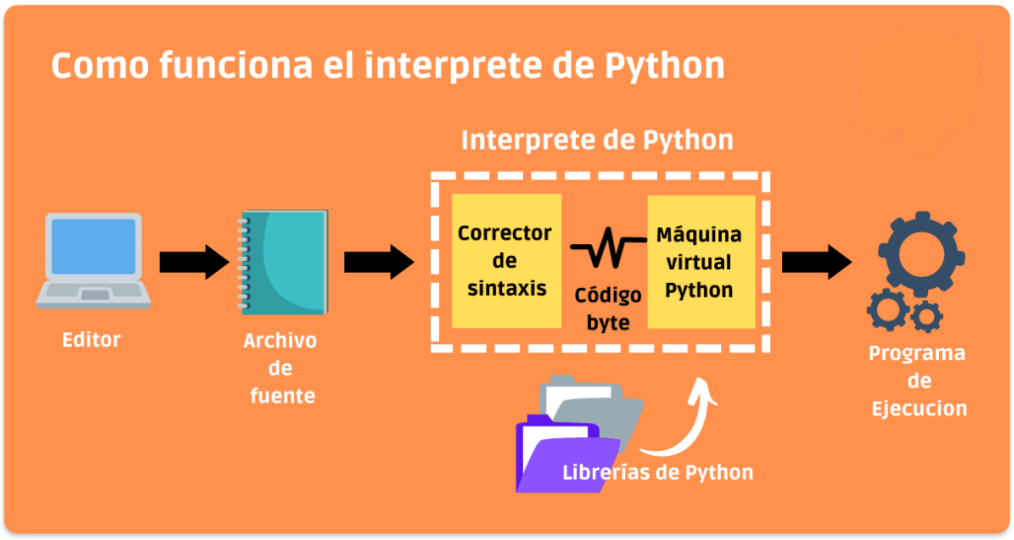

Figura 8: Esquema del funcionamiento de Python

Usos:

Python, aunque es de propósito general, es principalmente usado debido a sus características para el desarrollo web, de aplicaciones informáticas y el análisis de datos. En este último ámbito, Python adquiere una gran relevancia en campos emergentes como la ciencia de datos. Aplicándose como el leguaje más empleado en campos como la inteligencia artificial, machine learning, data mining, etc. También, puede ser usado para procesar texto, mostrar números o imágenes, resolver ecuaciones científicas y guardar datos. Y en otros muchos ámbitos gracias en gran parte por la gran cantidad de librerías que se ofrecen, las cuales permiten que pueda ser empleado en campos muy distinto como para el control de microcontroladores o el procesamiento de señales, etc.

Es un lenguaje que con el tiempo hay ido ganando importancia en la industria, siendo empleado actualmente por grandes corporaciones como, por ejemplo; en el motor de búsqueda de Google o YouTube, el sistema de transacciones de la bolsa de Nueva York o la NASA para programar sus equipos y maquinaria espacial. Además, ha conseguido afianzarse en los sistemas académicos dada la facilidad a la hora de aprenderse y la versatilidad que ofrece.

- **Ventajas**

Es multiplataforma. Gracias a que el intérprete suele estar en varios sistemas operativos, no es necesario adaptar el código a una plataforma específica.

Aumento de rendimiento en entornos web. Al ser interpretado es ejecutado por el cliente web, consiguiendo así disminuir la carga de trabajo del servidor.

Portabilidad. Por lo comentado anteriormente el programa puede ser llevado de una plataforma a otra sin tener problemas de compatibilidad.

Versatilidad. Puede ser usado en un gran abanico de escenarios sin tener que pasar por una curva de aprendizaje elevada para adaptarlo a cada uno de ellos.

- **Desventajas**

Velocidad de ejecución. Cuando hacemos el uso de un compilador, estamos transformando el programa a código máquina el cual es el empleado por el procesador. Mientras que, con el intérprete hay que hacer la correspondiente traducción de cada parte del código en tiempo real, lo cual conlleva a una pequeña ralentización. Además, este proceso hay que hacerlo siempre que se vaya a ejecutar el programa.

Necesidad de intérprete. Aunque es multiplataforma, siempre va a depender de algún framework o máquina virtual que haga la función de intérprete.

6.1.2 TensorFlow

Es la plataforma de Aprendizaje Profundo más importante del mundo. Es un desarrollo open-source de Google que, debido a su flexibilidad y gran comunidad de desarrolladores, lo han posicionado como la herramienta líder en el campo del Deep Learning.

Es una biblioteca de software para computación numérica, que utiliza gráficos de flujo de datos. Sirve para construir y entrenar redes neuronales, que permiten detectar y descifrar patrones y correlaciones, análogos al aprendizaje y razonamiento usados por los humanos.

Su arquitectura flexible permite implementar el cálculo a una o más CPU o GPU en equipos personales, servidores o dispositivos móviles con una sola API.

A partir de la popularización y mejora de esta, Google desarrolló Tensor Processing Unit (TPU). Se trata de una arquitectura construida en ASIC, específica para el aprendizaje automático. Es utilizado para ejecutar modelos más que para entrenarlos. Tras un largo periodo de pruebas demostraron que su rendimiento era mucho mayor para aplicaciones de aprendizaje automático que con los sistemas tradicionales. (TensorFlow. (n.d.). TensorFlow Documentation, s.f.)

Usos:

En cuanto a algunas de las aplicaciones que se le está dando actualmente, se usa comúnmente en tareas de clasificación de imágenes, detección de objetos, reconocimiento facial y segmentación de imágenes. Las Redes neuronales convolucionales (CNNs) son muy eficaces para este tipo de aplicaciones. También se aplica en la creación de sistemas de reconocimiento de voz, síntesis de voz y generación de música, a menudo utilizando redes neuronales recurrentes. Es altamente empleado en traducción automática, chatbots, generación de texto y modelos de lenguaje. Además de usarse como herramienta para el estudio de datos y usarse, por ejemplo; para predecir tendencias del mercado, análisis de riesgos, detección de fraudes o modelos de series temporales.

A parte de todos los usos anteriores, se ha iniciado la tendencia de emplearse la mejora de fotografías en smartphones, para ayudar a diagnósticos médicos con la detección de enfermedades a partir de imágenes (como se hace uso en este proyecto) o como ayuda para entrenar sistemas autónomos, como el caso de vehículos o drones, de forma que aprendan a navegar o tomar decisiones en tiempo real.

- **Ventajas**

Gran Comunidad y Ecosistema: Hay una amplia comunidad de usuarios y desarrolladores, lo que significa que es fácil encontrar recursos de aprendizaje, ejemplos y soporte. Hay mucha documentación y tutoriales disponibles.

Versatilidad y Flexibilidad: Ofrece múltiples APIs (de bajo nivel y alto nivel como Keras) y se adapta a diversas plataformas, lo que lo convierte en una herramienta muy flexible para desarrollar modelos de machine learning. Además, permite una gran escalabilidad puesto que permite ejecutar el mismo programa en varias máquinas y tanto sea en un entorno local como en la nube

Compatibilidad con Hardware Acelerado: Aprovecha aceleradores de hardware como GPUs y TPUs (Tensor Processing Units), los cuales mejoran considerablemente el rendimiento en entrenamientos de deep learning.

- **Desventajas**

Tamaño y Complejidad: En ciertos casos puede ser un poco pesado y complicado, especialmente cuando se compara con bibliotecas más ligeras como PyTorch. Para tareas más pequeñas o prototipos rápidos, algunas personas prefieren alternativas más sencillas.

Curva de Aprendizaje Pronunciada: A pesar de la simplificación introducida en TensorFlow 2.x y el uso de Keras, TensorFlow sigue siendo una biblioteca poderosa y compleja. Para aprovechar al máximo sus características avanzadas, es necesario tener un buen conocimiento de machine learning y deep learning. Y, aunque hay mucha documentación disponible, a veces puede resultar complicada de navegar debido a la amplitud y profundidad del ecosistema.

Consumo de Recursos: Puede ser exigente en términos de consumo de recursos de hardware, especialmente cuando se utilizan modelos grandes. Esto puede ser un inconveniente si se está trabajando en hardware limitado. Si se quiere obtener todas las posibilidades que ofrece usando los servicios de Google en la nube, requiere de una suscripción de pago.

6.1.3 Keras

Es una biblioteca de código abierto diseñada para facilitar el desarrollo de modelos de aprendizaje automático y, en particular, de aprendizaje profundo. se ha convertido en una de las interfaces de alto nivel más populares y accesibles para construir y entrenar redes neuronales. Actualmente Keras pertenece al paquete de TensorFlow como la API oficial de alto nivel. Además de ser de código abierto, se encuentra escrita en Python, facilitando su uso y reproducción.

Una de las principales características que definen a Keras es su facilidad de uso. Está diseñada para ser amigable con el usuario, lo que la convierte en una excelente herramienta tanto para principiantes como para investigadores y profesionales del machine learning. A través de su interfaz sencilla y clara, los usuarios pueden construir modelos de redes neuronales en pocas líneas de código, sin tener que preocuparse por los detalles técnicos más profundos que suelen requerir otras bibliotecas. Esto se logra gracias a su enfoque en la modularidad y la simplicidad, donde diferentes componentes de los modelos (capas, optimizadores, funciones de pérdida, etc.) se definen de manera clara y pueden combinarse fácilmente.

Keras ofrece soporte para una amplia variedad de arquitecturas de redes neuronales, incluyendo redes neuronales densas (fully connected), redes neuronales convolucionales (CNN) para procesamiento de imágenes, redes neuronales recurrentes (RNN) para datos secuenciales, y combinaciones complejas de estas arquitecturas. Además, también es compatible con las últimas innovaciones en deep learning, como transformers y técnicas de atención, lo que la hace relevante para tareas modernas de procesamiento de lenguaje natural (NLP) y visión por computadora.

También se destaca su portabilidad y compatibilidad multiplataforma. Al estar ya tan integrada con TensorFlow, permite ejecutar sus modelos en una gran variedad de dispositivos. Haciendo que puedan ser escalados fácilmente y desplegados en diferentes entornos tanto locales como móviles o en la nube. Además, se puede cambiar de infraestructura sin requerir de grandes cambios en el código utilizado.

Debido a todas estas características ha hecho que keras sea una de las herramientas más populares por toda la comunidad de machine y deep learning.

Keras cuenta de dos formas para generar una arquitectura de un modelo. API Secuencial donde se instancia un objeto de tipo Model y a partir de ahí, se van añadiendo las capas que conforman la arquitectura una detrás de otra. API Funcional se define una entrada de la que, a partir de la misma, se va definiendo el resto de la arquitectura. Indicando a cada capa cual es la entrada a cada capa de trabajada. Obteniendo como resultado un objeto modelo. (Keras. (n.d.). Keras Documentation, s.f.)

Figura 9: Esquema de abstracción de creación de una red neuronal

6.1.4 Ultralytics

Ultralytics es una plataforma avanzada centrada en el desarrollo y despliegue de modelos de visión por computadora con un enfoque especial en la detección de objetos utilizando inteligencia artificial. Reconocida por ser la creadora de los modelos YOLO (You Only Look Once), la plataforma ofrece una serie de herramientas y soluciones para simplificar el uso de modelos de aprendizaje profundo y facilitar la implementación de proyectos de IA para usuarios con diversos niveles de experiencia.

Ultralytics se utiliza principalmente para proyectos de detección de objetos, donde el objetivo es identificar y clasificar diferentes objetos en imágenes o vídeos en tiempo real. Estos casos de uso incluyen la seguridad (detección de personas, vehículos), la atención médica (detección de anomalías en imágenes médicas), y la automatización industrial (detección de defectos en productos). También es empleado para tareas de visión por computadora que van más allá de la simple clasificación, permitiendo el reconocimiento de patrones complejos dentro de imágenes, tales como el conteo de objetos, el seguimiento de objetos en movimiento o la segmentación de imágenes para separar diferentes regiones de interés.
Gracias a la posibilidad de exportar los modelos en formatos como TensorFlow Lite o CoreML, Ultralytics permite desplegar modelos de detección de objetos en dispositivos móviles, como aplicaciones que reconocen objetos en tiempo real o aplicaciones de realidad aumentada. Los modelos entrenados son posteriormente utilizados en sistemas autónomos como drones, vehículos autónomos y robots, donde la detección precisa de objetos en tiempo real es fundamental para la navegación, la toma de decisiones y la interacción con el entorno.
Además, tiene un gran impulso por investigadores y desarrolladores de IA, los cuales utilizan Ultralytics para crear prototipos rápidos de modelos de visión por computadora y probar nuevas arquitecturas o técnicas de entrenamiento, gracias a la flexibilidad y el rendimiento que ofrece el framework.

Ultralytics está diseñado para que incluso los usuarios con experiencia limitada en aprendizaje profundo puedan entrenar y desplegar modelos fácilmente. La simplicidad de su API y la posibilidad de utilizar modelos preentrenados lo hacen muy accesible para desarrolladores de todos los niveles. Una de las principales ventajas es la capacidad de detección de objetos en tiempo real. YOLOv5, el modelo más destacado de la plataforma es conocido por su velocidad y precisión, lo que lo convierte en una solución ideal para aplicaciones donde la latencia es crítica, como vigilancia, vehículos autónomos y análisis en vivo. Asimismo, la plataforma ha generado una gran comunidad de usuarios y desarrolladores que contribuyen con mejoras, tutoriales, ejemplos y soporte, lo que facilita encontrar ayuda o recursos para aprender y solucionar problemas.

Aunque es una excelente plataforma para detección de objetos, su enfoque está

más limitado en comparación con otras plataformas que cubren una gama más amplia de aplicaciones de aprendizaje automático, como clasificación de imágenes o segmentación semántica avanzada.

También, como la mayoría de los modelos de aprendizaje profundo, los modelos desarrollados en Ultralytics requieren grandes cantidades de datos de alta calidad para ser entrenados de manera efectiva. Si los datos están desequilibrados o son insuficientes, la precisión del modelo puede verse comprometida. Entornos de trabajo empleados para el desarrollo

Es una aplicación que proporciona al usuario de una serie de herramientas para facilitar el desarrollo de un proyecto. Para este proyecto se han empleado varios, distinguiéndolo entre aquellos alojados en la nube o los que son usados como aplicaciones instaladas de manera local. (Ultralytics YOLOv5. (n.d.). YOLOv5 Documentation, s.f.)

Figura 10: Diagrama de trabajo de ultralytics

6.1.4.1 Googel Colab

Es un entorno de desarrollo basado en la nube que permite a los usuarios escribir y ejecutar código Python directamente desde un navegador web. Es una herramienta gratuita ofrecida por Google, especialmente adecuado para tareas de machine learning, deep learning, y análisis de datos. Colab facilita la creación y ejecución de notebooks Jupyter, un formato ampliamente utilizado en ciencia de datos que combina código, texto, imágenes y resultados de ejecución, todo en un solo documento. Estos no requieren configuración y ofrecen acceso sin coste adicional a recursos informáticos, como GPUs a través de los servidores de la propia compañía.

Es ampliamente utilizado en cursos y tutoriales sobre machine learning y deep learning, debido a su facilidad de acceso y el soporte integrado de hardware acelerado, lo que permite experimentar con modelos sin necesidad de un equipo costoso. Para desarrolladores de IA o ingenieros de machine learning, Colab es útil para la creación rápida de prototipos de modelos o algoritmos, ya que permite iterar fácilmente sin preocuparse por la configuración de entornos locales. También es utilizado por científicos de datos para realizar experimentos, análisis exploratorio de datos (EDA), visualizaciones interactivas, y pruebas de modelos de machine learning en entornos

colaborativos. Gracias a sus recursos computacionales se presenta como una gran herramienta para entrenar modelos de redes neuronales. (Google Colab. (n.d.). Google Colab Documentation., s.f.)

- **Ventajas**

Facilidad de Uso: Al ser accesible desde cualquier navegador, no se requiere instalación de software, configuración de dependencias ni hardware especializado. Esto lo hace ideal para principiantes y usuarios que buscan una solución rápida y efectiva para ejecutar sus modelos. Además, ya viene preconfigurado con las principales librerías de machine learning, por lo que no es necesario configurar el entorno manualmente.

Colaboración y reproducibilidad: Permite a múltiples usuarios trabajar en un mismo notebook simultáneamente, algo que facilita enormemente el trabajo en equipo. Los notebooks de Colab se pueden compartir mediante enlaces, lo que facilita la colaboración y el intercambio de experimentos de manera eficiente. Asimismo, el almacenamiento y sincronización de archivos a través de Google Drive hace que sea fácil acceder y compartir notebooks, datos, y modelos desde cualquier lugar.

Acceso Gratuito a GPUs/TPUs: La capacidad de usar hardware acelerado sin coste adicional es una de las principales ventajas de Colab, especialmente cuando no se cuenta con acceso a GPUs en máquinas locales.

- **Desventajas**

Límites de Uso: A pesar de ser gratuito, Google Colab impone ciertas limitaciones en cuanto a los recursos. Las sesiones se desconectan después de períodos de inactividad y hay límites en la duración de las ejecuciones que pueden ser entre 12 y 24 horas. Además, la disponibilidad de GPUs y TPUs no está garantizada y puede variar en función de la demanda que se esté haciendo estos en el momento de su requerimiento.

Rendimiento Limitado: Aunque Colab proporciona acceso a hardware acelerado, este está compartido entre muchos usuarios, lo que significa que el rendimiento puede no ser tan consistente como en un entorno privado o dedicado. Y dado que es completamente en línea, una conexión a Internet inestable o lenta puede afectar negativamente la experiencia del usuario. Además, la desconexión del servidor a veces puede resultar en pérdida de progreso si no se guarda el trabajo regularmente.

Espacio de Almacenamiento Temporal: Los datos y archivos almacenados en la sesión se pierden una vez que se cierra la conexión con el servidor. Para mantener archivos, es necesario cargarlos continuamente desde Google Drive o subirlos al inicio de cada sesión.

No Ideal para Producción: Si bien Colab es excelente para experimentación y desarrollo, no está diseñado para implementaciones en producción. Las restricciones de tiempo y la desconexión automática lo hacen menos adecuado para procesos de producción a largo plazo.

Figura 11: Logotipo del entorno de Google Colab

6.1.4.2 Visual Studio Code:

Visual Studio Code (VS Code) es un editor de código fuente gratuito y de código abierto optimizado con soporte para operaciones de desarrollo como depuración, ejecución de tareas y el control de versiones. Ha ganado una inmensa popularidad entre desarrolladores de todo el mundo por su ligereza, flexibilidad y capacidad de personalización. A pesar de ser liviano, VS Code es extremadamente potente, ofreciendo las herramientas necesarias para un ciclo rápido de compilación y depuración, aunque dejando las características más potentes para los IDE más complejos.

Es usado para una infinidad de proyectos de distintos propósitos. El entorno adquiere un gran valor gracias a las extensiones. Con estas te permite programar en cualquier lenguaje de programación, modificar los estilos de la sintaxis para que sea lo más acorde al tipo de proyecto, incluir frameworks o herramientas de trabajo. Entre los proyectos más frecuente en el que se hace uso para su desarrollo son: el desarrollo web, aplicaciones móviles o para la programación de hardware como microcontroladores. (Visual Studio Code. (n.d.). Visual Studio Code Documentation., s.f.)

- **Ventajas**

Soporte Multilenguaje: VS Code soporta una amplia variedad de lenguajes de programación como JavaScript, Python, Java, C++, HTML/CSS, y muchos más.

Ligereza y Rapidez: Está diseñado para ser rápido y eficiente. A pesar de su riqueza en funcionalidades, sigue siendo poco pesado, lo que lo hace rápido de instalar y ejecutar en una variedad de sistemas operativos como Windows, macOS y Linux.

Extensiones y Personalización: Una de las principales fortalezas es su enorme Marketplace de Extensiones. Esto permite personalizar completamente

el editor agregando extensiones para soporte de lenguajes de programación, herramientas de depuración, snippets, temas, y más. Pudiendo adaptar el entorno a prácticamente cualquier flujo de trabajo o lenguaje de programación.

Depurador Integrado: A diferencia de muchos editores de código, VS Code incluye un depurador completamente funcional que permite a los desarrolladores ejecutar su código paso a paso, establecer puntos de interrupción (breakpoints), inspeccionar variables y analizar el flujo de ejecución, todo desde el propio editor.

Autocompletado Inteligente: Una característica muy interesante que integra es IntelliSense, la cual ofrece autocompletado de código de forma inteligente. Esta característica sugiere palabras clave y métodos a medida que escribes, basándose en la sintaxis del lenguaje y en las librerías instaladas, lo que mejora significativamente la productividad y precisión del código.

Integración con Git: Cuenta con integración nativa con Git, lo que permite a los usuarios gestionar el control de versiones directamente desde el editor. Puedes realizar commits, hacer pull/push a repositorios, revisar diferencias en archivos, y gestionar ramas de Git sin salir del entorno, sin tener que lanzar comandos desde el terminal.

Terminal Integrada: Incluye un terminal embebido, que permite ejecutar comandos del sistema operativo o scripts directamente desde el editor. Esto es especialmente útil para ejecutar el código, lanzar pruebas o la administración de sistemas, sin necesidad de cambiar de ventana.

- **Desventajas**

Funcionalidades Limitadas en Comparación con IDEs Completos: Aunque es un editor poderoso, no es un entorno de desarrollo integrado (IDE) completo. Para proyectos muy grandes o complejos, puede no ser tan robusto como IDEs más especializados, como Visual Studio, IntelliJ IDEA o PyCharm, que ofrecen herramientas más avanzadas de depuración, análisis de código y gestión de proyectos. Cuando se trabaja con proyectos extremadamente grandes o con muchos archivos, VS Code puede volverse más lento que un IDE completo optimizado para manejar grandes volúmenes de código y dependencias.

Dependencia de Extensiones: Muchas de las funcionalidades avanzadas que los usuarios esperan (como soporte completo para ciertos lenguajes, frameworks o depuración avanzada) dependen de extensiones de terceros. Esto puede requerir tiempo adicional para configurar y mantener un entorno eficiente y adecuado.

Curva de Aprendizaje para Personalización: Si bien es sencillo de usar en su versión básica, los usuarios avanzados que buscan personalizar el editor con extensiones y configuraciones específicas pueden enfrentarse a una curva de aprendizaje. La personalización excesiva también puede llevar a conflictos entre extensiones.

6.2 Estudio Previo

6.2.1 Modelo de entrenamiento (Red Neuronal)

6.2.1.1 Redes Neuronales

Las redes neuronales son un modelo computacional que emula el comportamiento del humano para procesar la información. Están formadas por multitud de neuronas. Estas neuronas son la unidad más básica y suelen estar organizadas en capas, conectándose entre sí para transmitir señales. Las redes normalmente se forman por una capa de entrada en la que cada unidad representa los parámetros de entrada, una o varias capas ocultas y una capa de salida donde cada unidad representa las posibles soluciones o clasificaciones. (neuronal networks, s.f.)

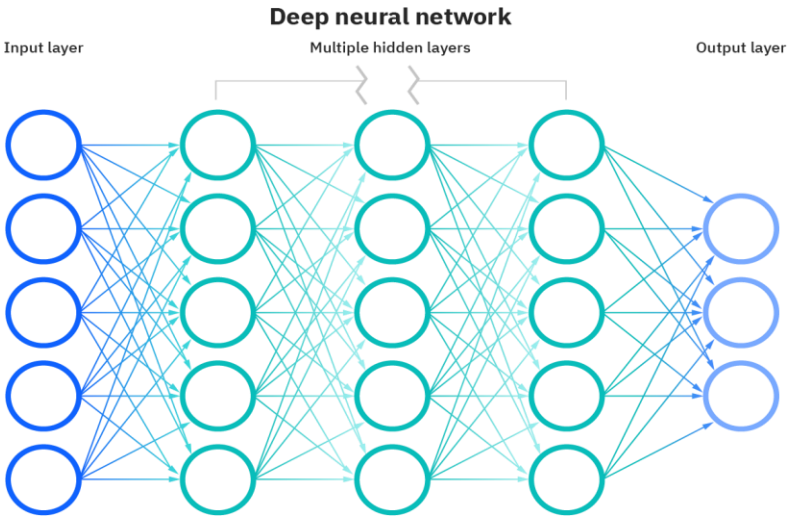

Figura 12: Aspecto de una red neuronal

Cada nodo individual o neurona funciona como un modelo de regresión lineal, obteniendo un resultado a partir de los datos de entrada, a los que se le ha aplicado una suma ponderada y un sesgo.

$$\sum_{i=1}^{m} w_i x_i + bias = w_1 x_1 + w_2 x_2 + w_3 x_3 + bias$$

Figura 13: Suma de las ponderaciones de una neurona

Para obtener una salida más compleja y que pueda conseguirse un modelo capaz de representar más acorde los datos, se le aplica algún tipo de función, que funciona como filtrado, a la salida obtenida por dicha suma de ponderaciones. Gracias a esto evitamos que el modelo obtenido no sea simplemente un modelo de regresión lineal y que se puedan encadenar las neuronas entre capas. Ya que, el

resultado obtenido de una cadena de sumas de regresiones lineales obtiene como resultado otra regresión lineal, lo que equivaldría a que el modelo se realizase utilizando una única neurona.

A dicha función se le conoce como función de activación. Algunas de estas funciones más utilizadas son: Sigmoidal, Lineal, Gaussiana, Escalonada, ReLu, etc.

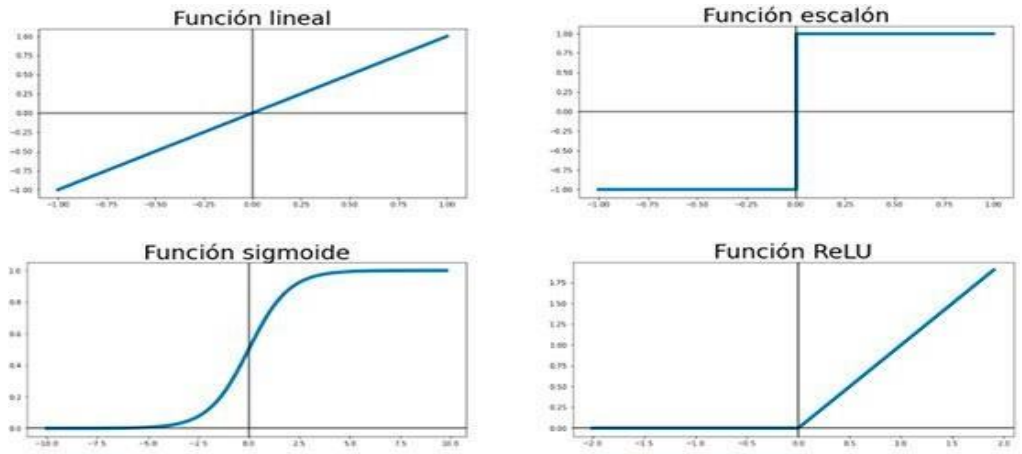

Figura 14: Funciones de activación

Las redes neuronales se pueden clasificar según su modo de aprendizaje. Este modo es el que define que técnica va a usar para ajustar sus parámetros en el proceso de entrenamiento del sistema.

- **Aprendizaje supervisado**

Consiste en ir mejorando a la red, a partir de realizar el cálculo del error obtenido por el resultado predicho sobre el resultado real de los datos de entradas de los cuales conocemos su clasificación. Con el fin de hacer varias iteraciones hasta que los resultados obtenidos sean lo más cercano posible a la salida esperada.

- **Aprendizaje no supervisado**

En este paradigma se desconocer sobre los datos cuál es su correspondiente clasificación. Se basa en que la red sea capaz de aprender a través de la observación de los datos de entrada y que se reconozcan sobre ellos ciertos patrones, características, regularidades de similitud para poder determinar cuál es la estructura interna que genera los datos.

- **Aprendizaje por refuerzo**

Se basa en crear un sistema de recompensas en el cual los resultados correctos son premiados a través de un refuerzo positivo, mientras que los erróneos son disuadidos.

- **Aprendizaje híbrido**

Utiliza una combinación del resto de paradigmas mencionados. La red aprende examinando cada uno de los datos de entrada, por cada uno de ellos realiza una predicción y si el resultado es incorrecto, ajusta las ponderaciones. Este proceso es repetido por muchas iteraciones, hasta haber alcanzado algún criterio de parada. A dichas iteraciones se les conoce como épocas.

Todas las ponderaciones con las que inicia pueden ser dadas por algún modelo ya creado o generadas de modo aleatorio. Por lo que, al inicio lo habitual es que la red de unas respuestas erróneas. La mejora de su predicción se consigue a través del entrenamiento que consiste en ese ajuste de las ponderaciones comentado anteriormente. El ajuste se consigue a través del cálculo del error obtenido tras los resultados obtenido por varias predicciones. Este sirve para modificar las ponderaciones de la red y ajustarlas de modo gradual en cada iteración. Las ponderaciones ayudan a conocer la importancia que tiene cada una de las variables de entrada, siendo las de valor mayor aquellas que dan más relevancia e importancia a dicha entrada. Con el cálculo del error se ajustan las ponderaciones, de un modo más agresivo a dichas ponderaciones que han tenido mayor peso para obtener la predicción errónea.

La siguiente fórmula representa el cálculo del error cuadrático medio, el cual es el más comúnmente utilizado.

$$MSE = \frac{1}{2m} \sum_{i=1}^{m} (\hat{y} - y)^2$$

Figura 15: Cálculo del error cuadrático

Para hacer el cálculo de dicho error y reajuste de las ponderaciones según el criterio comentado antes, se usa una técnica denominada **Backpropagation**:

Que consiste en realizar el cálculo de los ajustes de las ponderaciones, a partir del error obtenido como salida para cada una de todas las neuronas que componen la red. El resultado del error de salida es propagado hacías en sentido inverso. Desde la capa de salida hacía la capa de entrada. Así se reduce mucho el tiempo de cálculo para ajustar las neuronas, además de que se detecta fácilmente cuales han influenciado en mayor medida para el resultado de la predicción.

Con el avance de las iteraciones y mejora de los ajustes de las ponderaciones, se consigue hacer que la solución se acerque a puntos locales o puntos de convergencia. Tras un proceso largo iterativo se puede producir lo que se conoce como **Overfitting**:

El modelo obtenido se ha conseguido tras sobreentrenar el sistema de aprendizaje, obteniendo un sobreajuste de los parámetros. Por lo que para los datos de entrenamiento se consiguen unos resultados con un alto grado de precisión en el acierto de las predicciones y con un resultado de error muy bajo. Pero cuando se usa el modelo para predecir datos que no han sido usados en la etapa de entrenamiento, se consigue obtener un porcentaje muy alto de error. Esto es producido porque el modelo representa con mucha fiabilidad a los datos de entrenamiento, sin ser capaz de generalizar. Esto es lo que se conoce como que la red ha memorizado los patrones de los datos.

En la imagen se muestra lo que podría ser el valor del error de un sistema por cada iteración.

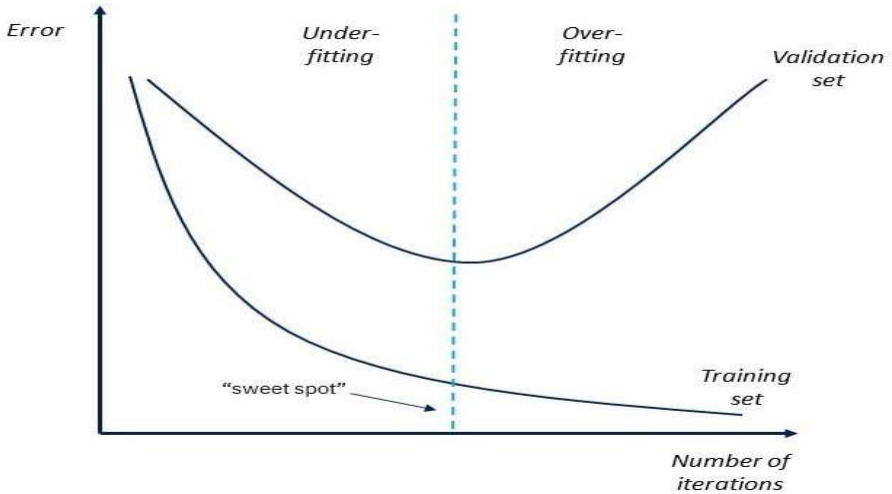

Figura 16: Gráfico de la evolución del error durante el entrenamiento de la red

El caso opuesto a Overfitting es cuando el sistema no es capaz de encontrar un ajuste de los parámetros para poder crear un modelo que represente en la medida de lo posible al conjunto de datos;

A esto se le conoce como **Underfitting** y puede ser producido por muchos motivos, ya sea que la red empleada requiere de más capas, más neuronas por capas, un ajuste más agresivo sobre el cálculo del error, etc.

Estos suelen ser de los problemas más habituales cuando se trata de trabajar un sistema de red neuronales donde de los factores más importantes a tener en cuenta, es ser capaz de crear un modelo con la mayor generalización posible para que sea representativo en la mayor cantidad de datos.

Las redes según la complejidad del problema y el tipo de datos de entrada, se pueden clasificar según su tipología y disposición de las neuronas. Siendo entre las más comunes:

- **Redes neuronales monocapa:** Son la red más básica y simple con una conexión de una sola capa. Usadas para clasificar patrones que son linealmente diferenciables.
- **Redes neuronales multicapa:** Se colocan una serie de capas ocultas intermedias entre la de entrada y salida. Consiguiendo crear modelos más complejos y que permitan eliminar información que no es relevante para la solución.
- **Redes neuronales recurrentes:** Estas cuentan con retroalimentación entre neuronas de diferentes capas, que se encuentran en la mismo o con una neurona así mismo.
- **Redes neuronales convolucionales:** Son las más complejas. Cuentan con varias capas diferentes, en las que cada una de ellas está entrenada para llevar a cabo una tarea diferente. Están diseñadas emulando un comportamiento muy similar al que hacen las neuronas de la corteza visual primaria de un cerebro biológico. Están especialmente diseñadas para el problema que supone el crear un modelo capaz de identificar imágenes grandes. Debido a que son muchos los parámetros de entrada.

Este último tipo es el usado para la implementación de nuestra solución. Debido a las grades capacidades que ofrece y ya que los datos de entrada empleado para el sistema se componen de muchas características, esto implica que la red neuronal va a tener muchas neuronas de entrada y justo para eso las convolucionales son las más apropiadas.

6.2.1.2 Métricas de evaluación

Para evaluar un modelo de redes neuronales se utilizan distintas métricas con los que medir su calidad y conocer fiabilidad como clasificador. Las siguientes métricas hacen uso de una serie de términos que son referidos al resultado obtenido tras una clasificación. Estos términos muestras un balance de lo obtenido tras el resultado de clasificar todo el conjunto de datos.

(evaluate the performance of a ml model, s.f.)

Dichos términos son:

- **True Positives (TP):** Estos son los valores positivos predichos correctamente, lo que significa que tanto el valor de la clase real como el de la clase predicha son sí.

- **True Negatives (TN):** Estos son los valores negativos predichos correctamente, lo que significa que el tanto valor de la clase real como el de la clase predicha son no.

- **False Positives (FP):** Cuando la clase real es no o no existente y el resultado de la predicción lo atribuye a alguna clase, asignándolo como sí.

- **False Negative (FN):** Cuando una clase es detectada como positiva, pero, se clasifica erróneamente a otra clase, asignándose como negativa.

Aplicando estos parámetros se puede realizar el cálculo de las siguientes métricas:

- **Accuracy**

La precisión es la medida de rendimiento más simple y se trata de la relación existente entre la observación correctamente predicha y el total de observaciones. Esto podría llevar a usarla a modo, que, si tenemos una alta precisión, nuestro modelo es el mejor. En cierta medida sí, podría ser una buena métrica para conocer la calidad de nuestro modelo, pero sólo es realmente válido cuando se tienen conjuntos de datos simétricos en los que los valores de los falsos positivos y los falsos negativos son casi iguales. Por lo tanto, hay que tener en cuenta otros parámetros para evaluar el rendimiento del modelo.

$$\boldsymbol{Accuracy} = \frac{\boldsymbol{TP + TN}}{\boldsymbol{TP + FP + TN + FN}}$$

- **Precision**

Es la relación entre las observaciones positivas predichas correctamente y el total de observaciones positivas predichas. La alta precisión se relaciona con la baja tasa de falsos positivos.

$$\boldsymbol{Precision} = \frac{\boldsymbol{TP}}{\boldsymbol{TP + FP}}$$

- **Recall**

Es la relación entre las observaciones positivas predichas correctamente y todas las observaciones donde la clase es sí.

$$\boldsymbol{Recall} = \frac{\boldsymbol{TP}}{\boldsymbol{TP + FN}}$$

- **F1 Score**

Es la media ponderada de Precision y Recall. Por lo tanto, esta puntuación tiene en cuenta tanto los falsos positivos como los falsos negativos. No es tan intuitivo como Accuracy, pero F1 suele ser más útil, especialmente si se tiene una distribución de clases desigual. Accuracy funciona mejor si los falsos positivos y los falsos negativos tienen un coste similar. Si el coste de los falsos positivos y de los falsos negativos es muy diferente, es mejor tener en cuenta tanto Precision como Recall.

$$\boldsymbol{F1\ Score = 2 * \frac{Precision * Recall}{Precision + Recall}}$$

6.2.1.3 Matriz de Confusión

Es una herramienta que permite visualizar el desempeño obtenido por un modelo. Representa los resultados obtenidos dividiendo cada columna el número de predicciones de cada clase y las filas en las etiquetas de las clases reales. Esto permite ver de modo claro que tipos de errores y aciertos está cometiendo el modelo a la hora de predecir sobre los datos del conjunto de entrenamiento.

		Actual	
		Positivo	Negativo
Predicción	Positivo	Verdadero Positivo (TP)	Falso Positivo (FP)
	Negativo	Falso Negativo (FN)	Verdadero Negativo (TN)

Tabla 6: Estructura de una matriz de confusión

6.2.1.4 Parámetros e hiperparámetros de la implementación de la red

Los parámetros son aquellos fijos determinados por la arquitectura de la red para que funcione correctamente. En cuanto a lo hiperparámetros, estos si son modificables y sirven para ajustar el modelo, cambiando como va a actuar durante el entrenamiento, con el fin de que consiga realizar mejores predicciones. Estos son los siguientes:

- **Batch size:** Definen el número de datos que pasarás por la red para realizar el correspondiente cálculo del error y corregir los pesos.

- **Learning rate:** Especifica la agresividad con la que va a actuar en la modificación de los pesos de las neuronas a partir del error calculado. Esta puede influir mucho en la velocidad con la que el modelo consiga ir mejorando o que el modelo avance de un modo más escalonado o lineal en la mejora de su predicción y reducción del error. Su valor oscila entre 0 y 1. Cuanto más cercano es a 1 realiza una modificación mayor sobre los pesos. Mientras que con un valor bajo cercano a 0, los pesos sufren una modificación más leve por cada iteración. Un valor muy diminuto podría hacer que se tardase mucho tiempo en alcanzar una solución óptima. Por otro lado, un valor muy alto haría que se modificasen muchos los pesos en cada iteración haciendo que cometiese muchos errores y dificultando mucho la labor de encontrar un modelo con un buen índice de predicción.

- **Epochs:** Establece el número de iteraciones de entrenamiento que se van a realizar.

- **Callbacks:** No es un requisito usarlo, pero resulta útil para conseguir que durante el entrenamiento se hagan ciertas peticiones que se solicitan a través de esta llamada. Algunas de estas pueden ser; que el modelo se vaya guardando por cada iteración en el que se ha conseguido mejorar, o que sí a partir de un número de épocas determinadas no ha conseguido mejora que deje el entrenamiento.
  Emplea las siguientes llamadas: ModelCheckpoint, Early stop.

6.2.2 Explicabilidad de la inteligencia artificial

La Inteligencia artificial explicable (XAI), nos permite poder acercarnos a la lógica que está detrás de los algoritmos de machine learning y deep learning. Estos algoritmos normalmente son empleados para encontrar las relaciones que existen entre los elementos que se han podido codificar en una base de datos.

Los algoritmos de predicción llevan muchos años entre nosotros, regresiones, clústeres, árboles de decisión… son herramientas que se han empleado en estadística desde siempre. Pero, con la aparición del Big Data, se pusieron en práctica nuevas formas de predicción (algunas de las cuales ya existían, pero no se habían podido utilizar por la cantidad de datos y necesidades de procesamiento que requerían) y trajeron consigo unos nuevos enfoques analíticos.

Una de las principales diferencias es que estás nuevas técnicas: de machine learning y deep learning, no requieren de hipótesis previas, mientras que las técnicas tradicionales sí. Es decir, con las técnicas tradicionales era necesario tener una idea previa (hipótesis) de cuales eran esas relaciones entre elementos que se trataban de identificar, mientras que, con las nuevas técnicas, esa hipótesis previa ya no es necesaria y se suple con una enorme cantidad de datos. Pero estas nuevas técnicas, igual que no requieren de una hipótesis previa, cuando generan un resultado lo hacen de tal manera que las relaciones que han identificado tampoco nos son visibles. No se explicitan las relaciones, no se entienden los resultados, sólo se predice un valor y se toma una decisión.

Esto causa un problema a todos los niveles:

Para los investigadores/ desarrolladores con los algoritmos porque les resulta muy difícil optimizarlos sin poder entender bien cómo funcionan.

En cuestiones éticas y legales, como los algoritmos que van a regir los coches autónomos. Cuando tomen decisiones que puedan afectar gravemente a la vida de las personas, como decidir frenar bruscamente para evitar un atropello, pero causar graves lesiones al conductor del vehículo. Este tipo de predicciones/ decisiones y cómo se toman, deben poder ser analizadas por los legisladores y responsables éticos para poder establecer unas normas justas y consensuadas.

Para los usuarios profesionales de estos resultados: Por ejemplo: En las decisiones críticas de atención médica, como el diagnóstico o los planes de tratamiento, es crucial comprender por qué un modelo de IA hace una recomendación o predicción específica. A pesar de la precisión de muchos modelos de IA avanzados, un desafío importante radica en su falta de transparencia a la hora de explicar el fundamento de sus conclusiones. La mínima interpretabilidad, combinada con las preocupaciones éticas y regulatorias, obstaculiza la aceptación y la confianza de los médicos.

Para los usuarios finales de estos resultados. Los consumidores están sujetos constantemente a decisiones opacas de algoritmos de machine learning y deep learning, desde a qué publicidad están expuestos, hasta el límite de su tarjeta de crédito o si se les concede o no una hipoteca, por tanto, deberían poder entender por qué se toman estas decisiones sobre ellos, en base a qué aspectos se decide que el límite de crédito de una persona sea X. Pero no sólo "deberían poder", sino que el derecho de acceso recogido en el RGPD así lo recoge: los usuarios tienen derecho a conocer la lógica de los algoritmos que los perfilan cuando en base a ellos se tomen decisiones que les afectan.

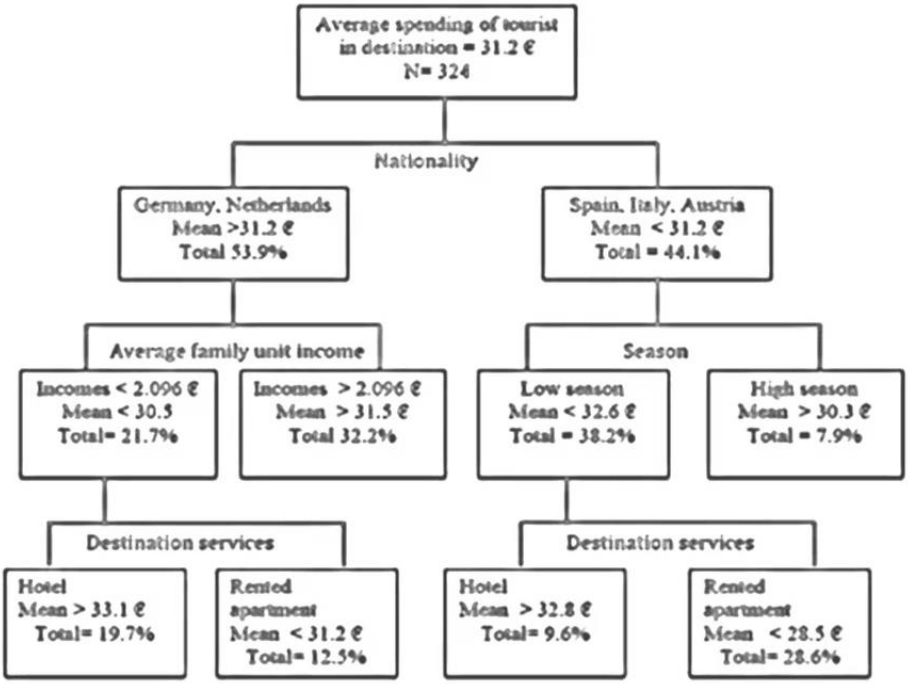

Figura 17: Esquema de funcionamiento de un árbol de decisión

Debido a esto, surge la necesidad de explicar los resultados y aparece el concepto XAI.

La **XAI** (Explainable Artificial Intelligence) o Inteligencia Artificial Explicable es un campo dentro de la inteligencia artificial, cuyo objetivo es hacer que los modelos y sistemas de IA sean más interpretables y comprensibles para los humanos. A medida que los sistemas de inteligencia artificial se vuelven más complejos, especialmente con el uso de redes neuronales profundas y otros algoritmos avanzados, se ha hecho más difícil entender cómo llegan a tomar decisiones. XAI surge como una respuesta a la que comúnmente se denomina "caja negra" de la IA, donde los resultados son difíciles de explicar. (Explainable Artificial Intelligence, s.f.)

Importancia de la XAI:

La capacidad de explicar el comportamiento de los sistemas de IA es crucial en muchos campos donde la interpretación, la transparencia y la responsabilidad son esenciales. Esto incluye áreas como la salud, el derecho, las finanzas y el gobierno, donde las decisiones automatizadas pueden tener un impacto significativo en la vida de las personas. A continuación, se detallan algunas razones clave para la relevancia de la XAI:

La relevancia de la Inteligencia Artificial Explicable (XAI) radica en varias razones clave. En primer lugar, está la responsabilidad y la transparencia. Los sistemas de IA deben ser capaces de justificar sus decisiones, especialmente en áreas críticas como el diagnóstico médico o las decisiones judiciales. La XAI facilita la auditoría de los procesos de toma de decisiones, lo que aumenta la confianza en estos sistemas. Otro aspecto importante es el cumplimiento de las normativas. En sectores regulados, como la banca o la salud, las organizaciones están obligadas a explicar cómo y por qué sus sistemas automáticos tomaron determinadas decisiones. Normativas como el Reglamento General de Protección de Datos (GDPR) en Europa exigen que las empresas proporcionen explicaciones sobre las decisiones automatizadas que afectan a los usuarios. Además, la XAI contribuye a mejorar la toma de decisiones, ya que al entender cómo un modelo de IA llega a una conclusión, tanto desarrolladores como usuarios pueden ajustar y optimizar los sistemas de manera más eficiente. Finalmente, es esencial para la detección de sesgos. La XAI permite identificar sesgos en los modelos, lo que resulta crucial para evitar resultados discriminatorios o injustos en aplicaciones como la contratación de personal, la concesión de crédito. (Beneficios XAI, s.f.)

Para lograr esto se tienen que abordar una serie de cuestiones claves como:

1. ¿Cuál es el razonamiento subyacente a la predicción del modelo?
2. ¿Qué factores contribuyen a esta predicción particular?
3. ¿Qué elementos juegan un papel fundamental en la configuración del resultado?
4. ¿Cómo funciona el modelo para llegar a sus predicciones?

Tipos de Explicaciones en XAI:

Existen varios enfoques para proporcionar explicaciones en sistemas de IA. IBM ha esbozado recientemente tres técnicas clave esenciales para lograr la explicabilidad en la IA. En primer lugar, es fundamental evaluar la precisión de la predicción del sistema. Las explicaciones locales interpretables independientes del modelo (LIME) se destacan como la técnica predominante para este propósito. En segundo lugar, es crucial garantizar la trazabilidad del sistema, y DeepLIFT (Deep Learning Important FeaTures) sirve como un caso de uso ejemplar. Por último, es de suma importancia comprender las decisiones generadas por el sistema. Este aspecto centrado en el ser humano se logra educando a los usuarios sobre los mecanismos y la lógica detrás del proceso de toma de decisiones del sistema.

Los más comunes son:

1. **Modelos Intrínsecamente Explicables**: Algunos algoritmos son naturalmente más fáciles de interpretar, como los árboles de decisión, los modelos lineales o los modelos basados en reglas. Estos modelos ofrecen transparencia porque sus estructuras permiten una interpretación directa.

2. **Explicabilidad Post-Hoc**: En este enfoque, la explicabilidad se añade después de que el modelo haya sido entrenado. Es común en algoritmos complejos como las redes neuronales profundas, que son difíciles de interpretar por sí mismos. Herramientas como LIME (Local Interpretable Model-Agnostic Explanations) o SHAP (Shapley Additive Explanations) se utilizan para crear explicaciones sobre cómo el modelo toma decisiones en casos específicos.

3. **Descomposición de Modelos**: Algunas técnicas dividen el modelo en partes más pequeñas para analizar cómo contribuyen los diferentes componentes al resultado final. Esto es útil en redes neuronales profundas donde cada capa puede representar una abstracción distinta.

4. **Visualización de Características**: Especialmente en modelos basados en redes neuronales, se pueden crear representaciones visuales que expliquen qué partes de los datos están influenciando más las decisiones del modelo. Por ejemplo, en el procesamiento de imágenes, los mapas de calor pueden destacar qué áreas de una imagen contribuyen más a una predicción.

Métodos Populares de XAI:

Antes de entrar en detalles, es útil hacerse una idea general de los métodos de interpretación y de cómo funcionan, tratando de clasificarlos. Las dos clasificaciones siguientes son las más comunes que se encuentran en la literatura. (LIME vs. SHAP, s.f.) (Explainable AI: A Comprehensive Review of the Main Methods, s.f.)

Las técnicas de explicabilidad pueden ser divididas por diferentes categorías.

1.Categorización según **localidad**:

- **Global:** explican el modelo en general, señalando sus reglas genéricas de funcionamiento sin tener en cuenta ningún conjunto o dato concreto.

- **Local:** Explican para cada dato individual cómo razonó el modelo y las reglas que llevaron a un determinado resultado. Por ejemplo, pasando una imagen (o una colección de imágenes) del rostro de una persona en una red de clasificación y comprobar si el modelo utiliza las señales

correctas en la imagen (por ejemplo, nariz, labios, cejas, etc.) para llegar a la predicción correcta.

2. Categorización basada en **especificidad**:

- **Específico del modelo:** como sugiere su nombre, un método específico del modelo solo se puede aplicar a un grupo específico de modelos. Por ejemplo, si se emplease un modelo que fuese un regresor lineal, simplemente comparando los pesos del modelo como medidas de la importancia de las características, lo cual es completamente intuitivo, se podría obtener que parámetros tienen un mayor impacto en la clasificación. Sin embargo, este enfoque no funcionaría para redes neuronales de propagación hacia adelante, ya que hay muchas no linealidades entre las características de entrada al modelo y las predicciones de salida del modelo. Por lo tanto, no podría usarse la misma lógica empleada para el modelo de regresión lineal.

- **Independiente del modelo:** un método de explicación independiente del modelo se puede aplicar a cualquier modelo de aprendizaje automático. Cuando se utiliza un método de este tipo, el modelo se considera completamente como una función de caja negra y no se requiere información sobre la estructura interna del modelo. Lo único que necesitamos saber es que el modelo es una función que proporciona una salida para cada entrada válida que recibe.

Algunas de las metodologías más conocidas son:

LIME (Local Interpretable Model-agnostic Explanations): LIME es una técnica que busca interpretar cualquier modelo de IA al crear modelos más simples que se comportan de manera similar al modelo original en instancias locales. En lugar de tratar de explicar todo el modelo, LIME se enfoca en explicar una predicción específica generando un modelo interpretable solo para esa predicción particular. LIME ha sido diseñado para ser aplicado localmente.

SHAP (Shapley Additive Explanations): SHAP es un enfoque que utiliza teoría de juegos para asignar valores a cada característica de entrada con el fin de entender su contribución a la predicción. Es conocido por ser un método consistente y unificador, ya que proporciona una forma de medir la importancia de las características de manera justa, utilizando los valores de Shapley del juego cooperativo. SHAP se puede utilizar tanto global como local.

Saliency Maps: Son una técnica utilizada en el campo del procesamiento de imágenes. Estos mapas resaltan las áreas de la imagen que tuvieron un mayor impacto en la decisión de un modelo, como una red neuronal convolucional (CNN), al realizar una tarea específica (por ejemplo, clasificación de imágenes).

Modelos Lineales Generalizados (GLM): Son modelos intrínsecamente

interpretables que pueden utilizarse en ciertas situaciones para mantener la simplicidad en la explicabilidad. Si bien no son tan poderosos en términos de precisión en comparación con los modelos complejos como las redes neuronales, son fáciles de interpretar.

DeepLIFT: Es una técnica que calcula las contribuciones de cada neurona a la salida de una red neuronal en comparación con una referencia. Esto permite a los desarrolladores comprender qué características (input) influyeron más en la predicción.

Desafíos de la XAI:

Los desafíos de la Inteligencia Artificial Explicable (XAI) son variados y complejos. Uno de los principales retos es encontrar un equilibrio adecuado entre precisión y explicabilidad. Los modelos más avanzados, como las redes neuronales profundas o los bosques aleatorios, tienden a ser más precisos, pero su complejidad dificulta que sean comprensibles, lo que genera una tensión entre alcanzar mayor exactitud y hacer que el modelo sea fácil de interpretar. Otro desafío importante es la seguridad y la confianza en las explicaciones proporcionadas por estos sistemas. Aunque los modelos XAI pueden ofrecer explicaciones, no siempre se garantiza que estas sean correctas o seguras. Existe el riesgo de que una explicación parezca plausible, pero en realidad sea incorrecta, lo que podría dar lugar a decisiones equivocadas. Asimismo, el sesgo en las explicaciones es otro obstáculo. Las técnicas de XAI pueden generar explicaciones que resalten características aparentemente importantes, pero que en realidad no lo son, lo que puede confundir a los usuarios. Por último, la complejidad en la presentación de las explicaciones es un aspecto crítico. No todos los usuarios tienen el mismo nivel de formación técnica, por lo que adaptar las explicaciones al público objetivo resulta complicado. Una explicación que sea comprensible para un ingeniero puede no ser útil para un médico o un abogado, lo que añade otra capa de dificultad en la comunicación de los resultados de estos modelos.

Aplicaciones de la XAI:

Las aplicaciones de la Inteligencia Artificial Explicable (XAI) abarcan diversos sectores. En el campo de la medicina, la explicabilidad es esencial, especialmente en diagnósticos basados en IA. Los médicos deben poder confiar en las recomendaciones del sistema, por lo que, por ejemplo, un sistema que detecta tumores en imágenes médicas debe justificar su análisis señalando las áreas específicas que lo llevaron a identificar un tumor. En el sector financiero, la XAI es igualmente relevante, ya que, en procesos como la concesión de préstamos o la evaluación de crédito, los sistemas de IA deben explicar de forma clara por qué se aprobó o denegó un préstamo, cumpliendo con las normativas vigentes y garantizando la transparencia. En el ámbito de los automóviles autónomos, la XAI es fundamental para entender las decisiones que tomó el vehículo, sobre todo en caso de accidente, lo cual es crucial para garantizar tanto la seguridad como la responsabilidad legal. Finalmente, en recursos humanos, la XAI desempeña un papel importante en los sistemas de IA utilizados para reclutamiento o evaluación de empleados. Es necesario evitar sesgos discriminatorios y poder explicar de manera transparente por qué un candidato

fue seleccionado o rechazado.

Futuro de la XAI:

El campo de la XAI está en constante evolución. Lamentablemente, en la actualidad la mayor parte de los esfuerzos de la investigación en XAI están orientados a los investigadores y usuarios profesionales de los algoritmos, mientras que prácticamente no hay avances en las explicaciones dirigidas a usuarios finales. En este momento se trabaja en dos grandes líneas: hacer algoritmos explicables por diseño y poder explicar algoritmos opacos en el diseño. En el segundo caso, se apuesta por técnicas visuales como los mapas de calor, también por técnicas semánticas que van dotando de significado a las capas de los algoritmos o por técnicas experimentales en las que se suministran distintos tipos de datos de entrada al algoritmo y se analiza el resultado para extraer la relación entre X tipo de dato de entrada — Y resultado.

Algunas tendencias emergentes incluyen:

- **XAI interactiva**: A medida que las aplicaciones de IA se vuelven más interactivas, es probable que surjan herramientas que permitan a los usuarios explorar las explicaciones de manera dinámica, preguntando al modelo "qué pasaría si" se cambiaran ciertos inputs.
- **Automatización de la Explicabilidad**: La generación automática de explicaciones personalizadas según el nivel de conocimiento técnico del usuario también está ganando terreno.
- **Desarrollo de estándares y métricas**: Con la adopción generalizada de la XAI, es probable que se desarrollen estándares y métricas para evaluar la calidad y la claridad de las explicaciones, lo que ayudará a comparar diferentes enfoques y asegurar la rendición de cuentas.

En el ámbito XAI queda mucho por hacer, pero es un tema cada vez más presente para responder a los distintos tipos de necesidades de transparencia, y esperamos que pronto se puedan ver sus primeras aplicaciones a las necesidades de transparencia de los usuarios.

6.2.2 XAI en la sanidad

La inteligencia artificial explicable (XAI) se presenta como una solución prometedora para abordar los desafíos de implementación de la IA/ML en la atención médica. Implica la creación de modelos de inteligencia artificial con una estructura de gobernanza, como herramientas o sistemas, que permitan a los usuarios finales comprender los resultados y los productos de los modelos de aprendizaje automático (ML) desarrollados, lo cual es crucial para este ámbito.

Como señaló Nigam Shah, profesor de Medicina (Informática Biomédica) y de Ciencia de Datos Biomédicos en la Universidad de Stanford, la interpretación de "explicable" en inteligencia artificial, es específica del contexto. La interpretabilidad de la IA se puede clasificar en tres tipos: explicabilidad de los ingenieros, explicabilidad causal y explicabilidad que genera confianza, lo que pone de relieve las diversas facetas de la comprensión de los sistemas de este

tipo. Así pues, en el diagnóstico médico, la explicabilidad causal puede ser crucial para comprender por qué un modelo recomienda un tratamiento específico, mientras que la explicabilidad que genera confianza se vuelve esencial para infundir confianza en los médicos que utilizan herramientas de diagnóstico impulsadas por IA. (Should AI Models Be Explainable? That depends., s.f.)

El Dr. Shah también destacó que la resolución del enigma de la IA en la atención médica requiere el establecimiento de colaboraciones interdisciplinarias. Esto implica reunir a expertos en este campo, incluidos estadísticos e ingenieros, profesionales de la salud, especialistas en ética y formuladores de políticas. Los componentes clave de este esfuerzo colaborativo incluyen la definición de estándares para una IA transparente en la atención médica, la creación de interfaces fáciles de usar para transmitir conocimientos impulsados por la misma y la educación tanto de los médicos, como de los pacientes, sobre las capacidades y limitaciones de este tipo de sistemas.

6.2.3 **Grad-CAM**

Grad-CAM es una técnica de visualización y explicación de los resultados de modelos de redes neuronales, especialmente de redes convolucionales utilizadas en tareas de visión por computadora. Proporciona mapas de calor que resaltan las regiones de una imagen que son más importantes para la predicción de una clase en particular.
El propósito de Grad-CAM es ayudar a los investigadores y usuarios a entender qué características de una imagen son más importantes para que la red realice una clasificación. Es decir, responde a la pregunta: ¿Por qué el modelo tomó esta decisión?

Grad-CAM funciona aprovechando los gradientes de la clase objetivo que fluyen hacia la última capa convolucional de la red para calcular un mapa de activación ponderado. Los pasos generales de su funcionamiento son:

1. Selecciona la capa de interés: Normalmente, la última capa convolucional de la red, ya que es la más cercana a la salida de la predicción de la red. Y por tanto son las neuronas con una mayor relevancia en el resultado

2. Cálculo de los gradientes: Se calculan los gradientes de la predicción de una clase objetivo (por ejemplo, "gato") con respecto a las características de la última capa convolucional. Estos gradientes indican qué características están influyendo en la predicción de la red.

3. Ponderación de las características: Los gradientes se usan para ponderar las activaciones de las neuronas en la capa seleccionada. En otras palabras, la importancia de cada neurona en la última capa convolucional es ponderada según su contribución a la predicción.

4. Generación del mapa de calor: Finalmente, el resultado se promedia y se aplica un mapa de calor sobre la imagen original, lo que visualmente destaca las áreas que tuvieron un mayor impacto en la predicción.

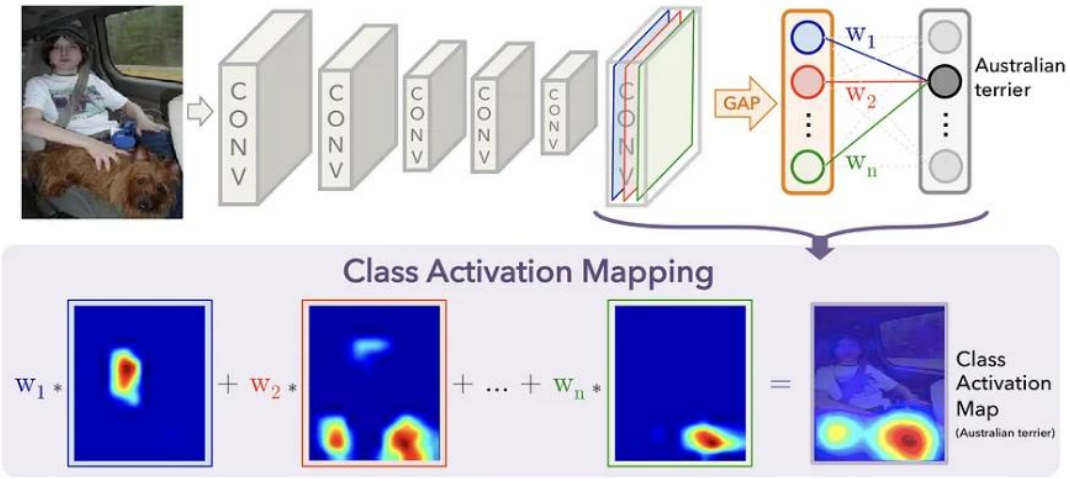

Figura 18: Diagrama de funcionamiento de Grad-CAM

Grad-CAM resulta útil en diversos escenarios. En primer lugar, es una herramienta valiosa para la detección de sesgos en los modelos. Si el modelo se enfoca en áreas irrelevantes o inapropiadas de una imagen para tomar decisiones, asimismo permite identificar estos problemas, lo que ayuda a mejorar la precisión y el sesgo cometido por el sistema. Además, facilita la interpretabilidad de las redes neuronales, ya que contribuye a entender cómo el modelo "ve" y procesa la información, haciendo más comprensible su funcionamiento. Por último, es útil en la detección de errores. Cuando el modelo comete errores de clasificación, Grad-CAM ayuda a los ingenieros a identificar qué partes de la información están siendo malinterpretadas, lo que facilita la corrección y ajuste del modelo.
También, es fácilmente aplicable a una amplia gama de redes CNN como; con capas completamente conectadas, utilizadas para salidas estructuradas o utilizadas en tareas con entradas multimodales o aprendizaje de refuerzo. (A Review of Different Interpretation Methods (Part 1: Saliency Map, CAM, Grad-CAM), s.f.)

Ventajas:

- Intuitivo: Permite visualizar de manera intuitiva las áreas de la imagen que están influenciando la clasificación.

- Aplicabilidad: Funciona bien con cualquier red convolucional y es fácil de integrar en modelos preentrenados.

- Post-hoc: Grad-CAM se puede aplicar después de entrenar el modelo, no necesita modificaciones en el proceso de entrenamiento.

Limitaciones:

- Resolución: El mapa de calor generado por Grad-CAM tiene la misma resolución que las activaciones de la capa convolucional seleccionada, lo

que puede llevar a que sea borroso o de baja resolución.

- Dependencia de capas específicas: Depende de las capas convolucionales profundas, lo que limita su aplicabilidad a otras arquitecturas que no tienen este tipo de capas.

- No es causal: Aunque Grad-CAM muestra correlaciones entre las activaciones y las predicciones, no necesariamente explica una relación causal.

Grad-CAM tiene diversas aplicaciones en distintos campos. En medicina, es especialmente útil para la interpretación de imágenes médicas, como rayos X o resonancias magnéticas, ya que permite a los médicos verificar que los modelos de IA se concentren en las regiones anatómicas correctas al hacer un diagnóstico. En el ámbito de los automóviles autónomos, facilita la comprensión de la toma de decisiones en los sistemas de visión. Esto es clave para saber qué partes de la imagen influyen en decisiones críticas como frenar o acelerar. Además, en el reconocimiento de objetos, con esta herramienta resulta valioso para entender por qué un sistema de detección identifica ciertos objetos, ofreciendo una mayor transparencia en el proceso de clasificación.

Con la idea de superar las limitaciones que presenta Grad-CAM, se han desarrollado algunas variantes como Grad-CAM++ que mejoran sus capacidades, especialmente útil parar aquellas imágenes que contienen múltiples objetos o cuando estos tienen una forma compleja o son muy pequeños.

Por lo que Grad-CAM++ se presenta como una alternativa con un mapa más refinado y preciso. Teniendo como mejoras principales; la sensibilidad a múltiples objetos y la precisión, consiguiendo mejorar la resolución espacial del mapa, haciendo que los bordes y los detalles finos de los objetos lleguen a ser más precisos.

La diferencia principal en cuanto a su implementación es debido a que para el cálculo de los pesos se utiliza una fórmula más compleja que incluye la segunda derivada en función de la perdida con respecto a las activaciones. Estos pesos asignados a las activaciones dependen de la primera y segunda derivada de la salida de la red con respecto a la activación en cada píxel. Permitiendo que se distribuya de manera más fina la importancia entre diferentes características, lo que ayuda a detectar de manera más precisa las áreas que son relevantes para la predicción.

Pero a pesar de todo esto también hay que tener en cuenta que, como principal desventaja con respecto al método original, es su mayor coste computacional. Y

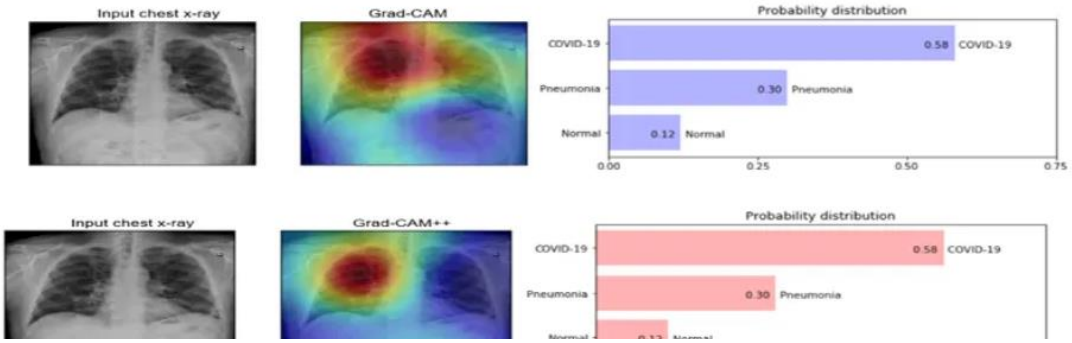

Figura 19: Comparación entre los distintos métodos de Grad-CAM

Para nuestro caso se ha optado por emplear Grad-CAM para realizar las pruebas. Ya que si se tienen en cuenta las ventajas que se consiguen con la versión ++, no aporta una gran mejoría. Para el caso en cuestión no hay que tratar con la detección de distintos objetos en una imagen o que estos sean de formas muy complejas. De este modo evitamos ese sobre gasto de recursos

Optando por otra vertiente y tipo de algoritmo, se ha usado otra técnica como son los algoritmos de detección de objetos en imágenes. Con la idea de poder encontrar, además del resultado de la clasificación, en qué parte de la imagen se encuentra dicho elemento que ha producido que se tome dicho resultado.

En este caso se ha optado por el algoritmo de You Only Look Once (YOLO), el cual ya tiene cierto recorrido y se ha ganado su popularidad para este tipo de finalidades.

6.2.4 **YOLO**

YOLO es una técnica de detección de objetos que realiza predicciones en una sola pasada a través de la imagen, es decir, detecta múltiples objetos simultáneamente. A diferencia de otros métodos como las Redes Neuronales Convolucionales (R-CNN), que requieren pasar por cada región de interés varias veces, YOLO y la imagen completa de una sola vez, lo que lo hace extremadamente rápido. (Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks, s.f.)

El algoritmo fue propuesto por Joseph Redmon y su equipo en 2015, el cual ha experimentado una notable evolución a lo largo de los años, adaptándose y mejorando en cada nueva versión tanto en precisión como en velocidad y eficiencia. Desde su primera versión, YOLOv1, que introdujo la innovadora idea de dividir una imagen en cuadrículas para predecir objetos y sus cajas delimitadoras, el modelo ha ido refinándose con el tiempo.
En versiones posteriores, como YOLOv2, se incorporaron mejoras clave como los Bounding Box Priors, optimizando la precisión de las predicciones mediante técnicas como el K-Means clustering. Esta versión también amplió considerablemente el número de clases detectables. La llegada de YOLOv3 trajo consigo la capacidad de detectar objetos más pequeños y complejos gracias a una arquitectura de red más profunda y la detección en múltiples escalas, marcando un hito en la detección de objetos a gran nivel de detalle.
La evolución continuó con YOLOv4, que se destacó por su mayor velocidad y precisión en comparación con otros métodos de la época, a la vez que optimizó el proceso de entrenamiento para ser más accesible con hardware de consumo, como las GPU convencionales. Aunque YOLOv5 no fue desarrollado por los autores originales, se ha convertido en una de las versiones más populares debido a su eficiencia y facilidad de implementación, convirtiéndose en un estándar en tareas prácticas de detección de objetos. Además de la gran documentación que puede encontrarse fácilmente de su implementación y la gran variedad de proyectos públicos de su uso aportados por una gran

comunidad que se ha implicado en él. (Glenn Jocher et al. (2020). YOLOv5 by Ultralytics, s.f.)

Las versiones más recientes, YOLOv7 y YOLOv8, han seguido este camino de optimización, enfocándose especialmente en mejorar la arquitectura y reducir el tamaño de los modelos para hacerlos más eficientes en dispositivos de baja potencia, como drones o cámaras en tiempo real. A través de estas sucesivas actualizaciones, YOLO ha mantenido su relevancia en el campo de la visión por computadora, adaptándose a las nuevas demandas tecnológicas y aplicaciones prácticas.

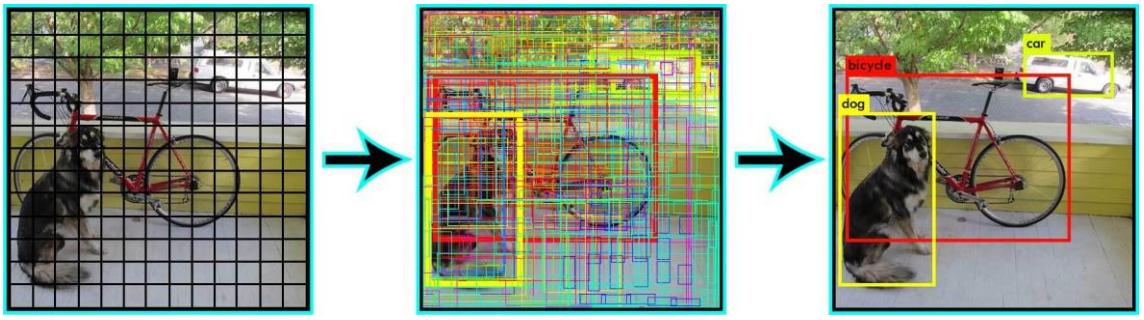

Figura 20: Proceso de detección de objetos por YOLO

Los principios fundamentales del algoritmo, se basa en la siguiente serie de pasos:

a. División de la imagen
YOLO divide la imagen de entrada en una cuadrícula de tamaño SxS. Cada celda en la cuadrícula es responsable de predecir:

- Una o más cajas delimitadoras (cuadros delimitadores) que incluyen la ubicación (coordenadas x, y, altura y ancho) y la confianza de que hay un objeto presente.
- Una probabilidad de que el objeto pertenezca a una clase determinada (por ejemplo, perro, coche, etc.).

b. Predicciones simultáneas
A diferencia de otros enfoques, YOLO hace todas las predicciones de manera simultánea: predice las cajas delimitadoras y las clases en un solo paso. Este enfoque único reduce considerablemente el tiempo de procesamiento, lo que permite realizar detección en tiempo real.

c. Uso de anclas
Las versiones más recientes de YOLO usan anclajes, que son conjuntos predefinidos de tamaños y formas de cuadros delimitadores que mejoran la precisión de la detección. Al ajustar estos anclajes, se mejora la capacidad de detectar objetos de diferentes tamaños.

d. Función de pérdida
YOLO utiliza una función de pérdida que equilibra tres aspectos:

1. **Error de clasificación**: Para predecir la clase correcta de objeto en cada caja.
2. **Error de coordenadas**: Para ajustar la precisión de las cajas delimitadoras.
3. **Confianza**: Para predecir si la caja contiene un objeto o no.

e. Intersección sobre Unión (IoU)

Un aspecto fundamental en YOLO es el uso de la métrica IoU (Intersection over Union) para evaluar qué tan bien se ajustan las cajas delimitadoras predichas a los objetos reales en la imagen.

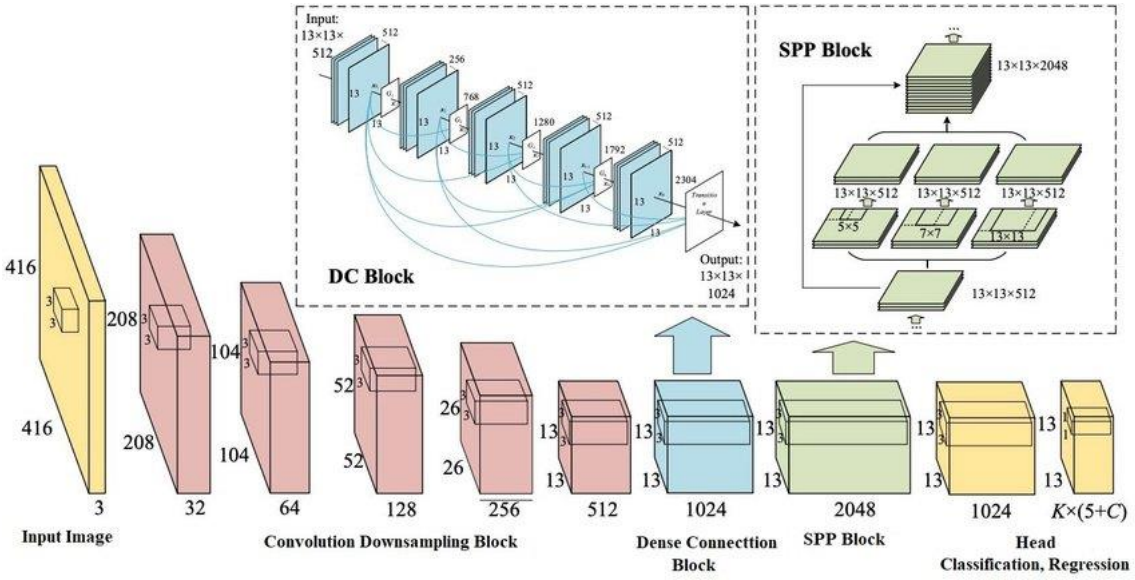

Figura 21: Arquitectura de la red de YOLO

YOLO presenta varias ventajas destacadas que lo hacen ideal para una amplia gama de aplicaciones. Su principal fortaleza es la velocidad, permitiendo la detección de objetos en tiempo real, lo cual es esencial en escenarios donde el tiempo de respuesta es crucial, como en la conducción autónoma o la videovigilancia. Además, tiene la ventaja de considerar la imagen completa al realizar las predicciones, lo que permite una mejor comprensión del contexto global. Las versiones más recientes, como YOLOv5 y posteriores, también se destacan por su facilidad de uso, ya que son simples de implementar y personalizar con conjuntos de datos específicos.
Sin embargo, también presenta algunas desventajas. Tiene dificultades para detectar objetos muy pequeños o que están muy próximos entre sí, debido a la división de la imagen en cuadrículas, donde cada celda solo puede detectar un objeto a la vez. Además, aunque es una técnica muy rápida, en ciertos casos puede no alcanzar la precisión de otros algoritmos más complejos, como Faster R-CNN, que pueden ofrecer un rendimiento superior en tareas de detección más exigentes.

YOLO tiene múltiples aplicaciones en diversos campos debido a su capacidad para realizar detección de objetos en tiempo real. En vehículos autónomos, se

utiliza para identificar obstáculos y otros vehículos de manera instantánea, garantizando una conducción más segura. En el ámbito de la videovigilancia, permite reconocer comportamientos sospechosos, así como identificar personas o vehículos en las cámaras de seguridad. También es clave en el reconocimiento en tiempo real, siendo usado en aplicaciones como la realidad aumentada, videojuegos o interfaces interactivas que requieren una identificación rápida de objetos. Además, los drones que operan en tiempo real se benefician de la rapidez de los algoritmos de YOLO para detectar terrenos, personas o identificar objetos en entornos complejos.

Al final esta técnica puede presentarse como una gran alternativa frente a otras de detección de objetos. Ya que, presenta ser mucho más veloz, pero pudiendo ser superado en cuanto a precisión.

Algoritmo	**Velocidad**	**Precisión**	**Tiempo de procesamiento**
YOLO	Muy rápido	Alta	Tiempo real
Faster R-CNN	Medio	Muy alta	Lento
SSD	Rápido	Media	Casi en tiempo real
Mask R-CNN	Lento	Muy alta	Lento

Tabla 7: Comparación entre distintos modelos de detección de objetos

6.3 DESARROLLO

Esta fase se va a dividir en varias partes de las que ha constado la realización del proyecto. Especificando en cada una de ellas varios aspectos como su funcionamiento o las decisiones tomadas a la hora de que metodología usar, además de algunos otros aspectos más específicos a destacar.

6.3.1 Adquisición de los datos (Características de los datos)

Es la fase inicial y la base para poder realizar las siguientes etapas del proceso en la implementación.

Todos los datos que han sido adquiridos provienen de distintas fuentes que garantizan en cierta medida, su calidad y validez, para ser empleados en sistemas como el que se trata en esta memoria. En la mayoría de los casos, los datos provienen de bases de datos que se hacen públicos para poder ser empleados para propósitos de investigación o de aprendizaje. Estas bases se formar a partir de instituciones públicas principalmente, como centros de investigación, departamentos de colaboración de las universidades, centros médicos. Lo que nos ofrece una cierta garantía de valor para hacer uso de los mismo. Cada uno de los datos, provienen de personas reales que han tenido que someterse a algún tipo de diagnóstico o prueba médica y que, con su consentimiento, han ofrecido la información para que puedan ser publicados en estas bases de datos. Pero manteniendo siempre su anonimato al ser publicados, solo teniendo esta información la institución donde el paciente se realizó las pruebas y firmó el acuerdo de consentimiento para su publicación.

Las plataformas que principalmente se han empleado son; Kaggle y Roboflow. Ambas albergan una gran cantidad de datos y de multitud de tipos. Y en la mayoría de los casos sus fuentes provienen de los medios mencionados. Asimismo, están diseñadas para crear una comunidad que a través de las herramientas que ofrecen se pueda aprender del ámbito de la inteligencia artificial, ya que dota de una serie de recursos tanto para adquirir dichas habilidades y siempre en conjunto con una comunidad que activamente publica sus experimentaciones. Las dos cuentan tanto con proyectos realizados que pueden servir de guía o como material de aprendizaje para conocer cómo se aplican, desde ciertas técnicas o algoritmos, hasta todo un proceso de manejar los datos y ver la conclusión que se ha obtenido con ellos. Al ser unas plataformas tan activas, la cantidad de contenido es casi ilimitada y siempre en crecimiento lo que es de gran utilidad para encontrar infinidad de recursos que puedan servir de ayuda para las necesidades que uno pueda llegar a tener.

En general se ha optado por buscar entre ambas plataformas, por todo lo comentado y por los conjuntos de datos que se proporcionan, ya que estos suelen estar preparados para emplearse en sistemas de inteligencia artificial.

Los datos que se han empleado para el sistema corresponden a imágenes médicas, donde se muestran una radiografía o ecografía de los pulmones.

Aunque para nuestro caso de estudio, podría haber sido perfectamente cualquier otro organismo del cuerpo, puesto que la relevancia de la solución propuesta no se plantea como una solución a este tipo de tumor en específico, sino en un ámbito más general, pero para la experimentación, se ha optado por enfocarse solo en un tipo de cáncer.

Dado que se han empleado diversas técnicas para el proceso de clasificación y detección de los objetos, que se expondrá más adelante, la información correspondiente a cada imagen que se ha empleado se muestra de dos modos. Aquellos datos que vienen con su correspondiente etiquetado, el cual indica si en dicha imagen existe la presencia o no, de un tumor maligno. Y otros que además de conocer esta misma información, son provistos con unas anotaciones extras que indican dentro de la imagen en que ubicación se encuentra dicha presencia de tumor. Esto implica que para los casos en los que no hay presencia, nunca podrá haber este tipo de anotación.

```
clase,  posición x,  posición y,     ancho,      alto

    0, 0.544921875, 0.693359375, 0.0078125, 0.0078125
```

Figura 22: Esquema de las anotaciones de un archivo de YOLO

En la implementación llevada a cabo se ha probado con multitud de datasets distintos, ya que cada uno presentan unas características en concreto, ya sea la cantidad de datos presentes, la calidad en las imágenes proporcionadas, el número de clases etiquetadas, sin los datos presentaban anotaciones de localización, etc. Esto ha llevado a un proceso de profunda búsqueda y para muchos de ellos tener que realizar pruebas para comprobar si podía ser empleado.

Finalmente se ha optado por utilizar dos conjuntos de datos. Uno que está estructurado en dos carpetas que corresponden a cada una de las clases (Nodule, NonNodule) y que en cada una de estas se encuentran las imágenes correspondientes con dicho etiquetado, a este conjunto para las próximas veces que se menciones, se le denominará conjunto A. El otro conjunto está dividido por carpetas de test, train y validación. Las cuales a su vez se dividen en images y labels, que como sus nombres indican, almacenan las imágenes y los archivos que corresponden con cada imagen con las anotaciones tal como se muestra en la figura anterior, a este conjunto se le va a denominar B.

6.3.2 Procesamiento de los datos

Conociendo los datos que se emplean y las características que presentan, se procede a la fase en cual estos datos son tratados por varios procesos para que puedan ser adecuados a las fases posteriores para poder implementar la solución.

Dada la distribución disponible por el conjunto A, a este se le ha creado las etiquetas correspondientes de forma que se han generado dos listas, que vamos a denominar labels e imágenes. Una la cual contiene las rutas de todas las imágenes, como su mismo nombre indica, de forma ordenada, tanto aquellas pertenecientes a la clase Nodule como a las Non Nodule. Y otra lista, labels, que se ha creado para tener las etiquetas relacionadas con cada una de estas imágenes. Así pues, cada lista está ordenada lo cual es de relevancia, ya que, la posición de cada elemento en ambas está relacionado de forma directa. Así, por ejemplo; el elemento de la primera posición de la lista labels, corresponde con el valor de la clase a la ruta de la imagen de la primera posición de la lista imágenes. Esto es importante ya que para la fase posterior de entrenamiento de los modelos de inteligencia artificial es necesario tener las etiquetas de los datos a usar.

Teniendo estas referencias a todos los datos del conjunto, se hace una división en dos para tener un conjunto de entrenamiento y otro de validación. La división se hace en un 80% para entrenamiento y un 20% para validación sobre el conjunto total. Esta separación se hace de forma aleatoria, pero intentando mantener cierta homogeneidad para que ninguno de estos subconjuntos se quede sin datos de un tipo o que esté muy desproporcionado en cuanto a la cantidad de cada uno. Esta división se hace gracias a una función, perteneciente al paquete de scikit learn, que se encarga de obtener los datos repartidos con estas condiciones automáticamente, solo teniendo que indicar que datos quieres separar.

Con los datos divididos y correctamente etiquetados, se procede a crear un generador de imágenes. Esta es una práctica muy común cuando se trabaja con redes neuronales para la clasificación de imágenes, especialmente al ser desarrolladas con bibliotecas como Keras o TensorFlow.

Para implementar este generador, se usa ImageDataGenerator (una clase de Keras) que facilita el preprocesamiento y aumento de datos, de imágenes. Su objetivo principal es generar lotes de datos de imágenes que se pueden alimentar a un modelo de aprendizaje automático, mientras que al mismo tiempo puede aplicar varias transformaciones a las imágenes de manera eficiente, todo en memoria.

Esto se presenta de gran utilidad por diversas razones. Como el preprocesamiento que se les aplica a las imágenes, permitiendo realizar transformaciones básicas antes de que sean enviadas al modelo, tales como reescalar, cambiar el tamaño, aplicar normalización, etc. Y también consigue mejorar el rendimiento de un modelo, especialmente cuando se tiene una cantidad limitada de datos, ya que es capaz de generar nuevas imágenes a partir de los originales, aplicando transformaciones como rotaciones, traslaciones, zoom, cambios de brillo, y más. Esto simula un conjunto de datos más amplio y diverso sin necesidad de recopilar más imágenes.

El uso de esta técnica aporta mejoras cuando se trata de implementar un sistema de clasificación de imágenes. Ya que consigue que se mejore la generalización del modelo. Al aumentar la cantidad de datos, permite al modelo ver más variedad de ejemplos durante el entrenamiento, lo que ayuda a reducir el sobreajuste (*overfitting*) y mejora la capacidad del modelo para generalizar a nuevas imágenes que no ha visto antes. También al aplicar transformaciones como rotaciones, traslaciones, zooms o inversiones hace que el modelo sea más robusto, ya que aprende a identificar los objetos en condiciones variadas. Esto es crucial en problemas de clasificación de imágenes, donde el modelo debe ser capaz de reconocer el mismo objeto desde diferentes perspectivas o condiciones de iluminación. Y, asimismo, dado que permite cargar lotes de imágenes en memoria en lugar de todo el conjunto de datos de una sola vez, facilita la carga de grandes conjuntos de datos. Esto es especialmente útil cuando se trabaja con imágenes de alta resolución o conjuntos de datos muy grandes, ya que evita problemas de memoria.

En concreto el generador aplica sobre el conjunto un reescalado, las normaliza y aplica una transformación aleatoria sobre ellas como un aumento. Además de adaptar su dimensión a la empleada como entrada para el modelo. Y las agrupa en batches de 32 imágenes para el entrenamiento.

Esta técnica solo se aplica sobre el conjunto de datos A, puesto que, en el otro conjunto, el modo de entrenamiento se efectúa de un modo diverso, no requiere de hacer este tratamiento sobre los datos. Solo es necesario tener las imágenes estructuradas como se ha definido para el conjunto B. Y escribir un archivo **yml** que indique como están distribuidos cada uno de los subconjuntos de datos, el número de clases a detectar y el nombre de dicha clase.

```
train: ../train/images
val: ../valid/images
test: ../test/images

nc: 1
names: ['0']
```

Figura 23: Esquema del archivo yml de YOLO

Para el tratamiento de las imágenes se ha usado la librería de OpenCV en Python (cv2). Es una de las herramientas más populares y poderosas para procesamiento de imágenes y visión por computadora. Diseñada para ejecutar tareas como el reconocimiento de objetos, el procesamiento de imágenes y video, la detección de movimientos, y la manipulación de píxeles, entre otras funciones.

Se ha empleado para hacer todas las funciones de cargar, visualizar y guardar las imágenes en el formato especificado, el cual siempre se ha empleado **jpeg**. También para hacer operaciones sobre ellas de igual forma que como se aplica internamente en el generador anterior, como normalizar, redimensionar las

imágenes ajustándolas a la entrada de la red neuronal en la que se vaya a utilizar o modificaciones de estas como adaptar el color, el cual puede variar en función del formato empleado.

Posteriormente se ha creado un conjunto, el cuál será de test, y se usará para evaluar los resultados y hacer las experimentaciones para ambas metodologías que se implementan. Este conjunto está formado a partir de los datos del conjunto A y el conjunto B de forma se puede analizar de un modo más preciso los modelos puesto que tendrán que clasificar imágenes que son pertenecientes a fuentes distintas. Este nuevo conjunto, utilizará el conjunto de test ya definido por el conjunto B, más algunos datos de su conjunto de validación. A su vez contendrá imágenes pertenecientes al conjunto A de ambas clases. En total consta de un total de 200 imágenes correspondiendo el 60% al conjunto B y el 40% al conjunto A. Puesto que los datos del conjunto A no viene con anotaciones de ningún tipo, se generar unos ficheros para cada una de las imágenes que indican la clase a la que pertenece. De esta forma se crea este conjunto con una carpeta con todas las imágenes provenientes de los conjuntos anteriores en las proporciones mencionadas y otra carpeta que correspondería a la información de etiquetado y en el caso de las que corresponden a las del conjunto B, con la información adicional de la localización de los tumores.

La justificación por la cual se ha seleccionado una porción mayor de datos provenientes del conjunto B, es debido a que estas imágenes cuentan con las anotaciones adicionales, por lo que son de mayor utilidad en fases posteriores cuando se quiera comprobar y hacer cálculos de la efectividad de localizar de los modelos. Algo que no puede comprobarse con los datos que pertenecen al conjunto A.

6.3.3 Implementación de las redes neuronales de clasificación (Modelos y entrenamiento)

Para esta parte se ha optado por emplear tres arquitecturas de redes distintas, para comprobar con cual se consigue obtener unos mejores resultados para posteriormente utilizarla para la siguiente fase.

El diseño de la entrada y salida es compartido para todas las arquitecturas. Siendo la entrada una capa input con dimensión 224x224, este es el motivo por el cual en el procesamiento de los datos hay que redimensionar las imágenes para que puedan ser aptas a esta capa. La salida por su parte es una capa densa, con dos salidas y con activación softmax. Esto indica que cada salida representa a cada una de las dos clases que se clasifica. La neurona que tenga un mayor valor de activación, es la que se da como la clase predicha por la red neuronal.

A la hora de implementar un sistema de redes neuronales, hay que tener en cuenta los hiperparámetros como se mencionó en el apartado anterior. Por lo que se tiene que asignar unos valores determinados para cada uno de ellos, con el fin de conseguir un sistema óptimo. Así se consigue obtener un modelo que sea

aceptable en cuanto a su porcentaje de predicción y error, y que no requiera de un tiempo excesivo para conseguirlo

Para que el análisis pueda ser más riguroso y justo, se ha decidido optar por usar dichos hiperparámetros con los mismos valores para todas las pruebas realizadas con los distintos modelos. La elección de los valores de los parámetros ha sido tras comprobar que se consigue un buen resultado al usarlos en dos modelos distintos.

Los valores empleados para dichos hiperparámetros a lo largo de todas las implementaciones son los siguientes:

- ➔ **Batch size = 32.**

Este valor suele ser muy típico empleado en un entrenamiento de redes, especialmente cuando no se cuenta con un conjunto de datos muy amplio. Aunque con este valor conlleva a que cada época de entrenamiento requiera de tiempos muy altos, ya que son más pasos por épocas al dividir en menos paquetes el conjunto de entrenamiento. Un valor muy alto dificulta mucho realizar un buen ajuste de los pesos, mientras que un valor bajo necesita de más épocas para conseguir un mejor ajuste. Por lo que con este valor se consigue un buen balance entre ambos casos.

- ➔ **Early stop = 4.**

Es el número de épocas, que tienen que pasar, sin que mejore el valor de val_loss para que se detenga el entrenamiento. Se estima que, si pasan este número de épocas sin mejorar, es debido a que no va a conseguir mucha mejoría a partir de muchas iteraciones siguientes o, lo más probable, que se haya alcanzado una situación de Overfitting

- ➔ **ModelCheckpoint**.

Con este parámetro se está indicando que en cada iteración compruebe si se ha conseguido mejorar el modelo, para ello comprueba si el valor de val_loss ha disminuido, si ha disminuido, este modelo es guardado automáticamente. Con esto se consigue que cuando se termine el entrenamiento, acabe registradose el modelo que ha conseguido un menor val_loss. De otro modo se almacenaría el último, el cual no garantiza que sea el mejor, de hecho, en muchas ocasiones suele estar sobre ajustado.

- ➔ **Epochs = 20.**

Este valor realmente no repercute mucho en el entrenamiento ya que para lo mayoría de entrenamientos se detienen antes de alcanzarlo debido al parámetro anterior. Y en caso de llegar, ya se ha alcanzo un buen nivel de ajuste.

- ➔ **Validation.**

Para la validación de cada iteración se usan el 20% del conjunto de los datos

cuando se aplicó la división, como se comentó en la sección del procesamiento de datos.

- ➔ **Learning rate = 'adam'.**

Se ha empleado el optimizador Adam como learning rate, el cual es muy popular y eficiente. Este optimizador realiza un seguimiento de dos momentos: la media y la varianza de los gradientes, lo que permite que se ajuste automáticamente la tasa de aprendizaje para cada parámetro. Esto lo hace muy adecuado para problemas con grandes cantidades de datos o parámetros, y generalmente proporciona una convergencia más rápida y estable si se compara con otros optimizadores. Y es especialmente útil cuando se trabaja con arquitecturas profundas y con ruido en los gradientes, los cuales son muy común en redes neuronales convolucionales o recurrentes.

La solución propuesta se ha llegado probando entre 3 implementaciones distintas de redes neuronales. Una de ellas ha sido creando la propia arquitectura de la red y el resto de los modelos, han sido de los ya probados y que están publicados en el portal de Keras llamado Keras Applications, la cual les da soporte. Específicamente los modelos seleccionados para las pruebas son Xception y VGG16. Estos modelos vienen ya incluidos como paquetes dentro de la biblioteca de Keras y han sido seleccionados para ello, debido a que, han ganado competiciones donde se intenta mostrar modelos con los que se consiguen llegar a grandes resultados de predicción. El uso de estos ya implica la definición de la arquitectura de la red e incluso unos valores iniciales de los pesos, aunque estos últimos no son necesarios de usar.

La comparación entre cada uno de los modelos mencionados se va a llevar a cabo usando todas las métricas y herramientas mencionadas en la sección anterior de las redes neuronales.

Todas las redes neuronales son entrenadas y validadas con el conjunto de datos A que se mencionó previamente.

Modelo Secuencial

Se ha diseñado una red neuronal, escogiendo cada una de las capas utilizando para ello la librería de Keras.

La arquitectura de la CNN implementada consiste en una secuencia de capas convolucionales seguidas de capas de agrupación (pooling) para la extracción de características, con funciones de activación ReLU para la no linealidad. En concreto cuenta con tres capas de convolución junto con estas de agrupación. Después de las capas convolucionales, la red tiene una o más capas completamente conectadas para la clasificación final. El uso de capas de regularización como Dropout y técnicas como el lote de normalización ayuda a prevenir el sobreajuste. Empleando la capa de entrada y salida como se ha descrito antes.

- **Gráfico de entrenamiento**

Lo que se observa en este gráfico es que tras unas pocas épocas donde el error de validación está disminuyendo, se mantiene en torno a uno valores fijos sin presentar ninguna mejoría en las últimas épocas. Para el caso de la precisión es muy similar donde se ve que da una sensación de que se queda estancado y no consigue mejora. Esto es debido a que tras unas pocas épocas se inicia a producir overfitting como se ha comentado. Esto es debido a que el modelo al tener una capacidad tan potente consigue modelar a la perfección y tras pocas épocas al conjunto de entrenamiento. Es por ello por lo que, al evaluarlo con el conjunto de test consigue empeorar su resultado.

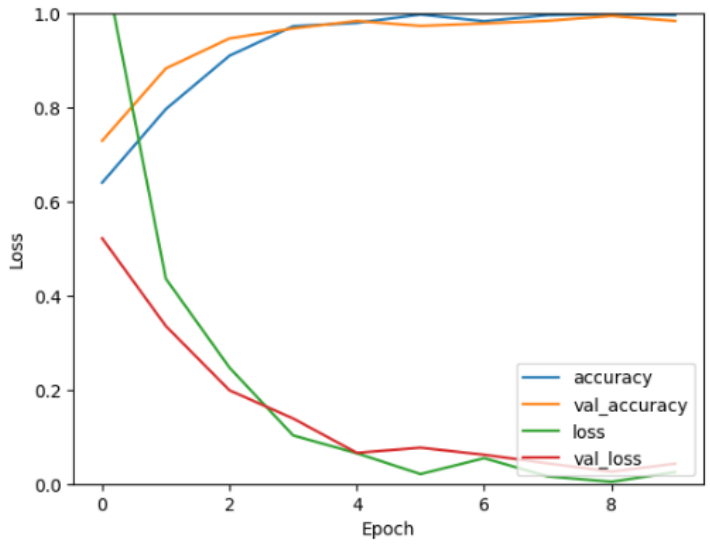

Figura 24 :Gráfico de evolución de entrenamiento de la red secuencial

- **Valores de las métricas**

Para este modelo se consigue los siguientes valores para las distintas métricas:

	Precision	**Recall**	**F1-Score**	**Accuracy**
Conjunto A	0.99	1.00	0.99	0.99
Conjunto Test	1.00	0.79	0.88	0.84

Tabla 8: Métricas del modelo secuencial

- **Matriz de Confusión**

Para este modelo se consigue también un buen porcentaje de acierto y al igual que en los anteriores el fallo que más comete es para el caso menos crítico.

Sobre el conjunto de test del entrenamiento del modelo

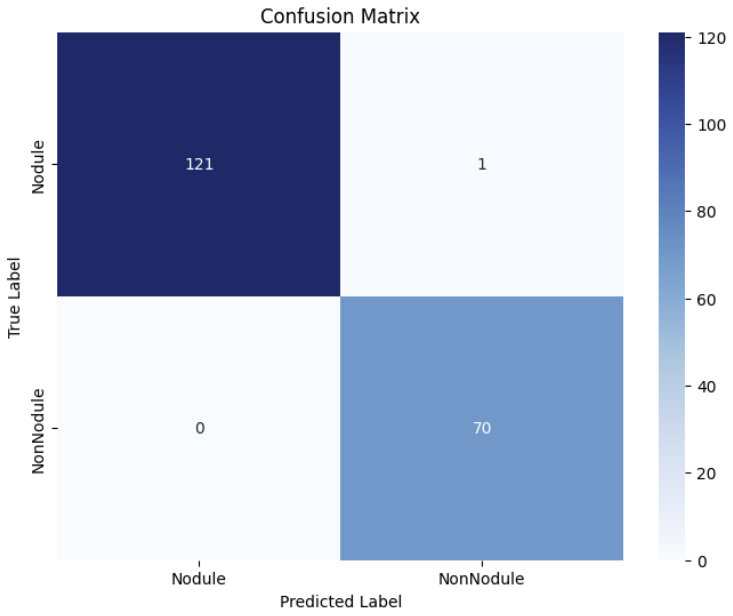

Figura 25: Matriz de confusión del modelo secuencial al set de train

Otra matriz sobre el conjunto de test para la evaluación del modelo

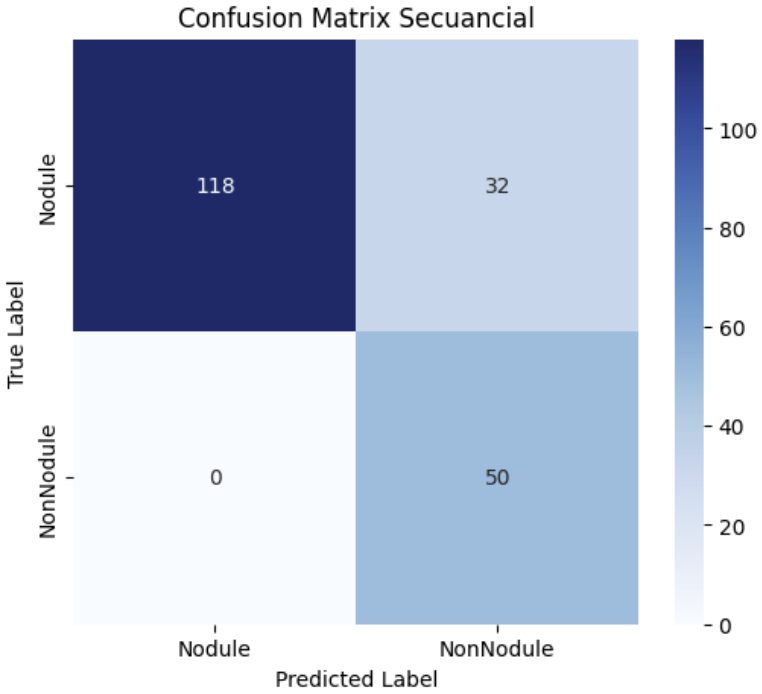

Figura 26: Matriz de confusión del modelo secuencial al set de test

Modelo Xception

El modelo **Xception** (Extreme Inception), disponible en Keras, es una arquitectura de red neuronal profunda basada en la idea de mejorar el modelo Inception de Google, pero reemplazando los módulos de convolución tradicionales por convoluciones separables en profundidad, que descomponen una convolución en dos operaciones. Una convolución por canal (profundidad), seguida de una convolución punto a punto (pointwise). Esto reduce el número de parámetros y mejora la eficiencia del modelo sin sacrificar el rendimiento. Xception ha demostrado ser eficaz para tareas de clasificación de imágenes y es una opción popular para transferir aprendizaje debido a su capacidad para aprender representaciones complejas con menor costo computacional.

Este modelo presenta un diseño con un número de 89 capas de profundidad, con un total de 22'9 millones de parámetros. Al contar con una mayor profundidad esto puede suponer un incremento en el tiempo de ejecución en cada iteración del entrenamiento. Especialmente para esta arquitectura suele tener un tiempo medio mayor a pesar de no contar con tantos parámetros debido a que realiza unas operaciones internas complejas que penalizan con un mayor tiempo de ejecución. Pero siendo como se comentaba más eficiente al emplearlo posteriormente para clasificar.

- **Gráfico de entrenamiento**

Lo que se observa en este gráfico es que la precisión aumenta rápidamente en las primeras épocas y se estabiliza cerca de 1, mostrando que el modelo aprende bien en los datos de entrenamiento y de una manera similar ocurre con los datos de validación. En cuanto a la pérdida de entrenamiento, disminuye rápidamente en las primeras épocas y alcanza valores cercanos a cero, lo que implica que el modelo está minimizando eficazmente el error en los datos de entrenamiento. El **val_loss** también disminuye inicialmente, aunque con ligeras fluctuaciones, pero sigue una tendencia general a la baja, lo que sugiere que el sobreajuste está bajo control. Por lo que se puede dar por un entrenamiento correcto donde parece ajustarse adecuadamente y sin ninguna evidencia a que se sobreajuste.

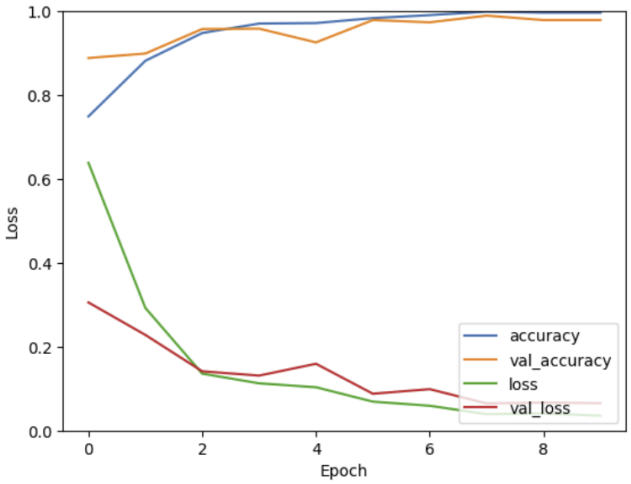

Figura 27: Gráfico de evolución del entrenamiento del modelo Xception

- **Valores de las métricas**

Para este modelo se consigue los siguientes valores para las distintas métricas:

	Precision	**Recall**	**F1-Score**	**Accuracy**
Conjunto A	1.00	1.00	1.00	1.00
Conjunto Test	1.00	0.84	0.91	0.88

Tabla 9: Métricas del modelo Xception

- **Matriz de Confusión**

Para este modelo se consigue también un buen porcentaje de acierto y al igual que en los anteriores el fallo que más comete es para el caso menos crítico.

Matriz sobre el conjunto de test del entrenamiento del modelo

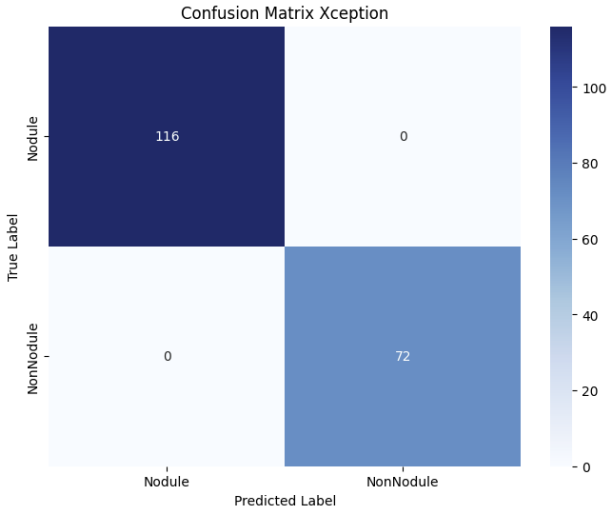

Figura 28: Matriz de confusión del modelo Xception al set de train

Matriz sobre el conjunto de test para la evaluación del modelo

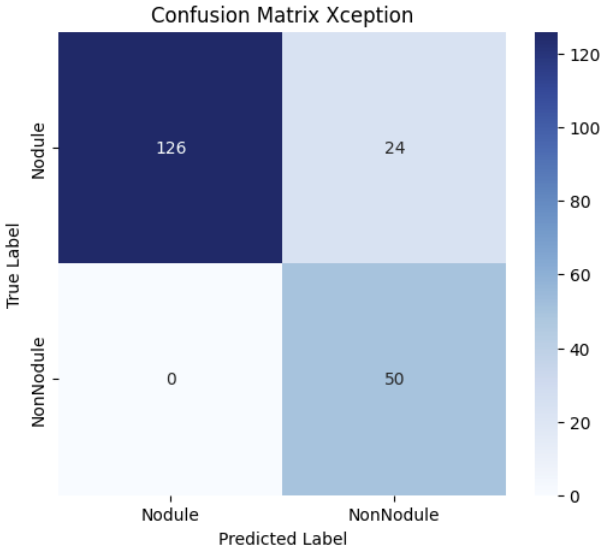

Figura 29: Matriz de confusión del modelo Xception al set de test

Modelo VGG16

El modelo VGG16, también disponible en Keras, es una arquitectura de red neuronal profunda propuesta por el equipo de Visual Geometry Group de la Universidad de Oxford. Su diseño se basa en una estructura simple y uniforme de capas de convolución con filtros de tamaño 3x3, seguidas de capas de pooling, lo que la hace fácil de entender y aplicar. Tiene 16 capas de profundidad (de ahí su nombre) y utiliza un gran número de parámetros, en torno a unos 138'4 millones, lo que contribuye a su capacidad para aprender características complejas. Esto a su vez supone que sea un modelo más pesado en cuanto a su almacenamiento. VGG16 ha sido ampliamente utilizado para tareas de clasificación de imágenes y es una referencia en transferencia de aprendizaje debido a su alto rendimiento y simplicidad arquitectónica.

- **Gráfico de entrenamiento**

Lo que se observa en este gráfico es que el modelo mejora tanto en precisión como en pérdida durante el entrenamiento, y este aprendizaje parece generalizarse bastante bien a los datos de validación, ya que las curvas de validación siguen de cerca las de entrenamiento. También se observa cómo hay un pequeño repunte en la curva de **val_loss** después de la segunda época, esto podría indicar un ligero sobreajuste, pero no parece ser grave dado que la curva vuelve a descender y se estabiliza.

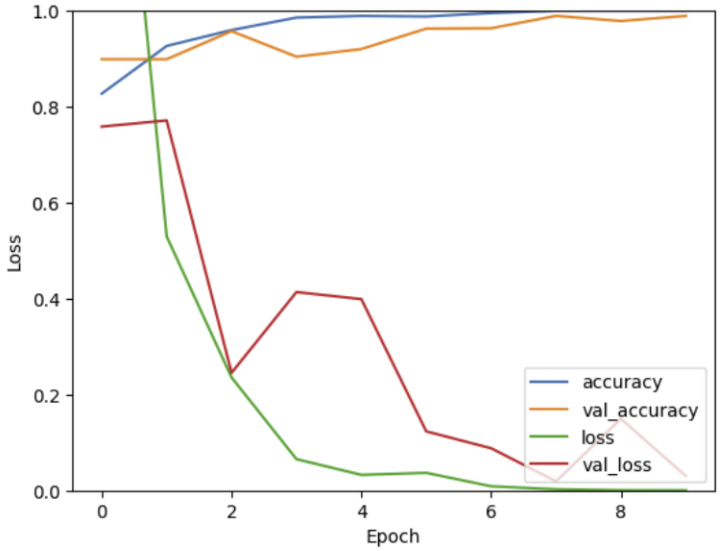

Figura 30: Gráfico de evolución del entrenamiento del modelo VGG16

- **Valores de las métricas**

Para este modelo se consigue los siguientes valores para las distintas métricas:

	Precision	Recall	F1-Score	Accuracy
Conjunto A	0.97	1.00	0.99	0.99
Conjunto Test	1.00	0.95	0.97	0.96

Tabla 10: Métricas del modelo VGG16

- ## Matriz de Confusión

Para este modelo se consigue también un buen porcentaje de acierto y al igual que en los anteriores el fallo que más comete es para el caso menos crítico.

Matriz sobre el conjunto de test del entrenamiento del modelo

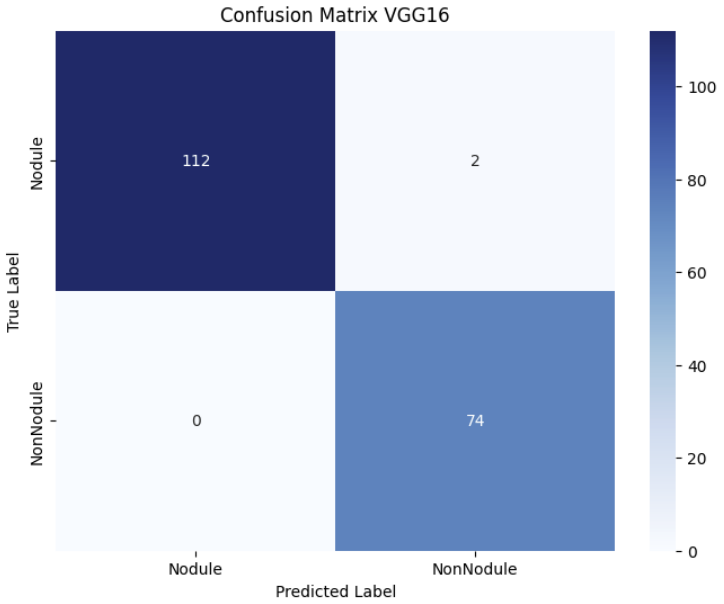

Figura 31: Matriz de confusión del modelo VGG16 al set de train

Matriz sobre el conjunto de test para la evaluación del modelo

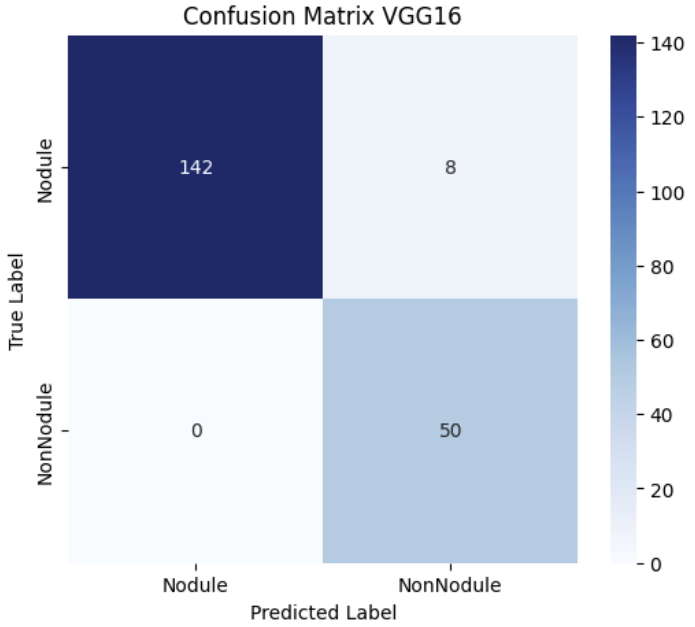

Figura 32: Matriz de confusión del modelo VGG16 al set de test

6.3.4 Grad-CAM

El modelo con el que se ha obtenido unos mejores resultados de clasificación será el que se utilice para aplicar Grad-CAM. (Grad-CAM: A beginner's Guide, s.f.)

Para aplicar esta técnica se ha empleado implementaciones basada en diversos artículos que explican el funcionamiento de esta metodología y dan recursos para experimentar con ella. Principalmente de medios de comunicación centrados en la divulgación científica, que para muchos de los casos se podía acceder directamente al repositorio con una propuesta de la implementación.

La técnica está definida como una clase, la cual define las funciones necesarias para llevar el proceso y dos parámetros necesarios para su ejecución. El parámetro model que hace referencia al modelo de red neuronal al cual se le va a aplicar y last_conv_layer_name que se refiere al nombre de la última capa convolucional del modelo, de la cual se analizará su output para posteriormente generar el mapa de calor. Esta capa no tiene por qué ser forzosamente la última, pero se recomiendo puesto que es la que tiene la implicación más directa con el resultado final que se obtiene por el modelo al hacer la clasificación. Con esto se estaría definiendo en cierta medida un nuevo modelo que devuelve tanto las activaciones de la última capa o aquella asignada y la predicción dada por el modelo.

En cuanto a las funciones que implementa la clase, son tres, que se encargan de crear el mapa de calor y representarlo.

El cálculo del Mapa de Calor, llamado como compute_heatmap, es un método que recibe una imagen en forma de array y opcionalmente un índice de predicción. Hace el cálculo del gradiente, usando una función definida por tensorFlow, de la clase predicha con respecto a la salida de la última capa convolucional. Posteriormente se calcula el mapa de calor al promediar los gradientes a través de las dimensiones espaciales, lo que permite entender qué características de la imagen son más relevantes para la clase predicha. Y el mapa de calor resultante se normaliza para que sus valores estén entre 0 y 1.

La superposición del Mapa de Calor con la Imagen Original, nombrada GetSuperImposedCAMImage. Este método combina el mapa de calor con la imagen original para crear una visualización que resalta las áreas de la imagen que más influyeron en la decisión del modelo. También ajusta tanto la imagen como el mapa de calor para que estén en el mismo rango y tamaño, y se aplican técnicas de visualización (como un mapa de colores) antes de fusionarlos.

La Visualización de Imágenes Superpuestas, definida como DisplaySuperImposedImages. Esta función presenta la imagen original, el mapa de calor y la imagen final superpuesta en un gráfico. Facilitando la comparación visual de cómo el modelo interpreta la imagen en cuestión.

Las imágenes antes de ser empleadas se les aplica, como se mencionó en la fase

de procesamiento de datos, todas las operaciones necesarias para que sea lo más apta posible para el método. Además de que ayuda a que pueda darse tanto una predicción mejor, como a reconocer los patrones de mayor interés.

Con esto definido, ya se puede utilizar sobre cualquier imagen para hallar su mapa de calor y mostrar qué zona es la que el modelo ha dado más importancia para clasificarla.

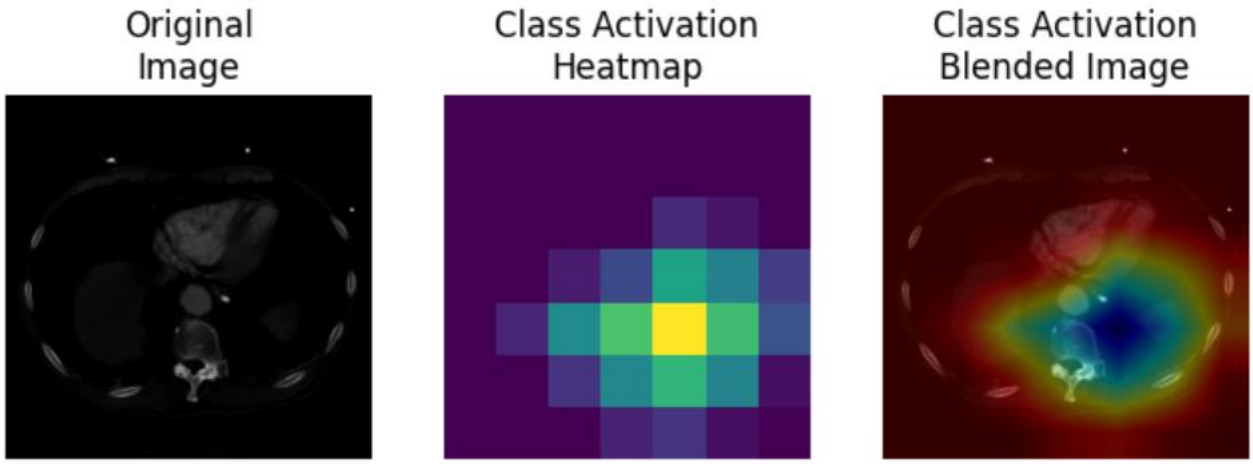

Figura 33: Aplicación de Grad-CAM sobre una imagen

Para comprobar su correcto funcionamiento o si es representativo el resultado que se consigue con esto, se ha aplicado sobre el conjunto de test, pero solo en aquellas imágenes que se proporcionan las anotaciones correspondientes a la localización de los tumores. Para hacer una evaluación y comparación entre las zonas que el modelo usa como relevante para la clasificación o las que están marcadas en las anotaciones.

6.3.4 YOLOv5

Esta técnica la vamos a comparar con lo conseguido previamente de usar un método de clasificación al que se le aplica Grad-CAM. Por lo que se va a utilizar como alternativa directa, para conocer cuál de las dos metodologías se presenta mejor.

El modelo de YOLOv5 es sencillo de implementar, ya que simplemente hay que importar en el proyecto la implementación del mismo, la cual es provista con todos los recursos necesarios para su uso. De hecho, el modelo en sí ya viene entrenado con un amplio conjunto de datos, por lo que podría utilizarse para la detección de objetos. Pero dado que no está preparado para la detección de tumores como se está planteando, es necesario de someterlo a un periodo previo de entrenamiento.

Para poder entrenar el modelo, hay que contar con una serie de requisitos como tener los datos con las anotaciones de localización y el archivo yml. Como se indicó en la fase de datos y que el conjunto B cuenta, por lo tanto, será el empleado para esta parte.

Una vez teniendo esta configuración previa con los datos, se puede realizar el entrenamiento. Para iniciarlo, simplemente hay que lanzar un comando para que se ejecute un script, el cual es provisto con los recursos que se importan. Para lanzar el comando hay que pasar una serie de parámetros que definen el entrenamiento. Los parámetros con sus respectivos valares empleados son; el número de épocas (32), la dimensión en píxeles de las imágenes (512), el tamaño del batch (16), el conjunto de datos que se va a emplear (ruta al archivo de yml) y los pesos de YOLO que se van a usar como punto de partida (en este caso se ha empleado el modelo más pequeño), ya que, hay entre varias opciones disponibles que dependen de la profundidad o el número de parámetros que tiene el modelo por defecto.

Durante el proceso de entrenamiento se proporcionan una serie de métricas como la Precisión (mAP): Media de precisión promedio, que mide la calidad de las predicciones del modelo, Losses (pérdidas): Las pérdidas de clasificación y de las cajas delimitadoras (cuánto se está equivocando el modelo), Tiempo por Epoch: El tiempo que toma completar una época de entrenamiento y la Confianza (confidence): Indica cuán seguro está el modelo sobre una predicción.

Tras finalizar, se genera un conjunto de resultados. El modelo entrenado con los pesos finales y una serie de gráficas que muestran el aprendizaje como curvas de pérdida o mAP para evaluar su desempeño.

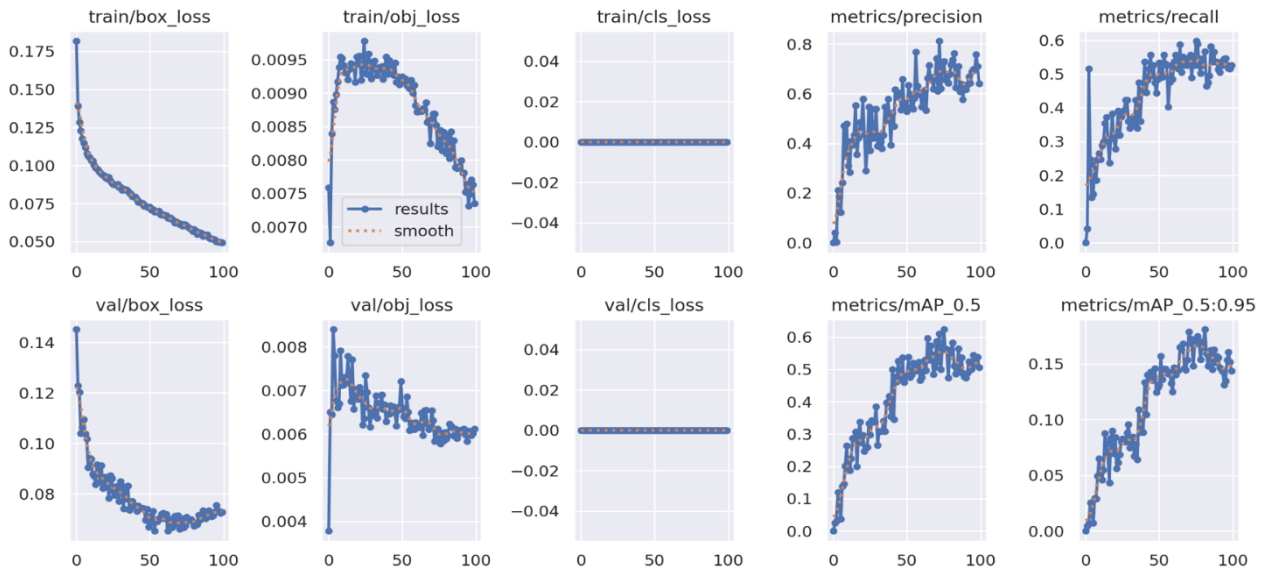

Figura 34: Evaluación de los resultados de evaluación de YOLO

Con el modelo empleando los nuevos pesos a partir del entrenamiento, se hace una prueba sobre el conjunto de datos de test. Para ello se procede a comprobar a que haga la correspondiente detección de todos los objetos que sea capaz de encontrar sobre todas las imágenes del conjunto y posteriormente poderlo evaluar. Hacer la detección, al igual que el proceso de entrenamiento, se ejecuta

lanzando un comando también proporcionado e indicando una serie de parámetros que consiste simplemente en la dirección del conjunto al cual hacer la detección, la dimensión de las imágenes y para este caso que incluya en un fichero de texto por cada imagen de entrada las ubicaciones correspondientes con cada detección. Tras ejecutar esto, se obtiene una copia de todas las imágenes, pero marcando los objetos e indicando a que clase pertenecen, en aquellas imágenes donde haya detectado alguno.

Figura 35: Imagen del conjunto test

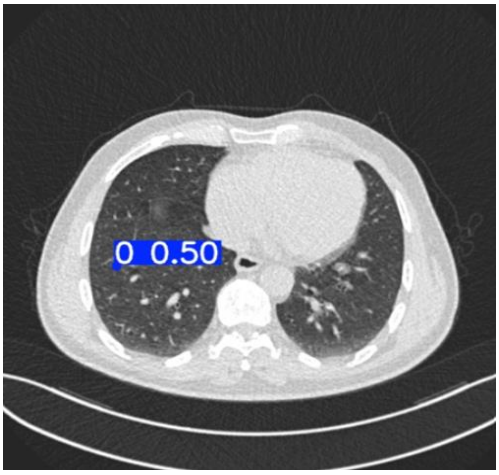

Figura 36: Imagen del conjunto de test con el resultado de la detección de YOLO

A partir de estos resultados, se puede comprobar con las clases reales, calcular las métricas de evaluación y representar la matriz de confusión. Para ello se ha tomado en que toda imagen a la cual no se ha marcado ningún objeto, se toma clasificada como que no tiene presencia de nódulos y por el contrario toda aquella en la que, si haya marcado algún objeto, clasificada como positiva la presencia de nódulos.

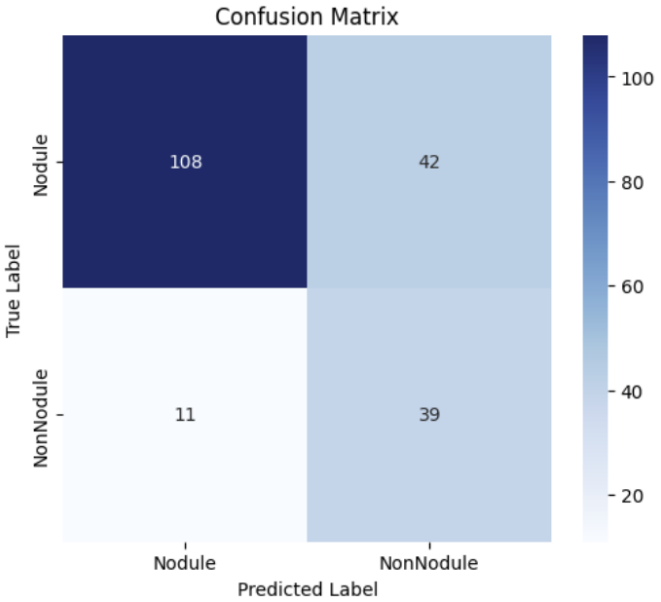

Figura 35: Matriz de confusión del conjunto de test con el modelo de YOLO

6.3.5 Evaluación, comparación de resultados y discusión

Tras lo obtenido por las fases anteriores, se va a analizar los resultados de cada una de las partes y hacer una comparación entre ellas.

Clasificando, el modelo con mejores resultados es VGG16 y con peores el de YOLO si observa lo obtenido por las matrices de confusión. Dado que este último está diseñado para la localización de objetos en una imagen, lo cual es de gran relevancia para lo que se está tratando, se va a comparar este con lo que se obtenga tras aplicar Grad-CAM a uno de los modelos CNNs, para conocer cuál en cierta medida encuentra mejor la zona donde se ubican los tumores. Para no hacer una comparación entre tantos resultados, primero se va a hacer un primer acercamiento entre a qué modelo es más interesante aplicarle esta técnica, para posteriormente ser el empleado en la comparación.

Grad-CAM se ha aplicado a los modelos de Xception y VGG16, ya que dentro de los modelos CNNs, son los que consiguen unos mejores resultados de clasificación.

Para decidir con cuál de los modelos optar a usar, se han hecho varias pruebas hasta determinar cuál parece ser más interesante. En concreto se ha probado con respecto a una serie de imágenes pertenecientes al conjunto de test. Inicialmente se comprobó que, al aplicarse en capas distintas del mismo modelo, los resultados cambian completamente, por lo que para ser más exhaustivos y llegar a una aplicación concreta, se optó por emplear siempre la última capa. Ya que, como se mencionó, esta es la capa más representativa a al llegar a tomar la predicción el modelo, mostrando aquellas zonas que han tenido una mayor relevancia.

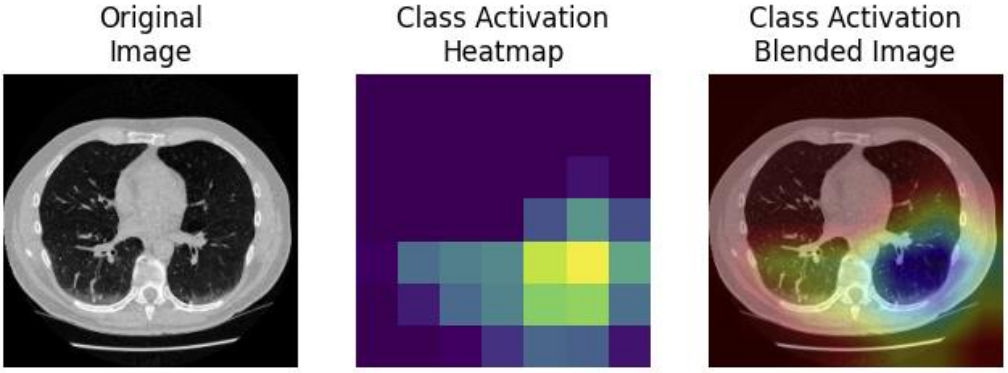

Figura 37: Resultado de aplicar Grad-CAM a Xception

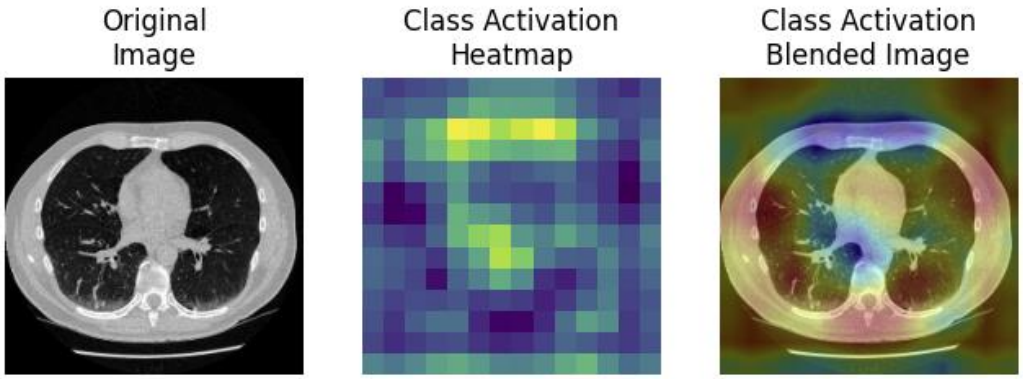

Figura 36: Resultado de aplicar Grad-CAM a VGG16

Las imágenes son un caso en concreto de todas las pruebas realizadas, pero que representa en gran medida el promedio de los resultados que se alcanza al aplicar Grad-CAM a cada uno de los dos modelos. Como se puede observar en el modelo de VGG16 queda una zona no tan claramente delimitada y marcando un área mayor siendo algo más difuminado y confuso comprender la solución. En contra parte el modelo Xception a nivel general define un área más específica como zona de real interés. Debido a esto se concluye a partir de aquí emplear Grad-CAM con Xception para el resto de la comparación.

Para de algún modo evaluar la precisión con la cual los objetos son detectados en la imagen, se va a calcular la intersección entre las áreas marcadas por los resultados reales con respecto a los obtenidos.

La Intersección sobre Unión (IoU) es una métrica utilizada para evaluar la precisión de los modelos de detección de objetos, como el algoritmo YOLO. IoU mide la superposición entre dos áreas: la caja delimitadora predicha por el modelo y la caja real o "ground truth" que representa la ubicación correcta del objeto. El cálculo se realiza dividiendo el área de intersección (donde ambas cajas se solapan) por el área de unión (la suma de las áreas de ambas cajas, menos la intersección). El valor de IoU varía entre 0 y 1: cuanto más cercano a 1, mejor es la predicción. Se usa para decidir si una detección es correcta (por ejemplo, si IoU > 0.5 se considera que el objeto ha sido detectado correctamente). Con este cálculo se puede medir el rendimiento que se tiene por los modelos al localizar los objetos en las imágenes, pudiendo evaluarlos y compararlos entre sí.

Este cálculo de la intersección se realiza sobre las imágenes pertenecientes al conjunto de datos, pero solo en aquellas que tienen anotaciones de las localizaciones para poder hacer el cálculo. De forma que, el resultado de la evaluación será la media del valor obtenido por la intersección de todas las imágenes.

Para ser un poco más flexibles y con la idea de conseguir unos resultados más favorables, se ha optado por ser lo menos estrictos posibles en cuanto al valor de la intersección para ambos casos. A modo que, cuando ambas áreas tengan, aunque sea un mínimo punto en común, se dé como localizado, dando a esa detección el valor máximo de 1. Si por el contrario no se produce ninguna intersección, se da como valor mínimo de 0. Además, para el caso de Grad-CAM, se ha optado por usar la mayor parte del área caliente marcada, a modo que sea más amplia el área a la que hacer la intersección. Por lo que, aquellas zonas más exteriores que delimitan el área las cuales tienen unos valores más bajos, ya que son zonas con un peso menor en cuanta a la predicción, también son utilizadas para el cálculo de la intersección. Esto implica, que en el caso de que el punto a localizar tocase en cierta medida esta zona, se diese como localizado el objeto.

Teniendo esto en cuenta, se ha conseguido los siguientes resultados; para YOLO un 36% de acierto, mientras que para Grad-CAM aplicado a Xception un 20%.

Para ambos casos el porcentaje no llega a ser muy significativo por lo que no se puede dar como mínimo para garantizar cierta confianza a depender de ellos. Pero este resultado puede deberse principalmente a que los nódulos que se pretende localizar son de unas dimensiones muy pequeñas. Esto hace que dificulte mucho la labor tanto para técnicas como YOLO que, aunque es un algoritmo muy preparado para conseguir unos buenos resultados y de una manera eficaz, no es tan capaz a la hora de hacer una detección de objetos de estas características. Y en cuanto a Grad-CAM ocurre algo similar, resulta difícil conseguir que la red logre encontrar un patrón en el cual pueda localizar medianamente a estos objetos. Ya que, al ser tan ínfimamente pequeños dentro de la porción de la imagen, se presentan casi como algo imperceptible.

Planificación del proyecto

8 PLANIFICACIÓN TEMPORAL INICIAL

8.1 DIVISIÓN DE LA PLANIFICACIÓN

Fase 1: Planificación, estudio, investigación

Consiste en conocer información sobre el ámbito del proyecto tanto a nivel de los conceptos, requisitos para su implementación…

Fecha Inicio: 01.07.2024 Fecha Fin: 14.07.2024

A01. Estudio sobre el tema y entorno del proyecto.

- T0: Hacer una investigación de la temática en la que el proyecto está destinado.
- T1: Comprender los conceptos, definiciones y objetivos a los que va a tratar el proyecto.

A02. Creación y estructuración de la memoria.

- T0: Crear la documentación con su respectiva organización de las distintas partes de las que va a constar y con los materiales necesarios a emplear.

A03. Requisitos y necesidades del proyecto.

- T0: Comprobar las posibilidades en cuanto a que entorno de desarrollo puede ser el mejor a emplear.
- T1: Defender cual es el mejor lenguaje de programación a usar para los objetivos que se pretende alcanzar.
- T2: Estudiar todas las librerías, el entorno de desarrollo, frameworks, apis a poder necesitar durante el desarrollo.

Fase 2: Inicio del proyecto

Lo que se pretende en esta fase es que tras su finalización se tenga todo lo necesario para iniciar con la implementación del desarrollo del proyecto, teniendo todo lo requerido tanto a nivel de instalación y configuración de las herramientas a emplear.

Fecha Inicio: 15.07.2024 Fecha Fin: 31.07.2024

A04: Instalación y configuración de todas las herramientas

- T0: Instalar a nivel local los entornos de desarrollo, el lenguaje de programación a usar, las librerías necesarias, etc.
- T1: Preparar a nivel cloud el entorno para hacer uso de el a nivel paralelo al entorno local. Entendiendo las distintas partes de las que consta y cómo hacer uso de ellas
- T2: Configurar las herramientas de la tarea anterior para que funcionen de manera correcta conjuntamente.
- T3: Entender el funcionamiento y cómo hacer uso en conjunto de ellas para llevar a cabo el desarrollo

A05: Inicio de la descripción del proyecto en la memoria

- T0: Documentar en base a toda la información recolectada realizada en la fase anterior e iniciar a argumentarla en la memoria con la estructura planteada.
- T1: Definir aquellas partes correspondientes a la que plantean las bases por las que se lleva a cabo la realización del proyecto.

A06: Comprobar la validez de los datos a usar para las pruebas

- T0: Buscar distintas bases de datos sobre imágenes de algún tumor y tengan anotaciones extras de las localizaciones.
- T1: Verificar que la base de datos está compuesta con las características necesarias y que los datos sean fiables y válidos para el sistema.

A07: Realizar la planificación del proyecto

- T0: Crear un itinerario temporal de la estructuración del proyecto que se pretende seguir.
- T1: Crear una planificación de los costos

Fase 3: Implementación y pruebas

En esta etapa se inicia a realizar diversas implementaciones de la solución que se propone, donde en cada una se irá probando con distintas modificaciones de las distintas partes de las que consta el desarrollo. Con el fin de hacer un recolecta de resultados a partir de cada solución obtenida.

Fecha Inicio: 01.08.2024 Fecha Fin: 30.09.2024

A08: Prueba inicial de la primera implementación.

- T0: Desarrollar un modo para manejar los datos que será usado a lo largo de todas las pruebas.

- T1: Decidir el modo de procesar los datos para que sean adecuados con el

sistema que se implementa.

- T2: Usar una de entre las redes neuronales para comprobar los resultados q ue se obtienen de la misma.
- T3: Aplicar las técnicas de explicabilidad sobre los datos de prueba

A09: Empezar a realizar distintas modificaciones y pruebas para obtener distintos resultados.

- T0: A partir de la implementación inicial, hacer distintas pruebas modificando el modo de procesamiento de las imágenes o cambiando de conjunto de datos.
- T1: Probar también con distintas implementaciones de diversos modelos de redes neuronales.
- T2: Modificar del modo de aplicarse las metodologías explicabilidad, configurando algunos aspectos de su funcionalidad

A10: Seguir documentando las partes de desarrollo del proyecto

- T0: Dar un esquema general del diseño del proyecto.
- T1: Explicar en profundidad las herramientas usadas.
- T2: Exponer como se efectúa el desarrollo.

Fase 4: Análisis

Tras los resultados obtenidos en la fase anterior, se va a estudiar y analizar cada uno de los alcanzados por las distintas soluciones. Donde se pretende evaluar la validez y fiabilidad que tendría el sistema final aplicando dicha implementación.

Fecha Inicio: 01.10.2024 Fecha Fin: 14.10.2022

A11: Analizar los resultados que se van obteniendo a lo largo de las distintas pruebas.

- T0: Recoger todas las versiones de las distintas implementaciones dadas con sus respectivos resultados obtenidos.
- T1: Crear un gráfico con la mejora que se ha obtenido por cada iteración de las etapas de la red para cada solución.

A12: Comprobar y revisar lo obtenido de los resultados de las pruebas

- T0: Estudiar cada resultado y ver el porcentaje de acierto que se ha conseguido.
- T1: Hacer una comparación de cada implementación con su evaluación.
- T2: Analizar cuál de todas las implementaciones ofrece una mejor solución.

Fase 5: Revisión, corrección y exposición final

Para concluir se hará la correspondiente documentación de la fase anterior y se revisará toda la presentación de la memoria a entregar, añadiendo las partes a

faltar, corrigiendo cualquier posible error y estilizando en la medida de lo posible hasta tener la versión final.

Fecha Inicio: 15.10.2024 Fecha Fin: 29.10.2024

A13: Exponer en la memoria los resultados de la investigación obtenidos tras todas las pruebas.

- T0: Importar a la documentación gráficos y tablas mostrando una comparación precisa y legible para una compresión fácil de las diferencias entre cada solución.
- T1: Explicar de forma argumentada como ha sido desarrollada cada una de las implementaciones y defender el resultado que se obtiene a partir de ella.

A14: Realizar una corrección y revisión de toda la memoria.

- T0: Observar que concuerdan los índices, las imágenes, las tablas, etc.
- T1: Comprobar que es completo y correcto cada uno de los distintos apartados.
- T2: Corregir alguna cosa en caso de encontrar algún fallo, incoherencia o mala explicación.
- T3: Realizar las modificaciones que fuesen necesaria para estar en el formato correcto antes de la entrega.

8.2 Diagrama de Gantt

Figura 38: Diagrama de Gantt de planificación inicial

9 PLANIFICACIÓN TEMPORAL FINAL

Con respecto a la planificación final, se compone de las mismas fases y tareas que las descritas en la planificación inicial, pero habiendo sufrido algunos cambios en cuanto a los tiempos finalmente dedicados para cada una de ellas con respecto a la que se pretendía realizar, debido a complicaciones surgidas a lo largo del desarrollo en algunas de las tareas a realizar, pero acabando y empezando sobre la fecha marcada. Entre algunas de estas tareas han sido las relacionadas con la fase 3 de implementación y con la fase 4 de análisis, ya que ha conllevado que estas se tuviesen que repetir de manera iterativa en varias ocasiones. Gracias a hacer una mayor distribución de las horas donde conseguir antes de lo esperado algunas de las actividades, compensando aquellas que han supuesto la dedicación de más tiempo debido esos fallos o dificultades encontradas durante su desarrollo. También algunas de las tareas finalizadas en ocasiones ha sido necesaria repetirlas de nuevo porque el resultado no era el esperado o por el hecho de hacer una revisión o mejora.

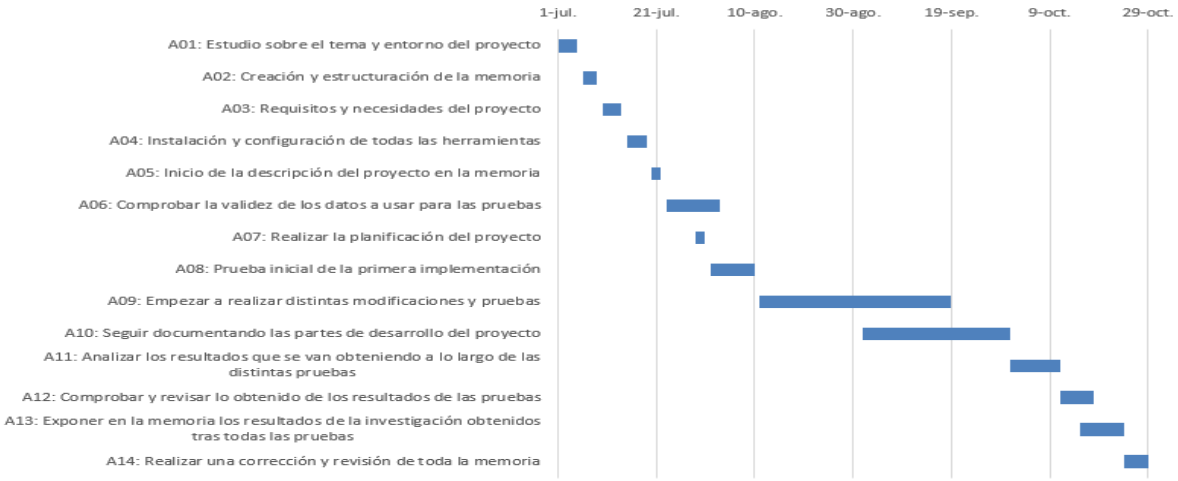

Figura 39: Diagrama de Gantt de planificación final

10 PLANIFICACIÓN DE COSTES INICIALES

Las tablas siguientes muestran las estimaciones de los costes a lo largo del desarrollo, los cuales pueden cambiar a lo largo del tiempo difiriendo con el coste del prototipo final.

Fase	Actividad	Coste	Fecha
Fase 1	A01 – A02 – A03	16%	01/07/2024
Fase 2	A04 – A05 – A06 – A07	18%	15/07/2024
Fase 3	A08 – A09 – A10	45%	01/08/2024
Fase 4	A11 – A12	10%	01/10/2024
Fase 5	A13 – A14	10%	15/10/2024

Tabla 11: Distribución de los costes iniciales

Se ha evaluado los costes asociados que se puede tener en cuanto a materiales para llevar a cabo el proceso de desarrollo. Siendo una estimación de lo que pueden suponer los mismos. Los costes proceden desde material necesario como el portátil personal, hasta herramientas tecnológicas necesarias como podría ser la versión pro de Google Colab. O el consumo energético, que podría hacerse una estimación de un consumo de 30kW al mes, provenientes de un uso de 0,25 Kw/hora y estando unas 120 horas al mes, y con un coste promedio de 0'12€ el kW.

Producto	Precio	Cantidad	Subtotal
Esquipo personal	800 €	1	800 €
Google Colab Pro	12 € al mes	4	48 €
Gasto Energético	3'6 € al mes	4	14'4 €

Tabla 12: Gastos en materiales iniciales

Gastos en personal

El desarrollo del prototipo se conforma por un equipo formado por un único integrante, el cual a lo largo del proyecto hará la función de distintos roles.

- **Jefe de proyecto**: Coordina las tareas y fases del proyecto, siendo el encargado de cumplir con la mayor fiabilidad posible, que se siga el itinerario marcado por la planificación inicial. Haciendo también, en la mayor medida de lo viable, que los gastos se ajusten lo máximo posible o en tal caso se disminuyan de lo estimado.

- **Ingeniero (Desarrollador)**: Realiza las implementaciones a las distintas soluciones que se hacen a lo largo del proyecto en todas las

partes que lo constituye.

- **Científico investigador:** Recopila y realiza investigaciones para ampliar el conocimiento de las distintas áreas de interés para la elaboración del proyecto. Formulando hipótesis, probarlas a través de diversos medios, dar estadísticas y datos, y plantear conclusiones basadas en la evidencia para la solución alcanzada.

- **Investigador del producto (Marketing)**: Es el encargado en hacer un estudio sobre la involucración del producto dentro de entorno, analizando a su vez lo que existe similar en el mercado, el público al que va dirigido, etc.

- **Analista**: Se encarga de evaluar los resultados obtenidos tras los diversos desarrollos de la solución, así como determinar cuál es la más interesante para el producto final.

La estimación del coste se calcula haciendo un promedio del sueldo considerado de cada uno de los diferentes perfiles mencionados. Asumiendo como gasto, la duración total que ha conllevado la finalización del proyecto.

La siguiente tabla muestra un salario medio de cada uno de estos perfiles, los cuales son basados en estimaciones medias para estos profesionales en España. A partir de estos datos se va a realizar el cálculo promedio del coste salarial implicado en el proyecto.

Empleado	**Jefe de proyecto**	**Desarrollador ingeniero**	**Investigador científico**	**Investigador de mercado**	**Analista**
Salario medio/ mensual Bruto	3.600€	2.970€	2.450€	2.880€	3.360€
Salario medio/ mensual Neto	3.000€	2.500€	2.100€	2.400€	2.800€

Tabla 12: Precio medio por profesión

A partir de estos datos, podemos estimar que el salario medio para el profesional en la realización del proyecto en sí, es de unos 2.560€ netos. En bruto en torno a unos 3.100€.

Por lo tanto, estimando que la media de horas de trabajo de un mes es de unas 160 horas. Podemos calcular a partir del salario anterior, que el salario por hora sería de unos 16€ netos y 19’32€ brutos.

Suponiendo que la duración ha sido alrededor de unos 4 meses, de los cuales invertidas unas 120 horas en cada uno de ellos. Se obtiene como horas totales empleadas en la finalización del proyecto, 480 horas.

Calculando a través del cómputo de números de horas totales invertidas en la realización del proyecto, por el precio medio a la hora, sería un total de:

480 horas * 19'32€ = 9273'6€ Brutos

480 horas * 16€ = 7.680€ Netos

Así como gasto total predecible para el desarrollo del proyecto sería del coste personal calculado anterior, sumado a los gastos en materiales que darían un total de

7.680€ + 60€ = 7.740€

10 PLANIFICACIÓN DE COSTES FINAL

Tras la ejecución del proyecto se ha visto ligeramente modificado el precio final del desarrollo. Se ha conseguido reducir levemente el precio final de la realización, debido a que no se ha invertido en ningún tipo de gasto en materiales y el tiempo de desarrollo se ha visto ligeramente reducido. El tiempo finalmente empleado para finalizar el proyecto, ha sido de 460 horas siendo 20 menos de las que estaban predichas. Obteniendo así, como gasto personal real del proyecto 460 horas * 19'32€ = 8.887'2€. Siendo este el gasto final.

11 ESTUDIO DE MERCADO

Se va a hacer un estudio completo del mercado para saber cuál sería el público más adecuado para promocionar el producto y como de bien se desenvolvería en él.

Clientes potenciales

Los principales clientes interesados, serían los centros médicos como; hospitales, clínicas y centros de investigación oncológica, tanto públicos como privados. Debido a que, estas instituciones buscan continuamente mejorar sus métodos de diagnóstico, y herramientas avanzadas pueden ayudar a obtener diagnósticos más precisos y en etapas tempranas, aumentando las probabilidades de efectuar un tratamiento exitoso. Además, estas herramientas pueden reducir los costos a largo plazo, lo cual es fundamental para aquellas entidades que gestionan un alto volumen de pacientes y requieren de una optimización de recursos.

También a farmacéuticas y empresas de biotecnología que realizan investigación para desarrollar tratamientos de cáncer. Este sector suele estar interesado en nuevas tecnologías que les permitan mejorar sus ensayos clínicos. Por otro lado, las compañías de seguros de salud, ya que podrían beneficiarse de mejores diagnósticos preventivos. Una detección temprana y precisa reduciría significativamente los costos asociados a tratamientos prolongados o intervenciones tardías. Las aseguradoras tienen un gran interés económico en herramientas que puedan ayudar a identificar enfermedades en etapas tempranas, y algunas incluso incentivan a sus usuarios a hacerse pruebas preventivas, pues esto reduce la siniestralidad y el gasto general en salud.

Otro sector para considerar sería, a los laboratorios de diagnóstico y a las empresas que realizan pruebas de laboratorio. Estas organizaciones, que también incluyen a empresas emergentes en el área de salud digital y diagnóstico, buscan herramientas innovadoras que les den ventaja competitiva y les permitan ofrecer pruebas de detección más rápidas, precisas o económicas.

Como se mencionó al inicio de este trabajo sobre el impacto del cáncer en la sociedad. Se estima que alrededor del 20% de la población en algún momento será diagnosticada con cáncer, dependiendo de la región y la edad promedio de la población.

De este modo si vemos la implicación que puede llegar a tener esto, en cuanto a posibles pacientes potenciales para cada uno de los centros descriptos. Podemos visualizar en qué porcentaje afectaría a cada uno de los mismos.

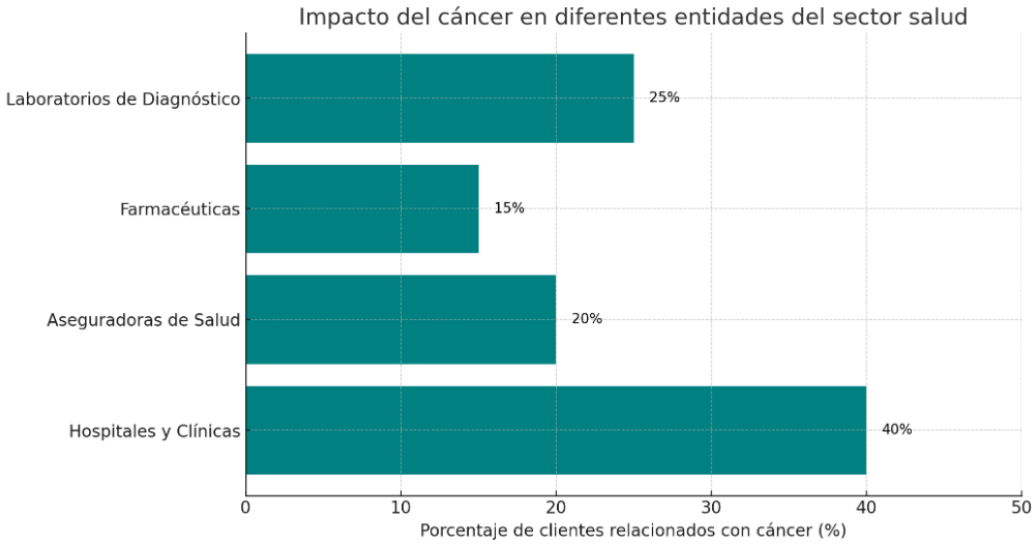

Figura 40: Impacto del cáncer por porcentaje de pacientes distribuidos en cada entidad del sector sanitario

Como se puede observar, el cáncer impacta de manera significativa en estas organizaciones, lo cual subraya la necesidad de contar con herramientas avanzadas de diagnóstico. (Datos del Cáncer OMS, s.f.)

Plan de comercialización

Tras ya conocer a los clientes a los que está destinado el producto, se va a diseñar un plan de amortización de este.

El producto para comercializar se propone como la venta de un servicio. Consistiría en tener la implementación del sistema propuesto alojado en la nube. La compra del servicio se pondría a modo de suscripción, con distintos planes. Estos planes se diferenciarían en función de la cantidad de análisis que te ofrece cada uno, pudiendo ser una distribución de la misma del siguiente modo; plan básico que te ofrece hacer 100 análisis, el cual puede ser conveniente para pequeños centros sanitarios como ambulatorios, un plan medio que ofrece 1.000 análisis, perfecto para centros sanitarios con una mayor atención a pacientes, como podría ser un pequeño hospital y, por último, un plan premium que ofreciese 100.000 análisis, pensado para aquellos hospitales o ciudades sanitarias con grandes capacidades que alberga la atención de un gran número de pacientes.

Realizándose la renovación o modificación, conforme el centro requiera de una mayor o menor cantidad de muestras.

El precio podría ser para cada una de las suscripciones; 500€ para el plan básico, 4.000€ para el plan medio y 200.000€ para el plan premium.

Conclusiones

Con este trabajo de fin de máster se ha dado una visual a metodologías con las que dar una explicación o interpretación a los resultados que se obtienen tras hacer una detección de la presencia de un tumor en una imagen, usando para ello algún modelo de redes neuronales convolucionales.

A la resolución alcanzada tras el análisis de los resultados que se han obtenido, se puede ver como estas técnicas pueden ser útiles, pero no alcanzan un nivel mínimo necesario para que pudiesen ser empleadas en un entorno real. Aunque se ve que parten con unas bases sólidas que hace que puedan plantearse como interesantes, ya que, para el caso en el que los tumores a localizar apareciesen en la imagen con una mayor dimensión podría suponer un mejor rendimiento en cuanto a su localización. Asimismo, visto que para clasificar el mejor modelo es VGG16 pero, sin embargo, para detectar objetos resulta ser más preciso emplear YOLO. Emplear ambas técnicas en conjunto podría suponer en crear un sistema más robusto y con el cual obtener una precisión mayor. Siendo esta como propuesta a la solución óptima que se puede implementar dentro de las opciones experimentadas.

A nivel general se puede estar satisfecho con desarrollo que se ha conseguido en el proyecto. Ya que, se puede decir que se han conseguido cumplir todos los objetivos que estaban dispuestos a satisfacerse con la finalización de este. Además, en gran medida ha sido posible ceñirse a la planificación del proyecto dada, a pesar de haber tenido que hacer algunas modificaciones debido al retraso de la ejecución de algunas fases por ciertos imprevistos encontradas a lo largo del desarrollo. En cuanto al coste planteado, también ha sido posible llevarlo a cabo sin añadir costes extras por motivos inesperados.

Tras la realización del trabajo se ha puesto en práctica muchos de los conocimientos que han sido adquiridos a lo largo de los estudios del máster. El desarrollo de este trabajo aúna conocimientos de muchas disciplinas distintas, además de servir como un acercamiento a un proyecto de investigación. Desde el punto de vista académico, ha servido para aplicar dichos conocimientos adquiridos, desde; el proceso de planificación y gestión de un proyecto, conceptos relacionados con materias vistas de algoritmia, procesamiento de imágenes, inteligencia artificial, interpretabilidad, etc.

La realización de este trabajo ha estado muy motivada por la capacidad de poder aplicar los conocimientos aprendidos, para poder resolver una problemática que ayuda a la mejora de la salud de las personas. Y en especial, a conseguir de alguna manera romper la barrera de confianza al uso de sistemas de inteligencia artificial, lo cual es unos de los mayores retos actualmente y en especial en un sector que su implicación es tan crítica como la salud. Así con esta motivación, ha sido posible dar forma a todos estos conceptos adquiridos para poder ser usados en la creación de un primer prototipo de la construcción de un sistema que podría resultar de una gran utilidad en el futuro en el ámbito sanitario.

Trabajo Futuro

Como propuesta a futuro y con el fin de la posibilidad de llevar lo presentado con este trabajo a la creación de un producto real en el mercado. Sería conveniente en profundizar y mejorar ciertos aspectos a partir de lo implementado y mencionado durante este proyecto.

Uno de estos aspectos sería la posibilidad de conseguir crear un modelo con mejores resultados. Con el que obtener una mayor fiabilidad de predicción, ya que, es muy importante que el error sea lo mínimo posible debido a que estos fallos, mencionados al inicio de este trabajo, suponen de un riesgo importante para la salud de las personas. Para poder mejorar esto sería conveniente de contar con un equipo más potentes capaz de ejecutar modelos más complejos, pero especialmente contar con una gran cantidad de datos para poder detectar una mayor variedad de casos diversos y que dichas imágenes contasen con nódulos de unas dimensiones algo mayores u que estuviesen todas ellas con las anotaciones adecuadamente. Así obteniendo un modelo más completo y con un reconocimiento de patrones más generalizado. De este modo que los modelos fuesen aplicados para cualquier tipo de cáncer y no solo en el de pulmón al que se ha usado para la experimentación.

Como otra metodología interesante que podría suponer una mejora con respecto a los resultados que se han logrado, consistiría en tratar las imágenes que se emplean tanto para el entrenamiento de los modelos como para posteriormente las técnicas de explicabilidad. A modo de que las imágenes fuesen troceadas en porciones de iguales dimensiones, de forma que, por ejemplo, cada imagen original se subdividiese en otras 16 imágenes u otra cantidad, las cuales todas con la misma altura y anchura. Así pues, se conseguiría ampliar el conjunto de datos y en especial, aquellas subimágenes con presencia de nódulos, aparecería con un tamaño mayor. Esto podría ayudar a los modelos a definir de un modo más claro los patrones que corresponden a la presencia de un nódulo. Pudiendo posteriormente obtener tanto una mejor predicción, como una mayor detección.

Otro de los puntos importantes sería crear un sistema, ya sea a modo de servicio como se expone en el plan de comercialización, el cual estuviese integrado en el sistema sanitario con el que los médicos pudiesen usarlo. Para que, a partir de una imagen de algún paciente, el sistema mostrase como resultado la presencia de un tumor, o, en caso contrario etiquetarlo como zona limpia. Además, que este sistema utilizase estas nuevas muestras para almacenarlas en una nueva base de datos con la que seguir mejorando el modelo y también puede ser útil para tener un registro que el médico pudiese usar a modo de consulta de los análisis de un paciente. A parte del sistema anterior, a modo independiente o como complemento de este último. Diseñar otro sistema, pero hardware, que estuviese integrado en los servicios sanitarios del médico especialista para que, mostrando una imagen determinada, a partir de ella, mostrase la predicción directamente a través de una interfaz gráfica como un display o por medio de una web o aplicación móvil, mostrando los resultados alcanzados.

Bibliografía

A Review of Different Interpretation Methods (Part 1: Saliency Map, CAM, Grad-CAM). (s.f.). Obtenido de A Review of Different Interpretation Methods (Part 1: Saliency Map, CAM, Grad-CAM): https://mrsalehi.medium.com/a-review-of-different-interpretation-methods-in-deep-learning-part-1-saliency-map-cam-grad-cam-3a34476bc24d

Beneficios XAI. (s.f.). Obtenido de Beneficios XAI: https://medium.com/@isrfoundation89/explainable-ai-xai-7f2710cb58f3

Datos del Cáncer OMS. (s.f.). Obtenido de Datos del Cáncer OMS: https://www.who.int/es/news-room/fact-sheets/detail/cancer

evaluate the performance of a ml model. (s.f.). Obtenido de evaluate the performance of a ml model: https://medium.com/analytics-vidhya/confusion-matrix-accuracy-precision-recall-f1-score-ade299cf63cd

Explainable AI: A Comprehensive Review of the Main Methods. (s.f.). Obtenido de Explainable AI: A Comprehensive Review of the Main Methods: https://medium.com/@dallanoce.fd/explainable-ai-a-complete-summary-of-the-main-methods-a28f9ab132f7

Explainable Artificial Intelligence. (s.f.). Obtenido de Explainable Artificial Intelligence: https://medium.com/mydatamood/qu%C3%A9-es-xai-e7548fbd9edb

Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. (s.f.). Obtenido de Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks: https://ieeexplore.ieee.org/document/7485869

Glenn Jocher et al. (2020). YOLOv5 by Ultralytics. (s.f.). Obtenido de Glenn Jocher et al. (2020). YOLOv5 by Ultralytics: https://github.com/ultralytics/yolov5

Google Colab. (n.d.). Google Colab Documentation. (s.f.). Obtenido de Google Colab. (n.d.). Google Colab Documentation.: https://colab.research.google.com/notebooks/

Grad-CAM: A beginner's Guide. (s.f.). Obtenido de Grad-CAM: A beginner's Guide: https://medium.com/@bmuskan007/grad-cam-a-beginners-guide-adf68e80f4bb

How to evaluate the performance of a machine learning model? (s.f.). Obtenido de How to evaluate the performance of a machine learning model?: https://medium.com/analytics-vidhya/confusion-matrix-accuracy-precision-recall-f1-score-ade299cf63cd

How to evaluate the performance of a machine learning model? (s.f.). Obtenido de How to evaluate the performance of a machine learning model?: https://medium.com/analytics-vidhya/confusion-matrix-accuracy-precision-recall-f1-score-ade299cf63cd

Keras. (n.d.). Keras Documentation. (s.f.). Obtenido de Keras. (n.d.). Keras Documentation: https://keras.io/

LIME vs. SHAP. (s.f.). Obtenido de LIME vs. SHAP: https://medium.com/@afanta/lime-vs-shap-a92623e95c4

Mayo Clinic. (s.f.). Obtenido de Mayo Clinic: www.mayoclinic.org

neuronal networks. (s.f.). Obtenido de neuronal networks: https://www.ibm.com/ar-es/cloud/learn/neural-networks

Should AI Models Be Explainable? That depends. (s.f.). Obtenido de Should AI Models Be Explainable? That depends.: https://hai.stanford.edu/news/should-ai-models-be-explainable-depends

Sociedad Española de Oncología Médica. (s.f.). Obtenido de Sociedad Española de Oncología Médica: www.seom.org

Sociedad Estadounidense del Cáncer. (s.f.). Obtenido de Sociedad Estadounidense del Cáncer: :www.cancer.org

TensorFlow. (n.d.). TensorFlow Documentation. (s.f.). Obtenido de TensorFlow. (n.d.). TensorFlow Documentation: https://www.tensorflow.org/learn

Ultralytics YOLOv5. (n.d.). YOLOv5 Documentation. (s.f.). Obtenido de Ultralytics YOLOv5. (n.d.). YOLOv5 Documentation: https://docs.ultralytics.com/

Visual Studio Code. (n.d.). Visual Studio Code Documentation. (s.f.). Obtenido de Visual Studio Code. (n.d.). Visual Studio Code Documentation.: https://code.visualstudio.com/docs

XAI in medical image analysis. (s.f.). Obtenido de XAI in medical image analysis:

https://www.sciencedirect.com/science/article/pii/S1361841522001177